

摘 要

本报告详细记录了本小组在综合设计 III 项目中的工作成果，该项目旨在开发一个多数据源融合的互联网个人贷款风控系统。报告分为五个主要部分：复杂工程问题归纳与实施方案可行性研究、针对复杂工程问题的方案设计与实现、测试环境构建与测试实施、知识技能学习情况以及团队协作与分工协作情况。

在复杂问题分析阶段，我们针对项目中的关键复杂问题进行了深入分析。通过对贷款流程的细致分析，我们识别并梳理了关键的功能点和数据流转机制，并总结归纳出三点复杂建模工程问题。

在方案设计和实现阶段，我们注重整体规划，完善了系统的总体架构、数据库设计、用户界面 UI、接口响应以及行为流程等设计。我们详细分析了整个项目在实现过程中的具体要求、重点参数和流程，确保了项目实施的高效性和准确性。随后，详细描述了前后端、机器学习评分卡、坏账预测模型、数据指纹、大模型接入等重点板块的开发实现过程。

在系统测试阶段，我们对系统功能进行了全面的测试与完善。我们首先进行了详实的测试样例编写和测试环境搭建，随后通过相关工具进行人工测试、交互性测试、自动化测试。测试结果显示，系统流程运转和接口响应正常，用户体验性好，性能优良。

在知识技能积累方面，小组成员在项目过程中积极参与学习，不断提升自身技能。通过实践积累了丰富的软件工程开发经验和技能，为项目的顺利进行提供了坚实的支持。在团队协作情况方面，小组成员表现出色，大家积极参与，分工明确，协作顺畅。我们定期召开会议，及时沟通交流，解决问题，确保了项目的高效执行。

通过本次项目，小组成员不仅在技术层面取得了显著成果，还在团队协作和沟通能力上得到了提升。我们相信，这些经验和技能将对我们未来的工作和学习产生积极的影响。

关键词：金融信贷、评分卡模型、坏账预测、风险控制

目 录

摘 要	I
目 录	II
第一章 复杂工程问题归纳与实施方案可行性研究	1
1.1 需求分析与复杂工程问题归纳	1
1.1.1 项目背景概要	1
1.1.2 数据流程分析	2
1.1.3 功能需求分析	4
1.1.4 市场调研	10
1.1.5 技术方案选型	11
1.1.6 核心复杂工程归类与建模	13
1.2 实施方案可行性研究	15
1.2.1 重难问题的实施方案构想	15
1.2.2 可行性分析	18
第二章 针对复杂工程问题的方案设计与实现	20
2.1 针对复杂工程问题的方案设计	20
2.1.1 总体设计	20
2.1.2 功能设计	22
2.1.3 数据库设计	25
2.1.4 UI 设计	29
2.1.5 流程设计	33
2.1.6 行为接口设计	34
2.1.7 信息采集维度设计	40
2.1.8 安全设计	42
2.2 针对完整系统工程的方案实现	43
2.2.1 开发工具、环境与规范概述	43
2.2.2 数据结构构建概述	46
2.2.3 前端框架及关键代码分析	48
2.2.4 后端框架及关键代码分析	60
2.3 针对核心重难工程的方案实施	69
2.3.1 信息采集	69
2.3.2 评分卡模型	72
(一) 数据集预处理	72
(二) 特征工程分箱与 WOE 编码	75
(三) 逻辑回归推演	79
(四) 评分卡转化	83

(五) 模型评估.....	86
2.3.3 坏账预估模型.....	88
2.3.4 多渠道提醒.....	91
2.3.5 智能客服大模型.....	95
2.3.6 数据指纹技术.....	97
第三章 测试环境构建与测试实施.....	101
3.1 测试环境与工具	101
3.2 测试功能点与样例	102
3.3 测试方法与实施	104
3.3.1 人工测试.....	104
3.3.2 交互性测试.....	105
3.3.3 自动化测试.....	107
3.4 测试结果与修复	111
3.4.1 测试结果.....	111
3.4.2 结果分析.....	113
3.4.3 修复情况.....	113
第四章 知识技能学习情况.....	115
4.1 编程语言与技术框架	115
4.2 系统设计与模式架构	116
4.3 问题解决与资源收集	117
4.4 项目管理与产品思维	119
4.5 先进技术与融合创新	120
第五章 分工协作与交流情况	122
5.1 分工协作与计划完成情况	122
5.2 协同交流情况	123
参考文献	127
致谢.....	128

- 说明:**
- 1、报告要求 8000 字以上。
 - 2、本模板仅为基本参考，请各位同学根据个人情况进行目录结构扩展。
 - 3、报告正文必须双面打印。

第一章 复杂工程问题归纳与实施方案可行性研究

1.1 需求分析与复杂工程问题归纳

1.1.1 项目背景概要

近年来，我国经济蓬勃发展，人民生活水平逐渐提高，消费观念也发生了深刻的转变。

在传统观念中，消费者会根据实际需求进行购物，并在自身经济能力的范围内进行消费，以满足生活所需。然而，随着社会经济的持续增长、消费者购买力的逐步提升以及网络购物便利性的增强，“预支消费”已逐渐成为一种新的消费趋势。这意味着消费者在购物时超越了实际需求，倾向于将未来的消费提前到现在。

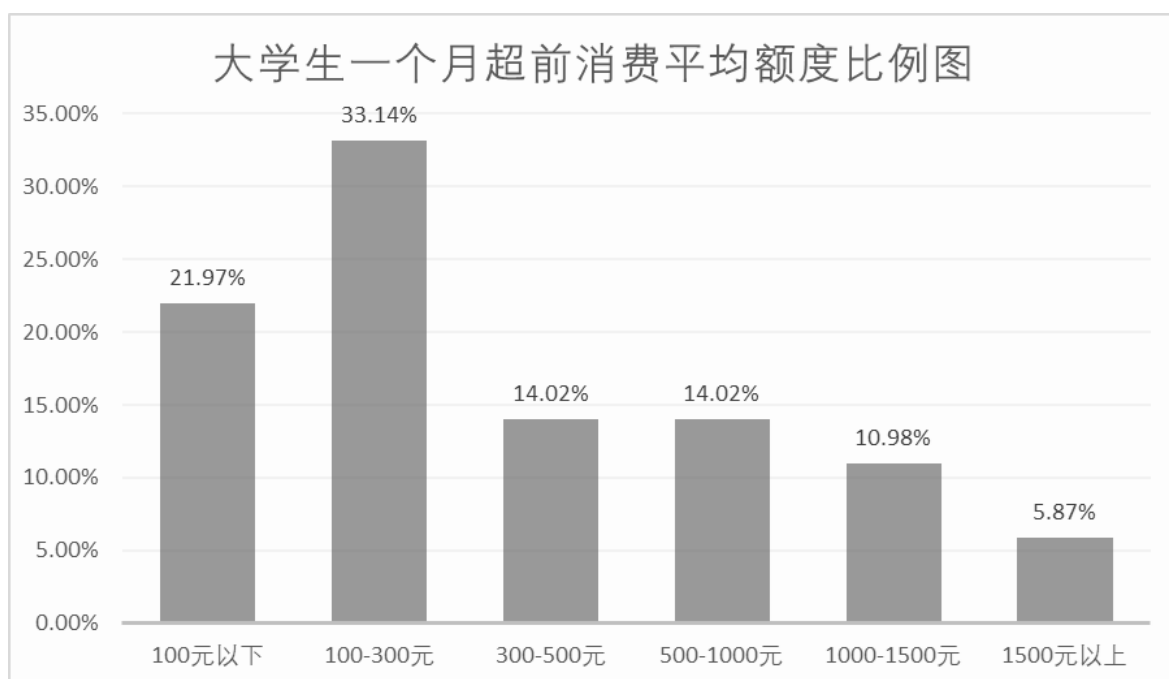


图 1：大学生一个月超前消费平均额度

据中国青年网报道[1]，我们可以看到，大学生群体已逐渐成为“超前消费”的一部分。近九成受访学生透露，他们曾使用过分期付款的方式进行消费。以河北科技师范学院的贾义存同学为例，他提到，随着步入大四，面临步入社会的压力逐渐

增加，日常生活开销也随之水涨船高。因此，许多同学都选择了通过小额借款的方式来弥补生活费的不足。

这种消费观念的转变也悄然推动了金融信贷市场的快速发展。用未来的钱来满足现在的需求，金融信贷市场也得以日益强大。阿里的花呗、蚂蚁积分、人人贷等金融网贷平台凭借高效便捷的服务迅速吸引大量的群众。中国人民银行发布数据显示，2021 年末，金融机构人民币各项贷款余额 192.69 万亿元，同比增长 11.6%；全年人民币贷款增加 19.95 万亿元，同比多增 3150 亿元。[2]

贷款数量的增加必然会带来金融风险的增加，贷款数量的增加必然会带来金融风险的增加，由于信息不对称导致金融网络借贷信用风险也日益凸显。一些信用良好且还款能力强的客户往往也因此被拒之门外。同时每年都有众多金融信贷机构因为资金链断裂、违约、骗贷等原因而倒闭停业，其数量已高达上千家。金融机构该不该借钱？如何有效管理贷款人情况，做出准确风险判断？贷款人又如何规划还贷日期？成为了亟需解决的问题。基于以上痛点，我们决定开发一款多数据源的移动金融贷款风控系统。该系统将通过多个数据源的整合，包括用户公开信息、财务信息、信用评分、历史还款记录等，并利用人工智能和大数据分析技术，实现对贷款人的风险评估和情况监控。

1.1.2 数据流程分析

本项目涉及到的数据量大、流程较多。数据流程分析是项目分析中至关重要的一环，它有助于我们更好地理解数据在项目中的传递、转换和应用过程。本部分将重点介绍数据流程分析部分，通过对贷款风控系统中的数据流动、处理和控制进行全面剖析。

本项目旨在为个人用户和银行提供一站式服务，通过手机 APP 便捷高效地收集用户的个人信息和财务状况等关键数据，并将其储存在安全可靠的数据库中。在获取用户数据后，我们将运用先进的数据分析和建模技术，对用户的风险等级进行评估。最后，将评估结果以网页形式展示给银行，便于银行根据评估结果决定是否对用户进行风险提醒或借款等操作。

本系统的软件系统结构和行为展示如图 2 所示。

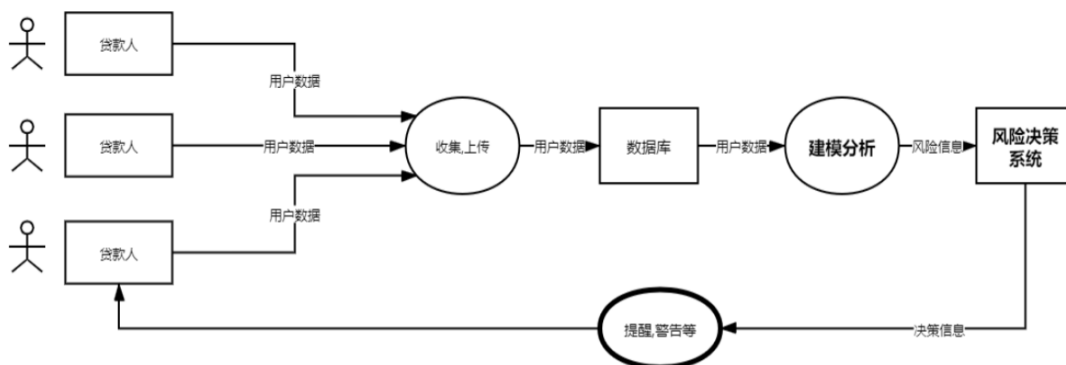


图 2：UML 用例图

为了确保贷款发放与管理的流程高效、安全且符合监管要求，我们进一步细化了核心功能。客户可以手动提交贷款申请，申请将进入贷前风险评估阶段。系统会根据客户的个人信息、财务状况以及其他相关因素给予对应的风险评级。一旦审批通过，系统将自动放款，进入贷中风险评估流程。在贷中阶段，系统会持续关注用户的日常行为数据，从而区分出高风险和低风险人群。对于高风险人群，系统将及时通知银行工作人员进行催收，以降低银行的风险。当用户完整还款后，该贷款项目的流程将结束。整体历经信息采集、贷前评估、贷中预防三个主要步骤，流程图如图 3 所示。

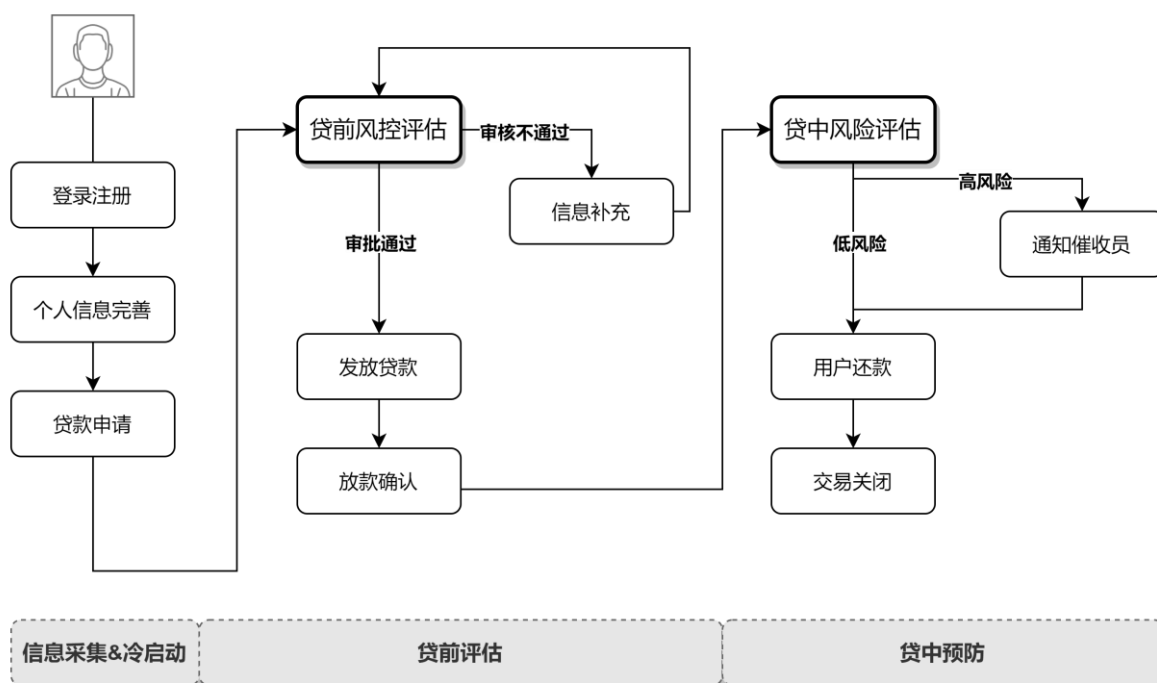


图 3：贷款核心流程图

图 4 的数据流图清晰地展示了用户贷款信息和个人数据在各个系统模块之间的传递过程。数据流图是系统分析与设计过程中的重要工具，它有助于我们更好地理解数据在系统中的流动、转换和处理过程。首先，借款人在填写贷款申请表时需要提供个人信息。这些信息将被传输到新增贷款数据库中进行记录和存储。接下来，贷款订单将在贷前资格审查与风险评估阶段生成。这个流程涉及到对借款人的信用评分和其他相关信息的评估。如果通过评估，贷款订单将被更新到贷款数据库中。然后，还款阶段开始，借款人需要按照约定的时间和金额进行还款。还款信息将被传输到更新贷款数据库中，以便跟踪还款进度和余额。此外，高风险预警功能会根据借款人的还款情况自动触发，以提醒催收员采取相应的措施。最后，催收员负责与借款人联系并催收欠款。他们使用系统提供的工具和服务来追踪借款人的还款情况，并采取必要的行动以确保债务得到清偿。整个数据流图展示了贷款过程中的关键步骤和数据流动路径，帮助我们更好地理解和管理贷款业务。

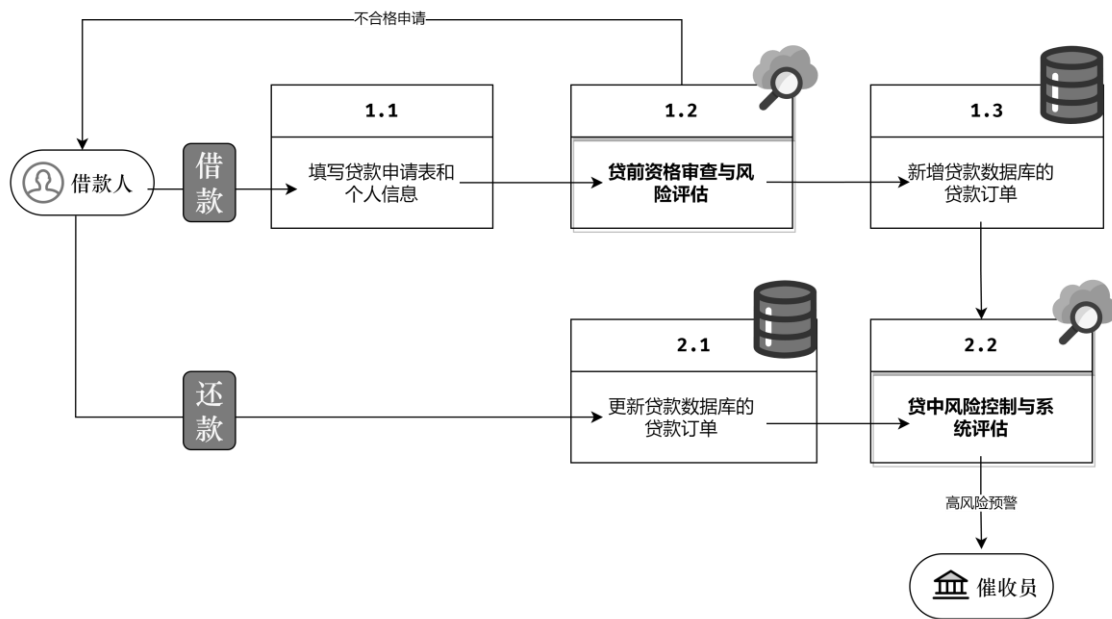


图 4：数据流图

1.1.3 功能需求分析

功能需求是贷款风控系统项目的重要组成部分，它们决定了系统的核心功能和业务处理能力。本文档将详细阐述本项目中的功能需求，以便为开发团队提供明确的指导，确保系统满足业务需求和预期目标。

首先，从宏观设计上，我们从功能分类、用户角色、性能安全三个方面定义了

三个总体需求——

● 功能分类需求

经过小组的深入调研，我们对腾讯微粒贷、360 融贷款、拍拍贷等成熟贷款平台的相关功能进行了全面了解。在此基础上，我们总结出了本项目的六大核心功能点，以期为用户提供更加便捷、高效的金融服务体验。功能需求如表 1 所示：

表 1：总体功能需求描述

功能点	具体描述
信息获取	① 用户填写：在贷款之前，用户需要填写一些必要信息，如个人基本信息、联系方式、工作单位等。这些信息将用于评估用户的信用状况和还款能力。 ② 授权采集：经用户授权，系统可以通过移动 APP 对用户相关信息进行采集，如 GPS 定位、IP 地址、通话记录等。这些信息可以进一步验证用户的身份和信用情况。
借贷还款	针对个人用户，可以进行基础的借款、还款操作，并可以通过 APP 自助查询全部贷款的详细情况和还款信息。
风险评估	建立机器学习模型，通过大数据，根据评分卡技术将不同的用户划分为不同风险等级，给出相应“信用分”。该模型可以根据用户的个人信息、财务状况、信用历史等多个维度进行分析和评估，从而准确判断用户的信用风险水平。
通知提醒	① 对于贷款人：系统将会发送还款提醒通知给用户，提醒其按时还款。提醒方式可以是邮件、APP 推送等形式，以确保用户不会忘记还款。 ② 对于银行：系统会实时监控高风险用户的情况，并及时发送通知给银行催收员，以便他们能够及时采取措施催收欠款。同时，系统也会向银行发送通知告知客户进行联系，以帮助银行更好地管理风险。
系统管理	针对管理员用户，提供 web 端进行管理操作。具体包括：贷款产品的上架、贷款申请的查阅、用户信息的浏览、数据大屏通览、阈值设置等
辅助功能	如注册登录、用户管理、数据大屏、产品管理、智能客服、邮件提醒等非核心功能

● 用户角色需求

在这个贷款平台综合性项目中，个人用户（借款人）、银行和管理人员作为项目的主要角色，各自具有不同的需求和职责。相关用户需求表如表 2 所示，详细列出了各角色的需求和期望。

表 2：用户需求表

用户角度	需求项	需求描述
贷款人	信息采集	①用户填写：在贷款之前。用户必须填写一些必要信息，如：用户必须要实名认证。 ②授权采集：经授权，能够通过移动 APP 对用户相关信息（如 GPS、IP、通话记录）进行采集并且能传输到数据库，供银行评分。
	贷款管理	用户可以在平台通过简单、快捷的流程完成借款申请并查看自己的贷款的详细信息，同时，可以获取相关还款催收提示信息。
	还款管理	用户可以自行查看还款的时间、金额和逾期时间。可以选择提前还款，还是正常还款。
银行	风险评估	通过建模分析，对贷款人进行风险评级，智能识别潜在风险用户，及时审批。以降低坏账风险，并确保贷款的安全性。
	预警反馈	对于高风险用户，及时发送通知告知催收员，并及时通知客户进行联系。
	贷款管理	查看贷款人的贷款申请，已还款金额。
管理员	全局管理	对于全局信息进行管理查看，并维护后台信息。

个人用户（借款人）是平台的核心参与者，他们关心的是能否便捷、快速地获得贷款。为此，我们将致力于提升借款申请、审批、还款等流程的效率，让用户在享受贷款服务时能够感受到轻松、愉快的体验。

银行作为贷款的提供方，关注风险控制和资金安全。我们将协助银行搭建一套完善的风险评估体系，并通过技术手段保障资金的安全流通，让银行在拓展业务的同时，降低风险。

管理人员在项目中起到管理和监督的作用，他们对平台运行状况、数据分析和决策支持有很高的要求。为此，我们将为管理人员提供实时的数据监控和分析工

具，以便他们能够及时了解平台运行状况，制定合理的决策，推动项目健康发展。

其中，个人用户主要使用移动终端（安卓端）进行操作，银行及管理员使用 Web 端网页管理后台进行操作。

● 产品质量需求：

在这个贷款平台综合性项目中，我们致力于打造一个集安卓、网页、服务器三端于一体的统一联合体。系统安全性与性能等因素是保证产品质量的客观需求，为了确保项目的顺利进行，我们已制定了详细的性能安全需求表（如表 3 所示）

表 3：产品质量需求

角度	具体描述
正确性	该产品功能实现度达到 100%
兼容性	能够适配市面上主流浏览器
提示性	产品有明确的功能引导和指引，界面美观 可快速冷启动，启动时间不超过 2s
性能	在正常的网络情况下，网络请求响应时间不超过 2s（查询类不超过 1.5s） 在异常的网络情况下，能够返回相应的报错信息 可以支持多用户多并发情形
可扩展	预留相关接口，可以变换对接不同数据库
可操作	对于普通用户（如银行工作人员）可以快速进行设置相关参数，用户交互性友好，可操作性强。
可靠性	在一般使用情况下，系统支持长期连续运行 各功能模块可相互独立工作，减少相互影响 数据在传输过程中保持安全，不会造成泄露
安全性	核心数据在存储过程中借鉴区块链底层加密技术，通过哈希编码可对数据进行二次验证。
健壮性	使用当前主流框架，异常情况较低

此外，针对贷款流程中的核心的几大流程功能，我们设计了相关具体的需求列表——

对于贷款申请这一复杂流程，我们设计了如表 4 的需求表。需求表从功能角色、功能描述、异常事件流、前置输入等条件出发，详细阐明了贷款申请这一功能的具体要求和具体内容。

进阶式挑战性综合设计项目 III

表 4：贷款申请功能需求表

需求名称		贷款申请	需求编号	APP_DKSQ
优先级别		高	角色	贷款人
功能描述		根据需求申请相应的贷款		
触发事件		贷款人在 APP 借款页面，点击【申请】触发		
异常事件流		1. 申请金额超过上限，会出现“金额超出最高限定”的提醒 2. 验证码输入错误，会出现“验证码错误”的提醒		
前置条件（输入）		用户选择贷款产品，传入产品 ID		
后置条件	最小保证	可以查看贷款的基本信息		
	成功保证	可以成功向银行发送贷款申请		
业务规则		1. 输入贷款金额（不超过最高额度），选择分期、计利方式、贷款用途 2. 确认贷款信息 3. 选择身份验证方式：①短信验证，②指纹认证 4. 申请成功		
其他备注		产品 ID 由管理端设置		

还款功能是仅次于借款功能的一大核心功能需求，需求表如下

表 5:还款功能需求表

需求名称		还款	需求编号	APP_HK
优先级别		高	角色	贷款人
功能描述		根据经济情况选择还款内容		
触发事件		贷款人在 APP 还款页面，点击【还款】触发		
异常事件流		1. 当不处于还款周期内，会出现“不能提前还款”的提醒 2. 当选择错误的期数还款时，会出现“还款需要按照时间顺序”的提醒		

第一章 复杂工程问题归纳与实施方案可行性研究

前置条件（输入）		贷款申请 ID
后置条件	最小保证	可以看到已还款和已逾期的期数
	成功保证	成功完成还款
业务规则		1. 进入申请详情页面，可以看到历史还款状态 2. 选择正常还款，选择想要还款的期数，之后进行支付 3. 选择提前还款，可以支付全部剩余还款金额
其他备注		

以上两个是用户 APP 的核心需求功能，对于管理员，我们也设计了相关需求表。例如，下表展示了 PC 端管理员查看用户申请的信息

表 6：风险提醒需求表

需求名称		风险提醒	需求编号	PC_FXTX
优先级		高	角色	银行
功能描述		查看全部风险提醒用户		
触发事件		在速易贷管理系统的风险提醒页面，点击【邮件提醒】触发邮件提醒		
异常事件流		1. 短时间内连续触发邮件提醒会予以阻止		
前置条件（输入）				
后置条件	最小保证	不会对数据产生影响		
	成功保证	成功查看全部风险提醒用户		
业务规则		1. 点击侧边栏，进入页面查看贷款人的风险等级 2. 点击【邮件提醒】向贷款人发送邮件 3. 点击【详情】可以查看用户的详细还款信息		
其他备注				

管理员同样可以对贷款产品进行新增操作，具体的新增需求可见下表。其配置规则较为复杂，在开发过程中需要进行多重表单验证。

进阶式挑战性综合设计项目 III

表 7：贷款产品新增需求

需求名称		贷款产品新增	需求编号	PC_CPGL_XZ
优先级别		高	角色	银行
功能描述		增加贷款产品		
触发事件		银行在速易贷管理系统的贷款产品页面，点击【新增】触发		
异常事件流		1. 当产品 ID 为空时，会出现“产品 ID 不能为空”的提醒 2. 当产品 ID 重复时，会出现“已存在，请换一个 ID”的提醒 3. 当信息存在空缺时，会出现“请填写完整信息”的提醒		
前置条件（输入）				
后置条件	最小保证	添加出现异常时，不会对已有产品产生影响		
	成功保证	成功添加贷款产品		
业务规则		1. 填写产品 ID 2. 产品名称 3. 选择产品发布状态 4. 填写最大借款值 5. 填写审批阈值 6. 添加产品利率与分期		
其他备注				

通过对需求的详细分析，我们可以更清晰的了解各个需求之间的具体业务逻辑，为后续开发奠定基础。

1.1.4 市场调研

随着金融市场的快速发展，贷款需求日益增长，同时风险也不断凸显。为了更好地满足市场需求，提高金融机构的风险管理水平，速易贷：智能贷款风险控制系统应运而生。本部分将从同类产品对比的角度，分析市场现状。

招商银行信贷风险决策系统作为传统金融机构的代表，拥有丰富的信贷风险管

理经验和市场认可度，在国内市场具有较高的地位。多年以来，招商银行致力于优化信贷风险管理体系，采用严谨的审批流程和专业化的风险控制手段，为客户提供了可靠的金融服务。其在传统信贷业务领域的深厚积累，使其在市场上具有较高的声誉和竞争力。然而，招商银行信贷风险决策系统的线上渠道较少，主要集中于线下进行审批、签约等环节。这种传统的操作方式在一定程度上限制了其业务拓展的速度和效率。相较于速易贷项目，招商银行信贷风险决策系统的线上业务较少，客户体验可能受到影响，流程较为繁琐。

在金融科技日益发展的当下，其他第三方决策系统在面对线上业务的竞争时，也积极寻求创新和变革。例如，芝麻信用分依托支付宝平台的支持，拥有强大的大数据和技术实力，用户范围广泛，已成为消费金融领域的重要参与者。然而，作为第三方信用评估机构，芝麻信用分在数据来源和准确性方面存在一定的局限性。首先，芝麻信用分的数据来源主要依赖于支付宝平台上的交易数据和用户行为数据。虽然这些数据量庞大，但在反映用户信用状况的全面性和客观性方面仍有不足。相较于招商银行等传统金融机构，芝麻信用分在数据来源方面相对单一，可能导致信用评估结果的准确性受到一定影响；其次，芝麻信用分作为第三方信用评估机构，其数据准确性在一定程度上受到外部因素的影响。例如，用户可能存在故意隐瞒或虚报信息的现象，这将对信用评估结果的准确性造成干扰。此外，芝麻信用分在数据清洗和验证方面也需要不断加强，以确保用户信息的真实性和可信性。

总体来说，各类竞品在市场中各具特色，相互之间存在一定的竞争。但在金融科技快速发展的背景下，智能贷款风险控制系统市场仍有较大拓展空间。通过持续优化技术和提升服务质量，速易贷项目有望在市场中脱颖而出，为金融机构提供高效、精准的风险管理服务。

1.1.5 技术方案选型

经过综合对比各大程序技术，并参考了相关文献和资料，项目前后端分离的方式进行项目开发。在此基础上，以 Gitee 作为协作平台，并设计了如下图所示的项目架构。本项目涉及的技术方案选型如下所示

- 1 前端：**Web 页面将以 Vue 为核心框架，配合 Element-UI 组件进行搭建。安卓端则选用 uniapp 框架，结合 uView 组件库，构建一个类似 Web 页面的安卓程序。通过 Ajax 实现与后端的网络请求交互，以获取需渲染的数据并呈现给用户。

- 2 **后端：**为确保与机器学习的兼容性，我们选择以 Ktolin 语言为主，搭配 Ktor 框架进行开发。数据库方面，我们拟采用 MySQL 来存储用户上传的数据序列信息，以 druid 作为数据库连接技术。
 - 3 **其他：**项目将采用 Gitee 作为代码托管平台，实现并行分支开发。同时，我们还将使用 Apipost 平台进行接口 API 的设计、调试和测试。
- 本项目的技术架构如图 5 所示。

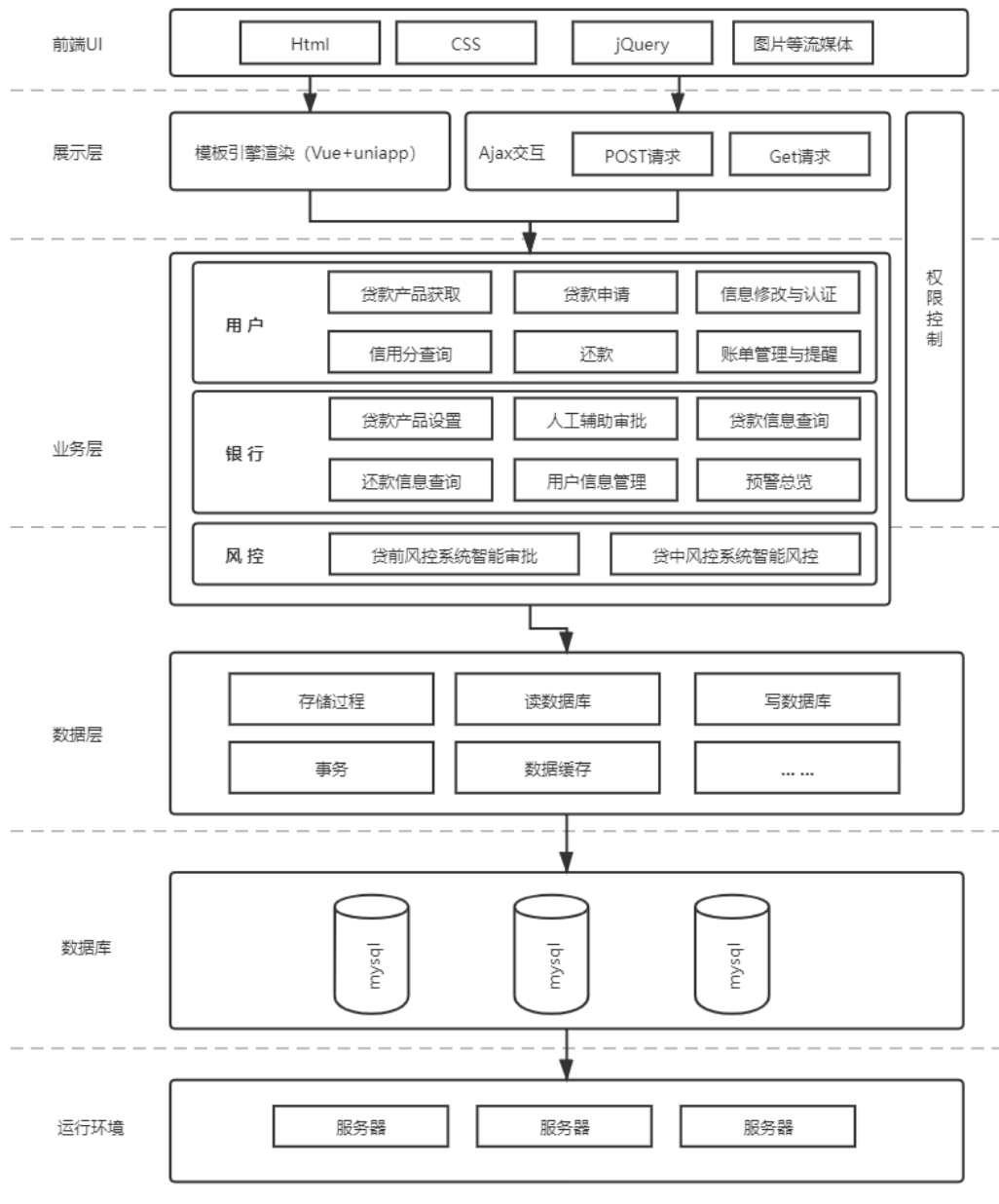


图 5：系统架构图

1.1.6 核心复杂工程归类与建模

项目可能存在一些复杂问题，下面对其进行简单归类。

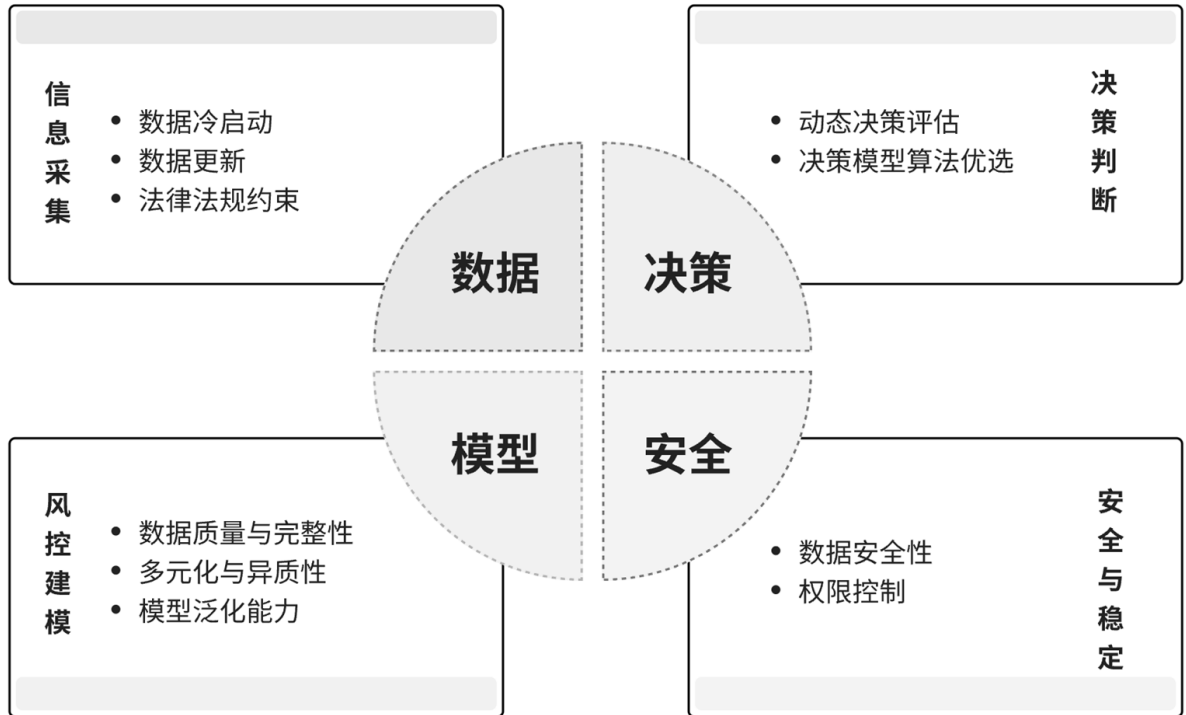


图 6：项目存在的四类复杂工程

- **信息采集：**信息采集条目多，采集方式复杂，获取困难。数据采集过程中，涉及众多信息维度，如借款人的基本信息、信用记录、还款行为、财务状况等。这些数据分布在不同的系统和平台，采集方式复杂且耗时。为确保数据的完整性和准确性，需要对各类数据源进行梳理和对比分析，制定合适的信息采集策略。具体问题包括——
 - （1） **数据冷启动：**在系统启动初期，可能面临数据量不足、数据质量不高、有效样本稀缺等问题。这会增加模型训练的难度，导致模型性能不佳。
 - （2） **数据更新：**随着业务的发展和市场环境的变化，数据需要不断更新和维护。如何在保证数据时效性的同时，确保数据的准确性和完整性，是数据采集环节需要关注的问题。
 - （3） **法律法规约束：**金融风险控制系统需要遵循相关法律法规，如我国监管部门发布的金融科技创新监管政策。同时，还需要关注数据隐私保护、借款人权益保障等伦理问题。这些约束在一定程度上增加了

系统的复杂性。

- **风控建模：**违约评分预测模型多样，选择性广，这个领域内问题较多，具体可以细化为这几个方面——
 - （1） **数据质量与数据完整性：**在风险控制系统中，数据的质量与完整性对于模型的准确性至关重要。然而，金融机构在获取和处理数据时，可能会遇到诸如数据缺失、不一致、错误等问题。这些问题可能导致模型输出不准确，从而影响风险评估和贷款决策的效果。
 - （2） **多元性与异质性：**金融机构的业务范围广泛，涉及不同行业、领域和客户类型。这就要求风险控制系统能够处理多元性和异质性的数据，如不同类型的贷款产品、借款人的信用状况、还款能力等。在解决这一问题时，需要设计复杂且具有泛化能力的算法，以适应各种场景和数据特点。
 - （3） **模型泛化能力与过拟合风险：**在训练模型时，可能会遇到过拟合现象，即模型在训练数据上表现良好，但在未知数据上表现较差。为了解决这个问题，需要关注模型的一般解释能力，采用交叉验证等技术来降低过拟合风险。
- **决策判断：**预测阈值确定困难，能否进行准确的决策判断难以预料。无论是贷前的审批还是贷中的催收提醒，精准的决策能有效减少人工干预成本，因此，提高决策判断的精准度也是非常重要的问题。具体问题可以细化包括——
 - （1） **动态决策评估：**随着市场环境、政策法规和借款人状况的不断变化，风险评估需要具备较强的动态性。这意味着风险控制系统需要不断调整和优化模型参数，以适应不断变化的风险态势。此外，还需要关注时间序列数据的特点，如季节性、周期性等，以便更准确地预测风险。
 - （2） **决策模型的算法选择：**速易贷风险控制系统需要选择合适的机器学习算法来处理和分析大量数据。在众多算法中，如何选择适合金融风险控制的模型以及如何优化模型参数，是一个复杂且富有挑战性的任务。此外，如何平衡模型的预测效果与解释性也是一个需要关注的问题。
- **系统安全性与稳定性：**金融风险控制系统涉及大量敏感数据和关键业务，如何确保系统的安全性至关重要。这包括数据加密、权限控制、故

障应对等方面的安全保障。同时，系统的稳定性也至关重要，以确保在各种异常情况下，系统能够正常运行。

1.2 实施方案可行性研究

1.2.1 重难问题的实施方案构想

针对提出的复杂工程问题，我们分类构想了如下的综合性解决方案。

- **信息采集**

如何解决数据量问题？在解决数据冷启动问题时，我们可以采用数据增强、迁移学习等方法，借助少量有效数据扩展训练数据。此外，通过网络上的公开数据集建立开源数据集，借助外部数据来弥补内部数据的不足。为了确保数据的时效性，我们需要建立实时数据更新机制，并建立数据质量检查流程，在特定时间弹出提醒告知用户更新个人基础信息。对异常数据进行清洗和校验，以提高数据的准确性和完整性。

如何获取用户信息？在 APP 运行期间，经用户授权，通过 APP 定时调用安卓原生 API 获取浏览器历史记录、应用列表、位置信息、联系人信息等网络基础信息，并将采集的用户数据传输至后端。在用户申请贷款时，要求用户填写贷款申请表并完善个人信息，提供学历、职业、收入、住房情况、银行账户情况等信息；同时，可以利用数据库中已经存在的交易数据作为交易特征，纳入信用评分中。

- **风控建模**

在风控建模方面，我们要关注数据质量与数据完整性，通过数据预处理技术如数据清洗、去重和填充缺失值等来提高数据质量。对于不一致和错误的数据，我们可以通过数据对比分析和人工审核的方式进行修正。

同时，我们需要更多关注模型的解释性，选择具有较好解释能力的模型，如决策树、逻辑回归等。基于此我们小组又做了细致的建模方法的调研——

建立模型的方法可分为统计学方法和非统计学方法。统计学方法中，线性回归和逻辑回归应用最为广泛。非统计学方法主要包括神经网络和遗传算法。下面，对这两种方法进行详细对比。

若采用统计学方法，需根据样本进行参数估计以建立模型。线性回归和逻辑回归具有参数调整简单、可解释性强等优点，便于非计算机专业人员操作。借助现有资料的专家模型进行调整，可提高预测精度。此外，评分卡模型内部逻辑关系清晰，

通过逐步抽离字段构建模型，直观易懂，能直接展现变量间的内部逻辑。（如图 7 所示）

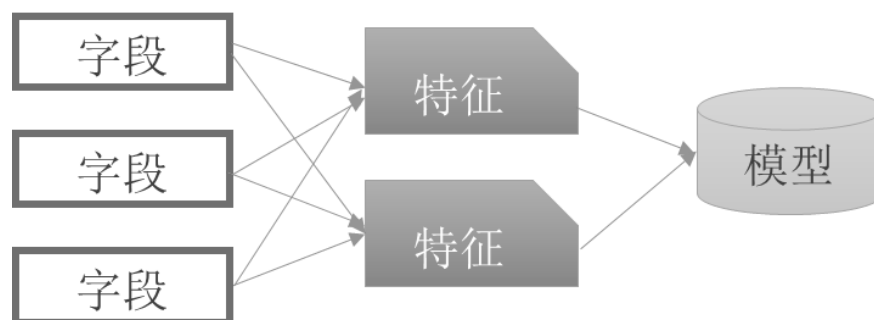


图 7：评分卡模型生成逻辑

若采用非统计学方法，神经网络模型通过对训练样本进行训练而得到。遗传算法与神经网络类似，从潜在解的总体出发，通过变异或交叉产生新一代解。然而，此类方法求解过程未知且复杂，可解释性较弱，难以分析数据间的关联情况及相关关系。而且，在进行小幅度调整时，可能需要消耗较多时间，从效益角度看并不合适。

表 8：统计学建模与非统计学建模的对比

建模方法	统计学方法	非统计学方法
参数调整	简单	复杂
可解释性	高，解释变量简单易懂	低，存在黑箱问题
样本需求	一般	较大
可调整性	略低	一般
自动化程度	略低	高
训练成本	低	高

综上，我们选择基于统计学的评分卡模型进行贷前风险评估的数据建模，该模型简单直观，易于理解和操作。

• 决策判断

在决策判断方面，我们要关注动态决策评估，建立风险评估动态调整机制，根据市借款人状况调整阈值、参数。

根据评分给用户划分等级，系统自动拒绝低等级用户的贷款申请，接受高等级用户的贷款申请。对于信用等级中等的用户，由资料审查员审查用户贷款申请，结

合用户资料和经验判断决定是否通过贷款申请。

信用分的阈值采用专家系统与模型训练结合的方式，参考现有的论文资料的基础上，得出一个合理的授信区间，再由训练出的评分卡模型给出具体的分数值阈值，若分数值处于区间内则接收；不处于则继续补充数据训练。

• 系统安全

在系统安全性和稳定性方面，我们需要确保数据安全，采用数据加密、权限控制等技术。同时，建立故障应对机制，如备份等机制，确保系统在异常情况下正常运行。

首先，用户登录必须进行登录验证，我们通过路由守卫、权限验证方案可以实现防用户权限入侵；同时，在敏感操作发生时，验证用户基本网络状态（如 IP 等信息是否与地址匹配），并通过邮件等方式进行提醒。

然后，对于核心数据运用区块链底层技术进行仿区块链上链。对于贷款申请信息添加上混合哈希值，用于对数据进行防篡改验证。

整个系统安全流程设计如图 8 所示。

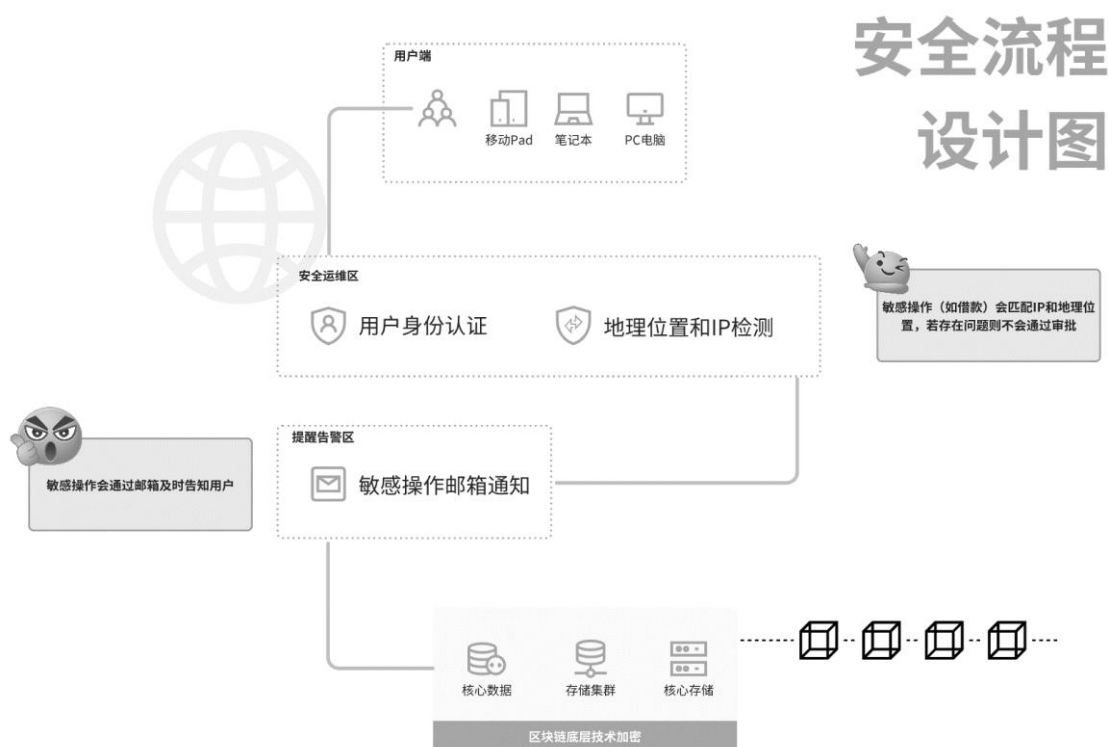


图 8：系统安全流程设计图

多重功能方案可以有效防止攻击，在一定程度上保证系统安全性。

1.2.2 可行性分析

下面我们从技术可行性、开发可行性、市场效益可行性三个角度来具体阐述项目复杂问题可行性。

1 技术可行性

- 合适的技术栈选择：在多数数据源的金融贷款系统中，关键技术在于信息的动态展示和管理。这意味着从数据库中读取数据、收集用户信息，并对用户的相关数据进行业务处理。我们选用了 Ktor 和 Druid 等进行后端技术开发，Vue 和 UniApp 框架构建用户界面，使得系统的部署、应用和维护更加便捷。同时，MySQL 数据库提供了方便的数据库管理功能。前后端分离的开发模式更有利于团队协作。

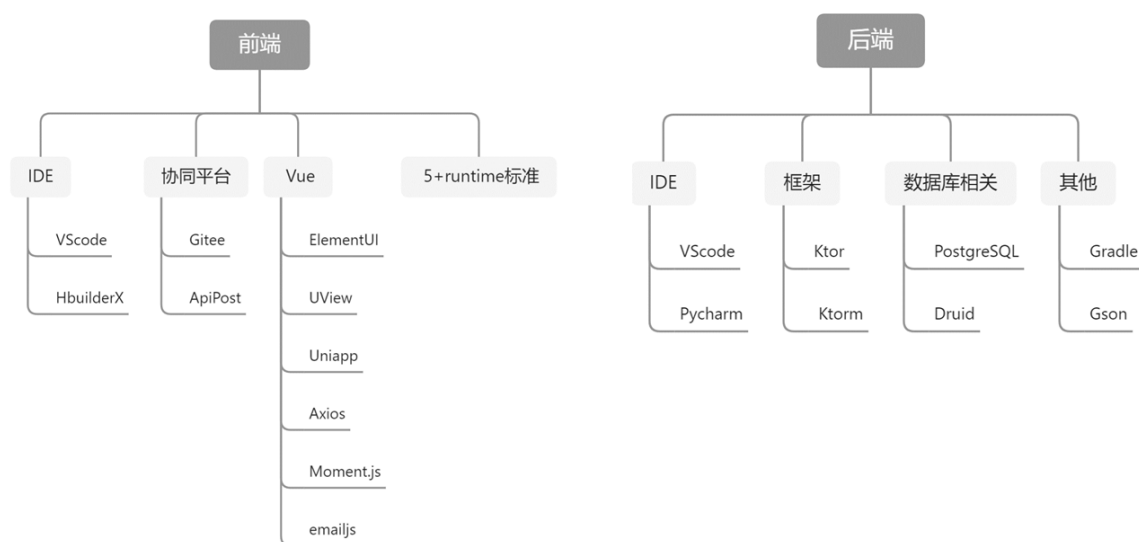


图 9：前后端技术栈选型

- 易于获取 APP 数据：通过申请相关权限，APP 可以轻松获取浏览器历史记录、应用列表、位置信息和联系人信息。目前，各大应用厂商均对 APP 开放了相关 API 调用，我们所采用的 UniApp 前端框架也已集成相关 API。在实现过程中，只需在用户同意后调用 API，APP 即可自动获取并传输数据至后端。
- 贷前数据建模方法选择：在研究和比较“基于统计学方法”和“基于非统计学方法”的优缺点后，项目采用基于统计学的建模方法。原因如下：
 - a) 基于统计学的方法简单易懂，易于解释。通过矩估计法和极大似然

估计法求出参数建立模型。这类模型对非专业技术人员更具可交互性，便于实际运行中银行管理员等按需调整。若要提高精度，可以适当增加样本数据量，加强模型参数拟合。

- b) 非统计学方法建立模型的优点在于它能描述建模数据中的复杂线性关系，预测精度较高。然而，由于计算过程基本为“黑箱”，可解释性较差。另外，模型建立过程较为困难，管理人员可交互性较低，操作性不强。

2 开发可行性

- 业务流程优化：小组对项目流程进行了全面梳理和优化，以提高业务处理速度和效率。例如，采用自动化审批流程、大数据分析等技术手段，可以有效简化业务操作，降低人力成本。
- 项目管理优化：采用敏捷开发、持续集成等现代项目管理方法，提高项目开发效率。同时，加强项目风险管理，确保项目在遇到问题时能迅速调整和应对。

3 市场效益可行性

- 有效减少人力资源成本：绝大多数贷款申请可以由程序自动判断，只有信用评分在临界值附近的中等信用等级用户的贷款请求才由人工审核，这样充分利用了数据模型，人工工作量小，效率高。对公司来讲可以减少一定的人员开支。
- 提高用户贷款效率，增加用户满意度：线上申请、智能审批、贷款管理，极大的方便了用户贷前、贷中、贷后系列流程的办理；机器学习可以支持秒级审批，极大的提升了放款速度。用户满意度因而增加。

第二章 针对复杂工程问题的方案设计与实现

2.1 针对复杂工程问题的方案设计

2.1.1 总体设计

系统整体开发采用前后端分离模式，将前端和后端分离开发。使用 B/S 架构进行设计（图 10），通过使用标准接口进行交互，我们可以降低不同模块之间的耦合性，提升系统的可扩展性和可维护性。在此基础上，采用前后端分离的架构使得前端和后端可以分别独立地进行开发与维护，进一步降低耦合度，提高开发效率和代码质量。同时，多个开发人员可以协同合作，共同开发同一项目，从而实现高效的并行协作。

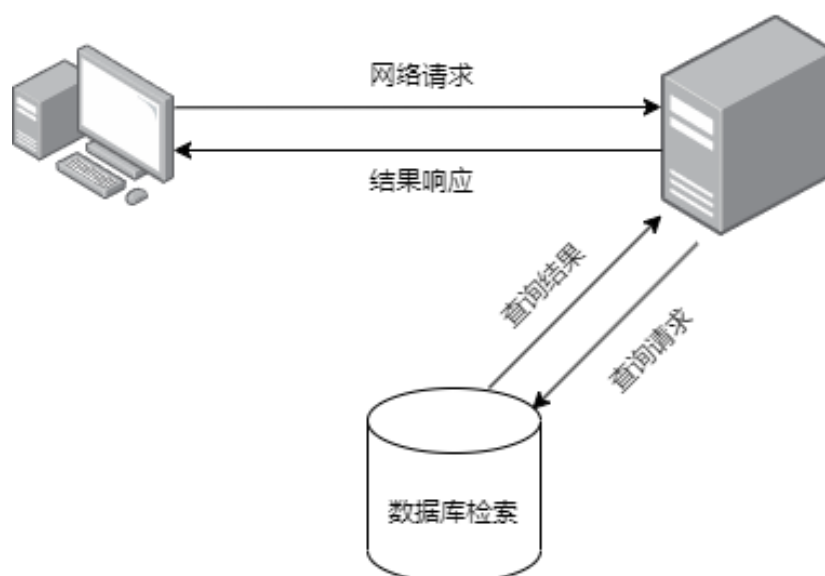


图 10：B/S 架构

前端利用 Vue 相关技术分别搭建了管理员端和用户端，后端使用 Kotlin 搭建，数据库采用关系型数据库，利用评分卡模型对用户的还款概率和风险进行智能评估与预测。总体设计方案如下：

□ 前端：

前端主要负责处理业务逻辑和渲染页面，考虑到系统的双端用户特性，我们选择使用 Vue 和 Uniapp 技术分别构建管理员端和用户端的前

端页面。这种框架组合使得项目更轻量、高效，并具备高定制化和渐进式增强等特点，避免了重复开发相似的代码，提高了开发效率。

Vue 作为一套用于构建用户界面的渐进式框架。与其它大型框架不同的是，Vue 被设计为可以自底向上逐层应用。Vue 的核心库只关注视图层，不仅易于上手，还便于与第三方库或既有项目整合。另一方面，当与现代化的工具链以及各种支持类库结合使用时，Vue 也完全能够为复杂的单页应用提供驱动。

我们选择了 uView 和 element-ui 这类高可视化、实用性强、低内聚高适配的组件库作为 UI 框架。这些 UI 库提供了丰富的可视化组件，便于开发者快速构建美观且交互丰富的用户界面，提升了应用程序的用户体验和整体质量。

同时，我们使用 axios 进行 API 请求与响应，与后端进行通信。并通过 webpack 打包生成静态资源文件，以实现更快的加载速度。

此外，搭配 Vue-Router、MomentJS 等第三方库进行设计和处理，使项目更加灵活。Vue-Router 帮助我们实现页面路由控制，而 MomentJS 提供了方便的时间格式化和处理功能。这些库在提升开发效率的同时，也增强了应用程序的功能性和易用性。

□ 后端：

后端负责处理业务数据并返回响应，采用 Kotlin 和 Ktor 快速构建 RESTful API，实现高效数据处理。同时，使用 Druid 连接池技术保障数据库高效访问。为保障系统安全性和可维护性，引入身份验证、请求过滤、日志记录等功能。此外，采用缓存技术优化系统响应速度，提升用户体验。

总之，后端采用现代化技术和开发方法，兼具高性能、高效率、可维护性和安全性，为前端提供可靠后台服务支持。后期需持续优化性能、增加新功能，以满足用户需求。

□ 数据库：

我们选择 PostgreSQL 作为数据库，主要基于其高度可靠性、可扩展性、丰富的数据类型和强大的事务处理机制。此外，PostgreSQL 支持存储和查询数组类型数据，满足业务需求。

为简化数据库操作，我们采用 Ktorm 作为 ORM 框架，其轻量级、简洁的 DSL 和序列 API 减少了代码量和重复劳动。Ktorm 还支持类型

安全和可组合性，提高代码可读性和可维护性。

在数据库设计方面，我们遵循巴斯 - 科德范式规范，避免数据冗余和不一致。按主题将数据存储在不同的表中，确保表内列具有原子性和唯一性。同时，考虑数据扩展性和性能，建立适当的索引和分区。

综上，通过选择 PostgreSQL 数据库和 Ktorm ORM 框架，遵循巴斯科德范式规范进行设计，我们实现了高效、可靠的数据存储和查询，确保了系统数据稳定性和性能。

2.1.2 功能设计

基于综合用户需求和竞品（如 360 贷、京东白条等）分析的基础上，我们对系统的功能模块进行精简分离。为了更清晰描述系统与外部参与者之间的交互过程和功能点，帮助设计者对这些需求进行分类和组织，确定系统的各个模块之间的依赖关系和功能关系。我们进行总结提炼，得出如图 11 的用例模块分析图。

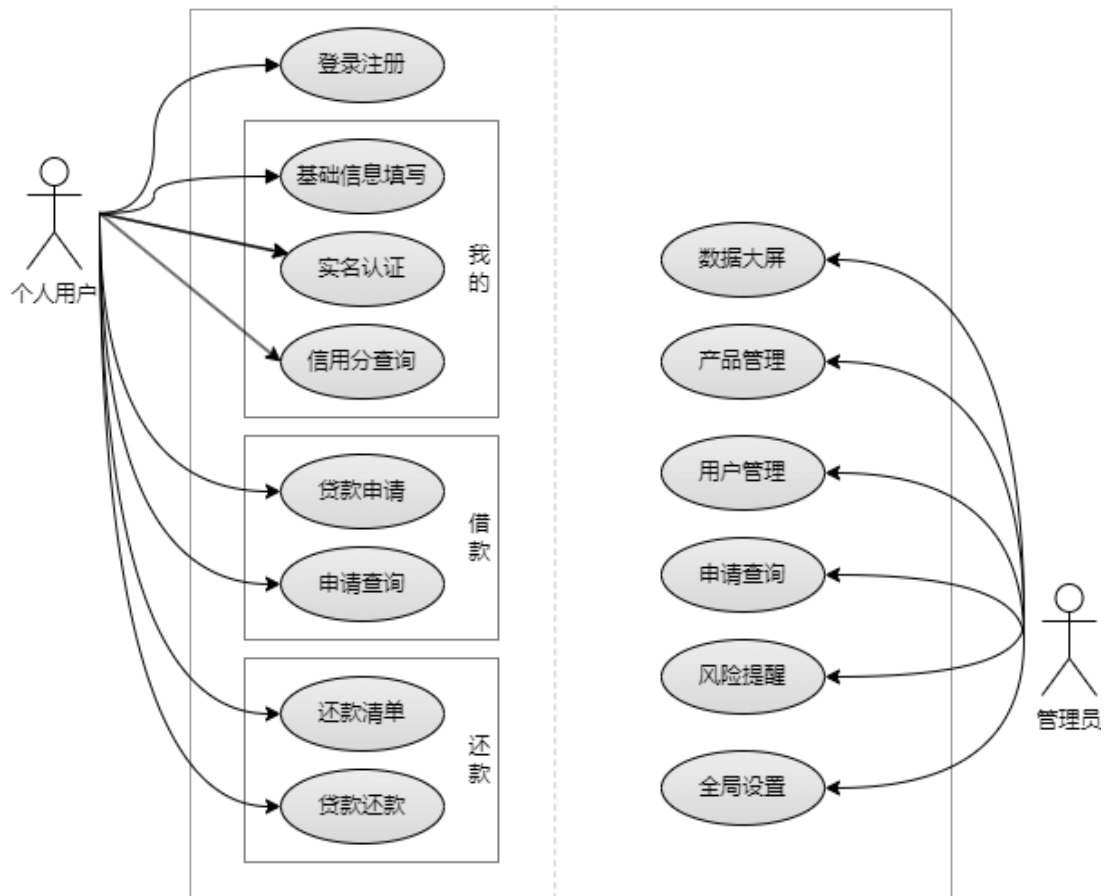


图 11：用例模块图

第二章 针对复杂工程问题的方案设计与实现

通过对用例的分析和归纳，我们将系统划分为多个模块，每个模块负责处理一个或多个相关用例，从而实现高内聚、低耦合的系统设计。此外，我们整合和提炼了功能模块，使其更加清晰和具有针对性，以满足不同用户的贷款需求。在设计过程中，我们着重考虑用户体验和系统稳定性，确保系统流畅、高效和易用。

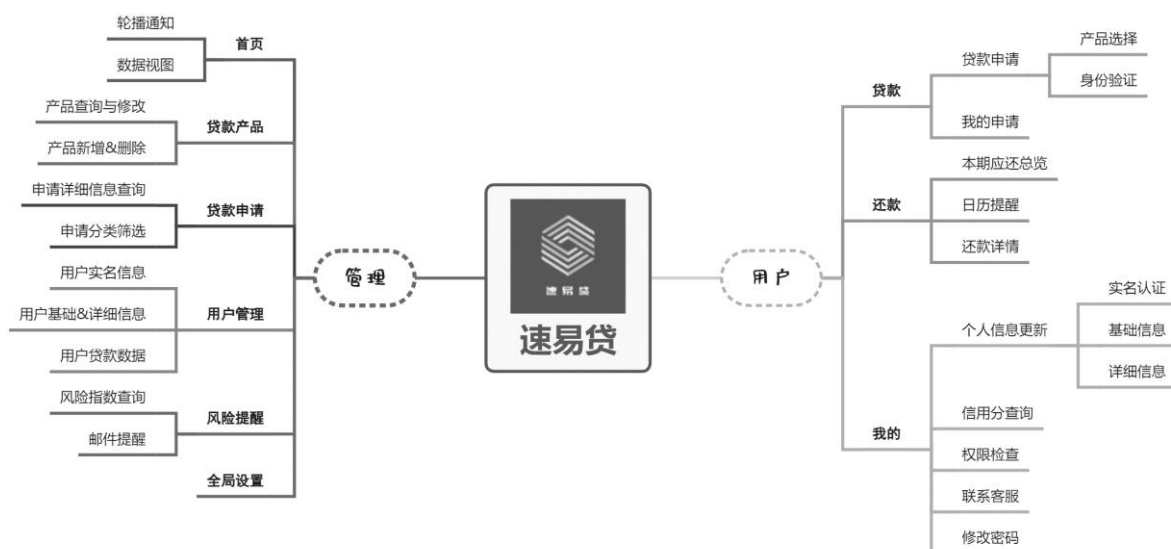


图 12：功能模块设计

单元模块设计如图 12 所示。用户端提取了核心的三大板块——贷款、还款、我的，合计 10 项功能单元；管理端提取了六个主要页面，合计 12 项核心功能单元。

对用户端而言，我们将功能板块分为贷款、还款和我的三个部分，使用户在操作时更便捷、友好。用户无需深入了解整个系统结构可直接根据需求选择相应板块。同时，内部单元细化有助于用户快速找到所需功能或信息，提升用户的体验。

对于管理端，我们将页面和核心功能单元划分为 6 个主要页面和 12 项功能单元，使管理员在管理过程中更简单明了。各页面分工明确、独立，便于管理员进行分类管理和精细化操作。合理的功能单元划分也有利于系统安全性和可靠性，避免单一功能失效导致整个系统瘫痪。

这种功能划分有利于系统开发、维护和扩展。各模块职责清晰，可独立开发和测试，便于分工合作，提高开发效率和质量。模块化划分也有助于后期系统扩展和升级，便于维护和升级。

我们在下表具体罗列了本项目所需的必要功能和开发排期。

表 9:项目功能详细设计

需求编号	功能端	功能模块	需求描述	优先级	开发排期
APP_DLZC	APP	登录、注册		1	一期
APP_XGMM		修改密码		2	一期
APP_SMRZ		我的	实名认证	1	一期
APP_JCXX			基础信息更新	1	一期
APP_XXXX			详细信息更新	2	一期二期
APP_XYCX			信用分查询	1	一期二期
APP_QXJC			权限检查	2	一期
APP_LXWM			联系我们	3	二期
APP_ZNKF			智能客服（大模型）	3	二期
APP_CPCX		借款	产品查询	1	一期
APP_SQDK			申请借款	1	一期二期
APP_WDSQ			我的申请	2	一期
APP_HKQD		还款	还款清单	1	一期
APP_HKTX			日历还款提醒	3	二期
APP_DKXQ			贷款详情	1	一期
PC_DL	PC	登录		1	一期
PC_SJDP		首页	数据看板（内嵌页）	2	一期二期
PC_LBTZ			轮播图片通知	3	二期
PC_LED		数据大屏（独立网页）		2	二期
PC_CPCX		贷款产品	贷款产品查询	1	一期
PC_CPXG			贷款产品新增/修改	1	一期二期

第二章 针对复杂工程问题的方案设计与实现

PC_CPSC			贷款产品删除	2	一期
PC_SQCX		贷款申请	用户申请查询	1	一期
PC_SQSX			申请分类筛选	2	一期
PC_SQXQ			用户申请详情	2	一期二期
PC_YHCX		用户管理	用户查询	1	一期
PC_YHXX			用户信息查询	1	一期
PC_YHSS			用户搜索	2	一期
PC_FXSQ		风险提醒	风险申请查询	1	一期二期
PC_FXXQ			风险申请详情	2	一期二期
PC_FXYJ			风险用户邮件提醒	3	二期
PC_QJSZ		全局设置	阈值设置	2	一期二期
HT_DQFK	后台	风控模型	贷前风控模型	1	一期二期
HT_DZFK			贷中风控模型	1	一期二期
HT_SJJM		数据存储	贷款信息加密存储	2	二期

* 一期代表综设 II 开发，二期代表综设 III 开发

2.1.3 数据库设计

基于 Power Designer，我们的小组完成了符合四大范式标准的数据库设计，确保了实体和属性的规范化。通过运用多表外键，我们有效地组织和管理了数据，避免了数据冗余和不一致性。同时，我们遵循了关系型数据库设计的最佳实践方案。

如图 13 所示，我们设计了五张表来管理贷款的核心数据与信息。这五张表分别是用户表、用户信息表、贷款产品表、计息方式表、贷款申请表。通过这种设计，我们实现了对贷款业务数据的全面管理和高效处理。

在此过程中，我们充分考虑了实体之间的关系，确保了数据的完整性、一致性和准确性。这种规范化的数据库设计将为我们的业务系统提供强大的支持，有助于提高系统性能和稳定性，降低维护成本。同时，这种设计也为未来的功能扩

展和需求变更提供了极大的灵活性。

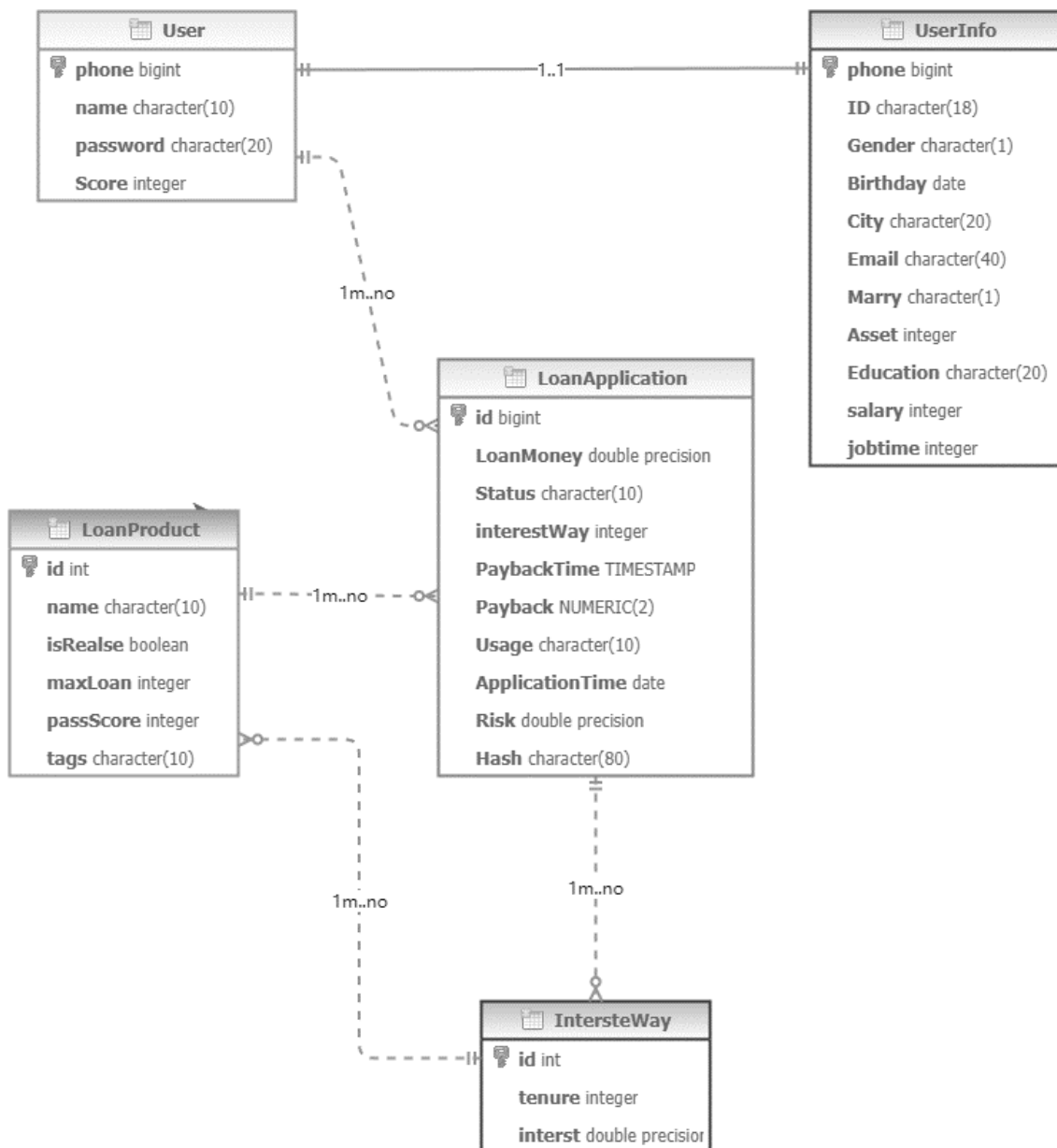


图 13：数据库 E-R 图模型（核心）

其中，用户表是贷款管理系统中最重要的关系表之一。它记录了用户四个方面的信息：登录信息、实名认证信息、基础信息和详细信息。登录信息是使用 app 所必需的，包括手机号码、用户名和密码，在用户注册时，这些信息被保存在用户表中。在用户登录时，应用程序将用户输入的密码与用户表中对应元组的数据进行比对，以确保用户账户的安全性。

实名认证信息包括身份证号和真实姓名，可以确保同一个人只能通过一个账

第二章 针对复杂工程问题的方案设计与实现

号申请贷款，防止重复贷款。基础信息性别、生日、所在城市、电子邮件地址，详细信息婚姻状况、财富、教育程度、职业、工资和工作时间。这些信息被用于计算用户的信用分，从而决定是否能够获得贷款。

数据库设计规范有序，符合四大范式：

- (1) 第一范式（1NF）：确保每列保持原子性，也就是列不能可分。在数据库中，每个类都有自己的属性和字段，这些字段是不可再分的，因此符合第一范式。
- (2) 第二范式（2NF）：在第一范式的基础上，非主键列必须完全依赖于主键，而不是部分依赖。在数据库中，每个类都有唯一的主键（例如 UserID, ProductID 等），并且其他字段都依赖于这些主键，因此符合第二范式。
- (3) 第三范式（3NF）：在第二范式的基础上，任何非主键列不能依赖于其他非主键列。在数据库中，每个类中的字段之间没有明显的依赖关系，因此符合第三范式。
- (4) 第四范式（4NF）：第四范式是针对多对多关系的规范化。在数据库中，没有明确的多对多关系，因此不需要考虑第四范式。

此外，为了满足更高的查询性能要求，我们不仅为常见查询和属性创建了视图，还通过索引进一步优化了数据检索效率。这样做不仅可以提高数据库查询速度和数据访问速度，还能减轻用户在大量查询操作下的负担。通过合理设计视图和索引，我们可以更好地满足各种类型的查询需求。同时，编写一系列的存储过程，对用户的贷款申请进行审核、审批，并更新用户的贷款状态。通过存储过程，我们可以将复杂的业务逻辑封装在数据库中，方便后续的维护和管理。

表 10：项目设计的索引、视图、存储过程

类别	示例	说明
索引列	UserID	用户 ID，用于快速查找用户信息
	LoanID	贷款 ID，用于快速查找贷款信息
	ApplicationDate	申请日期，用于按申请时间进行排序和筛选
视图	UserLoanView	整合用户信息和贷款申请信息的视图，方便快速查询
	LoanProductView	整合贷款产品和贷款申请的视图，方便匹配
存储过程	LoanApprovalProcedure	处理贷款申请审批的存储过程
	UserInfoUpdateProcedure	更新用户信息的存储过程
	PayBackProcedure	还款的存储过程

数据库安全性也是重要的设计规范因素之一，在实际的数据库开发中，我们进行了多方面的权限控制和安全管控。首先，需要严格控制对数据库的访问，只有经过授权的用户才能访问数据库，并且根据角色和职责设置不同的访问权限，删除接口需进行身份验证后才能调用进而删除数据库系统数据，此外，没有涉及全表删、全类删等关键删减操作的数据接口。其次，限制用户对数据的操作，禁止删除重要数据，限制对敏感字段的访问等。定期备份数据库，以防数据丢失或系统故障。

此外，为了避免脏读等问题的出现，保证数据的一致性和完整性，尽可能避免数据误操作带来的风险。通过精心整理了数据库事务的时序整理（时序图见图 14），设计事务时序，在数据库维护和管理方面将大大提升效率和准确性，降低了查询结果的出错率，保证了数据的可靠性和稳定性。

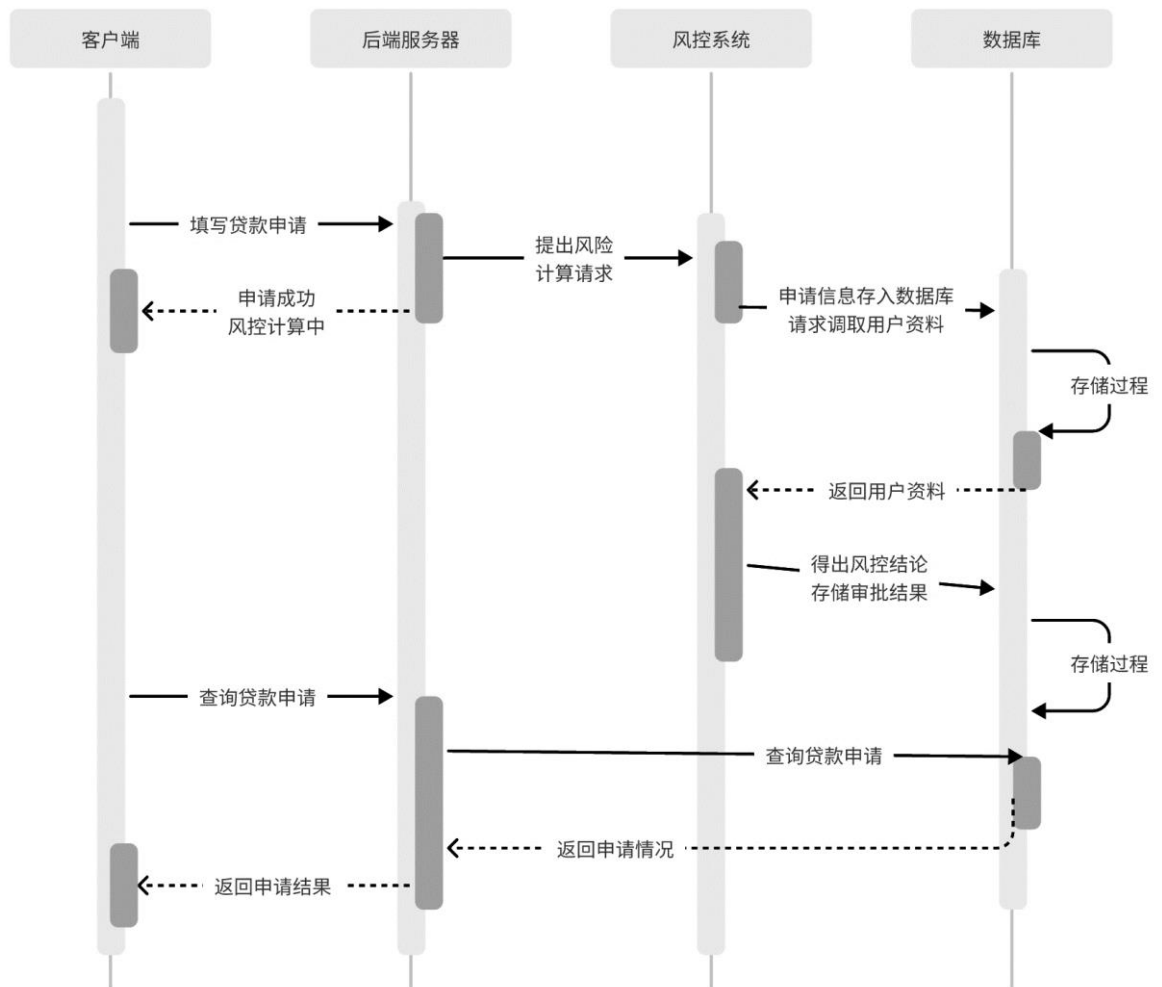


图 14：贷款申请与申请查询的时序图

假设有一个用户想要申请贷款，他通过客户端提交了一份贷款申请。后端服务链会接收到这个申请请求，并调用相应的服务进行处理。在这个过程中，后端服务链会先验证该用户的身份和权限，确保他是合法的用户且有资格申请贷款。然后，后端服务链会调用存储过程来对数据库进行操作，处理用户的贷款申请。

在存储过程中，我们会先对数据库进行查询操作，获取该用户的信用评分、资产情况等信息。然后，根据这些信息和贷款申请的具体情况，进行审批逻辑的处理。如果审批通过，我们会将审批结果存储到数据库中，并将申请结果返回给客户端；如果审批不通过，我们也会将结果返回给客户端。

通过这种方式，我们可以保证数据流的正确流向，确保数据库的数据逻辑处理正确性。

2.1.4 UI 设计

金融贷款产品的 UI 设计要注重用户体验和用户需求，让用户在使用产品时感到舒适、方便和安心，并能够清晰地了解贷款产品的各项细节和信息。设计原则包含设计原则

- 简洁明了：设计应简洁而不复杂，避免过多的视觉元素和信息，以突出核心内容。
- 用户友好：界面应易于理解 and 操作，遵循常见的交互模式，避免用户感到困惑。
- 直观性：导航和布局应直观明了，使用户能够快速找到所需功能。
- 一致性：UI 元素和设计风格在整个 APP 中保持一致，以提高用户体验。
- 适应性：设计应适应不同的设备和屏幕尺寸，确保在不同设备上都能获得良好的用户体验。

我们为贷款产品设计了如下的 UI 规范。

首先，颜色设定上，采用品牌识别度高的颜色，选取了较为常见的蓝色和紫蓝色作为主题色，以传达专业、可靠的形象。以营造稳重、信赖的氛围。深邃而典雅，符合产品整体调性。以黄绿辅助色辅助搭配，在深邃中有一丝灵巧。

其次，图标和按钮上。在设计中考虑图标的可读性和适应性，确保在不同设备上都能清晰呈现。按钮设计明确的按钮，引导用户进行操作。使用醒目的颜色和文字提示用户按钮的功能，以提高交互性。



图 15：整体色彩风格设定

同时，我们对前后端进行拆解，以组件化形式设计将应用程序拆分成一系列可重用的组件。通过统一的设计风格 and 独立的组件，组件化设计提高了用户体验和开发效率，提高重用性。

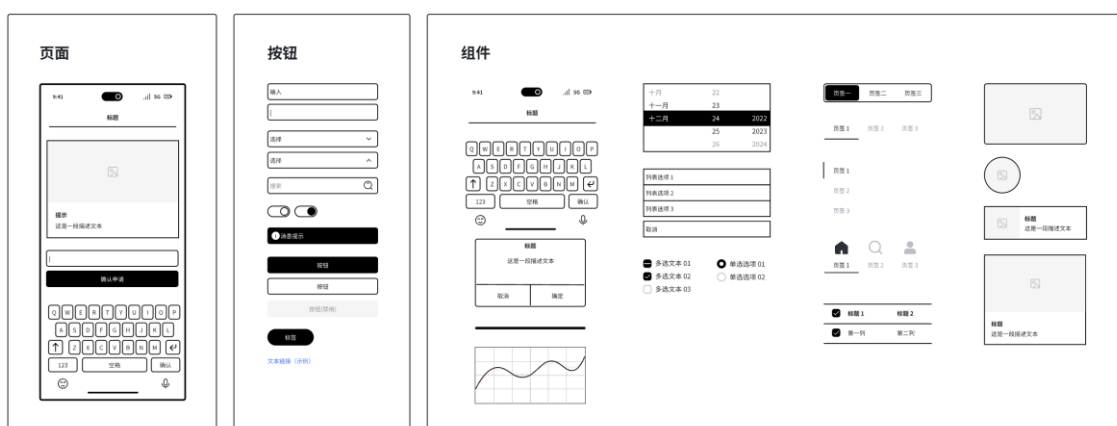


图 16：用户 APP 设计组件拆解（ElementUI）

利用相关软件，我们设计了如所示的基本页面——



图 17：用户端界面设计

第二章 针对复杂工程问题的方案设计与实现

图 17 展示了用户端的首页、还款页、我的页的界面设计图，整体设计美观大方。借款页面作为 APP 进入的第一屏，展示 APP 的核心功能和重要信息，如贷款申请、个人资料、我的申请等。使用明显的导航条和标签页，方便用户快速找到所需功能；贷款申请页包含贷款申请的详细信息和条件，如贷款金额、期限、利率等。提供清晰明确的选项和步骤引导用户完成贷款申请流程；还款管理页展示用户的还款计划和记录，包括每期应还款金额和还款日期。提供灵活的还款方式和提醒功能，帮助用户便捷地管理还款事务。

如下是我们的管理端页面——

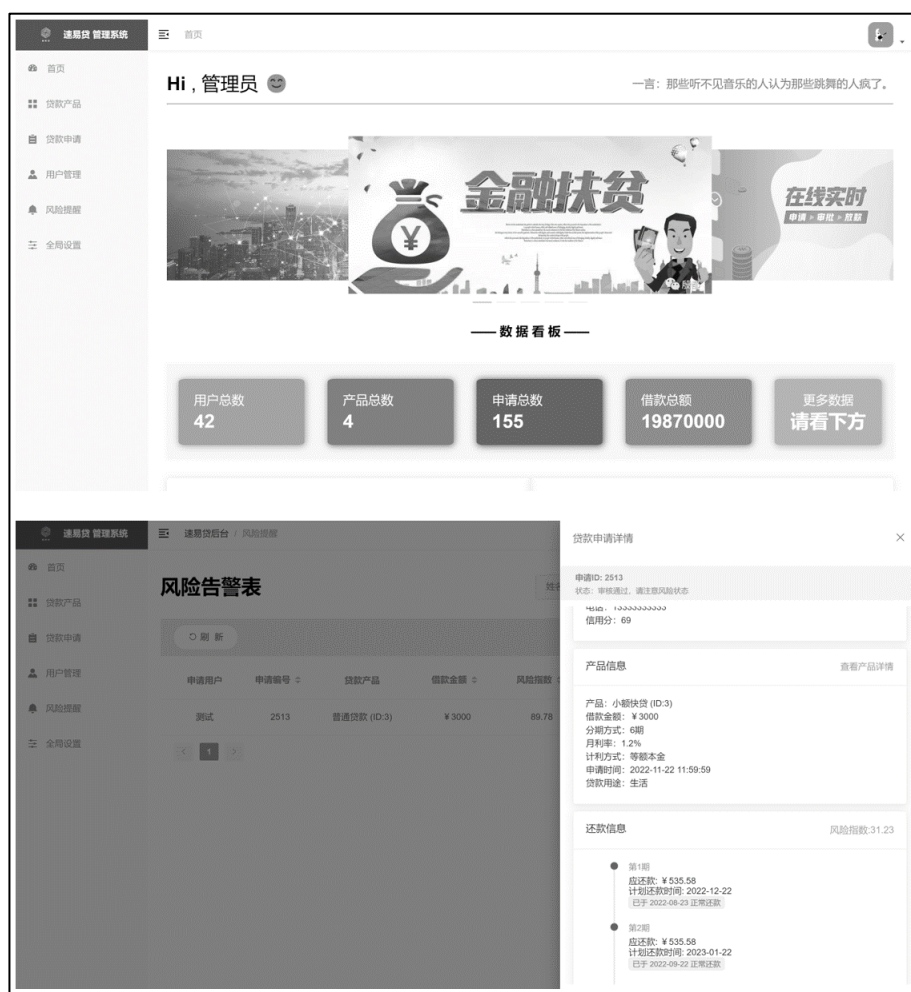


图 18：管理端界面设计

管理端界面设计在细节和整体架构上均注重突出核心功能，不仅在视觉效果上做到简洁明了，还在操作体验上力求高效便捷。

左侧分栏布局使得信息的分类和展示更为合理有序，使用户能够快速定位到所需信息。通过整合抽屉式和卡片式布局，我们将各个功能模块进行了精细的划分，

并确保每个模块之间的逻辑关系清晰明了。这种设计方式不仅提高了用户对管理系统的使用效率，还增强了用户对系统功能的理解和记忆。



图 19：大屏界面设计

大屏界面设计亦是如此，分离卡片设计突出重点，分散聚焦重心。

整个页面布局注重用户体验和功能性，元素设计都充分考虑到了用户的需求和习惯。例如，核心功能的按钮和文字说明简洁明了，易于辨识和理解，使用户能够快速找到并使用这些功能。

整体界面设计重视用户体验，突出核心功能、信息展示简洁明了、流程操作便捷、入口设计多样化、提醒提示适度，并具备脱敏安全保障。重点表单填写及时反馈，尽量减少用户对操作目标的记忆负荷（图 20）。这些反馈方式可以有效地引导用户更好地完成表单填写，并减少用户的操作难度和出错率。同时，及时反馈还可以提高用户对系统的信任度和满意度，从而提升用户的使用意愿。

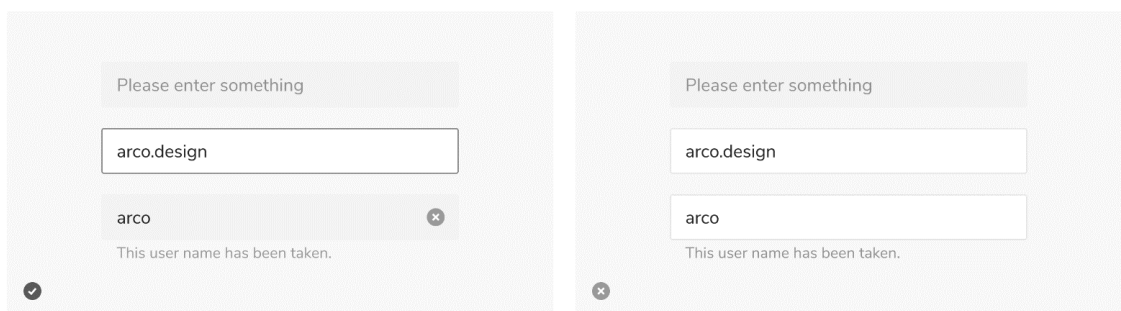


图 20：反馈性表单与普通表单

第二章 针对复杂工程问题的方案设计与实现

界面设计美观大气，视觉感受强，符合金融贷款产品 UI 特性，充分考虑用户需求，将核心功能放在显眼位置，并辅以适当的图标和文字说明，方便用户识别。

同时，保持信息展示的简洁明了，并将信息分类清晰地呈现出来，提高用户的阅读效率。多样化的入口设计，如小组同时在“借款”和“我的”提供贷款申请入口，方便用户根据自身需求快速找到相应功能。

2.1.5 流程设计

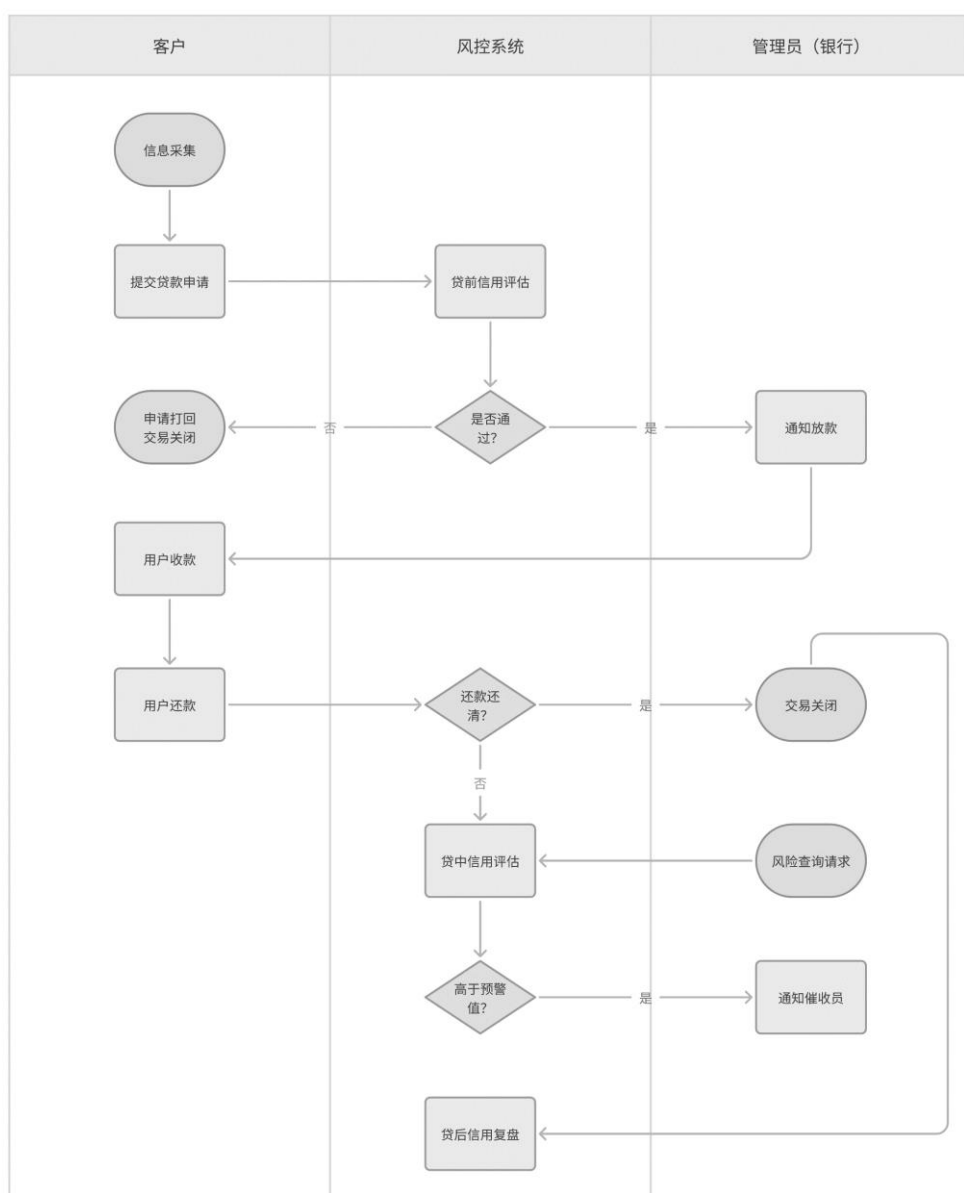


图 21：贷款主要流程泳道图

贷款风险管理和还款管理是一个复杂的系统工程，涉及客户、风控系统和银行管理员等多个环节的信息交流和协同操作。信息的流畅管控对于整个系统的高效运行至关重要。为了达到这一目标，我们需要将所有的信息流通过程梳理清晰，以泳道图图 21 的形式展示。

具体来说，我们可以将贷款风险管理的过程分为三个核心阶段：贷前信用评估、贷中风险评估以及贷后信用复盘。这三个阶段构成了整个流程的关键节点。在贷前信用评估阶段，客户需要提供包括基本信息、工作信息和收入状况等一系列数据，为后续的风险评估和评级提供依据。在此基础上，我们利用多种数据分析工具进行综合判断，完成对客户的信用评估和风险分级，从而为不同级别的客户提供定制化的授信方案和利率方案。这样的设计方式不仅能够帮助我们更好地理解各个步骤的具体内容，同时也使我们能够更加精确地掌握贷款业务的风险控制和运营流程。

2.1.6 行为接口设计

接口是前后端沟通的桥梁，良好的接口规范可以有效减少后期前后端联调的时间，增长开发效率。

为了确保接口设计的有效性和实用性，在开始设计之前就做好充分的需求分析和功能预判。首先，我们需要深入理解用户的需求，通过对用户可能的行为模式和操作习惯的预测，来确定哪些功能和操作需要被纳入接口设计的范畴。

		冷启动与产品选择阶段		借款阶段		还款阶段	
角色行为模型	用户行为	注册，填写基本信息	在借款页面进行产品选择	用户填写借款信息	用户查询借款状态	用户查看还款详情，并还款	
	管理行为	分析基本信息，给出基本信誉分	在管理端进行产品的增删查改	管理员可查看用户填写借款信息和审批情况		管理员可查看用户还款情况	管理员提醒用户还款
产品交互模型	前台行为	用户在指定页面填写基本个人信息。	用户点击借款按钮，页面进行展示	用户在指定页面进行借款信息的输入	用户点击指定按钮，可浏览历史借款记录	用户点击指定按钮，页面展示还款账单，用户选择后进行支付	管理员点击管理页指定按钮，发送提醒
	后台行为	存储基本信息	接口返回可展示的产品信息	存储用户的借款信息	接口正确返回相关信息	返回同步剩余还款信息	
	风控模型	运行风控模型，给出基本信誉分		风控系统根据申请信息和信誉分进行二分类判断		更新用户信誉分	更新用户信誉分

图 22：角色行为与产品交互模型

第二章 针对复杂工程问题的方案设计与实现

图 22 展示了角色行为与产品交互模型蓝图，初步构建了用户可能的动作流。我们对其中核心部分做细化，形成如下用户行为表格。

表 11 反映了用户可能的常见行为以及对应的系统响应——

表 11：用户常见行为与系统响应表

角色行为	系统响应	描述
角 色：个 人 用 户		
用户登录	返回登录结果	用户填写相关登录信息进行登录，系统对登录信息进行校验（判断密码正确性），返回登录的结果。
用户填写相关基础信息	返回保存结果	用户填写相关基础信息、实名认证等，系统判断信息是否合法、是否有遗漏，并返回判断结果。
用户浏览产品详情	返回产品数组	前端展示产品信息数据，提供操作按钮供用户选择
用户提交贷款申请	返回贷款申请号，告知是否成功	用户选择相关信贷产品，进行细化的分期选择后，提交申请。系统返回申请编号用于后续处理和查询。
用户还款	更新还款记录	用户还款时及时向数据库记录还款信息。系统会自动更新客户的还款信息，并计算相应的风险情况指数。
用户查询贷款申请	返回全部贷款申请信息	从数据库查询相关申请数据信息，并返回。
用户查询信用分	返回信用分	用户提出信用分请求，后台通过调取用户基础信息等计算信用分并予以返回。

表 12 反映了管理员可能的常见行为以及对应的系统响应——

表 12：管理员行为与系统响应表

角色行为	系统响应	描述
角 色：管 理 员		
管理员登录	返回登录结果	用户填写相关登录信息进行登录，系统对登录信息进行校验（判断密码正确性），返回登录的结果。
修改贷款产品	返回修改结果	管理员对相关贷款产品的利率、分期、阈值等进

信软学院进阶式挑战性综合项目 III 报告

		行设置，系统对管理员的设置数据库之中进行保存更新操作。
查询贷款申请	返回贷款申请信息	管理员通过贷款申请号查询某一信贷产品的贷款信息，系统在数据库进行查询处理
查询用户	返回用户信息	管理员通过手机号查询某一用户的个人信息，系统在数据库进行查询处理，返回用户的基础信息、实名信息、详细信息、贷款信息等
风险提醒	返回提醒结果	管理员提出请求，系统根据评分卡以及阈值判定是否为风险申请。并及时通过邮件提醒还款用户。
全局设置	返回设置结果	管理员对全局阈值（基准分、风险分）等进行设置，系统对管理员的设置数据库之中进行保存更新操作。

冗杂的接口设计、管理也是需要解决的一大问题，小组采用“模块-功能-接口”逻辑树推演设计接口，将接口按模块分大类、功能分小类，重用合并可复用接口。做到接口设计的精简、有序。

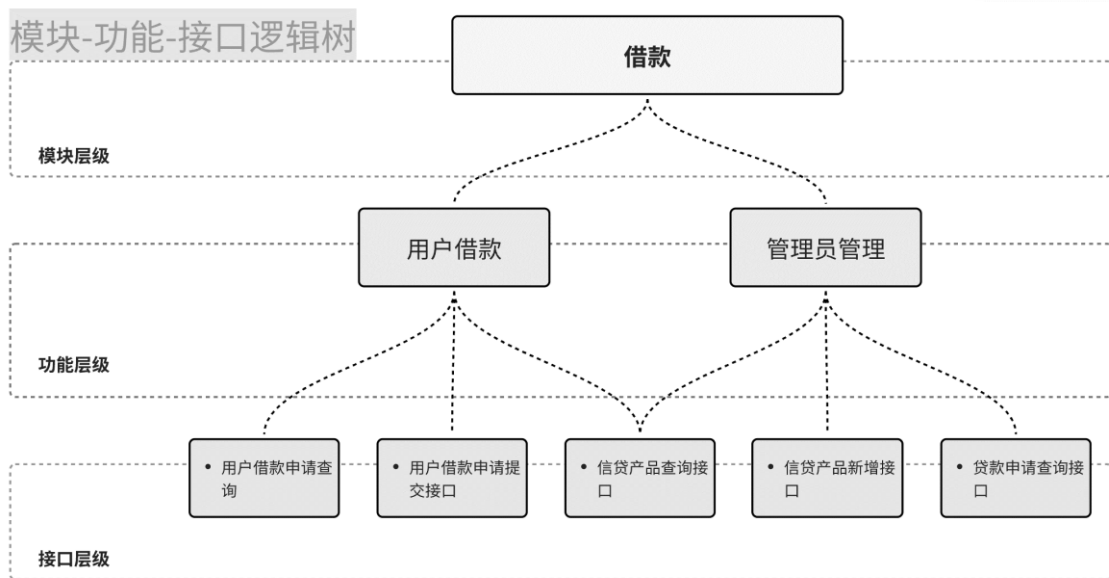


图 23：模块-功能-接口逻辑树（以借款为例）

实践表明，在后续开发中，通过这种接口逻辑树的构建可以有效减少因为接口冲突导致的代码编写调整时长。有效提高了开发效率，加快开发进度。

基于以上行为响应和逻辑树推演，我们对相关接口按“增、删、改、查”进行了分类的聚合整理。设计的接口如图 24 所示：



图 24：接口总览

接口设计遵循一致的 RESTful 架构风格，这种风格借助统一的资源标识（URI）来表示服务中各种操作，并通过不同的 HTTP 方法（如 GET、POST、PUT、DELETE）对资源进行处理。这一设计理念有助于提升系统的可扩展性、可移植性、可重用性和可靠性。

为了降低前后端的耦合度、提高代码可读性和可维护性，我们根据业务逻辑和数据结构设计了 API 请求与响应的数据格式。同时，我们采用<code,msg>模型来生成响应的 JSON 数据，确保了风格的一致性。

在考虑接口性能优化方面，我们采用了诸如缓存数据、Cookie 等技术，以减少接口调用，并加强用户登录过程的安全性。

我们通过使用国产 Apipost 接口管理设计软件，我们对软件进行了接口设计。在设计过程中，我们遵循了设计规范。

- (1) **一致性的交互模型：**无论是获取资源还是删除资源，都使用相同的交互模型<code,msg>，便于开发者理解和记忆。
- (2) **简单的 URI：**URI 应该尽量简洁明了，避免重复和冗余。我们采用驼峰命名法进行设计编写，名称统一。
- (3) **标准的 HTTP 方法：**通过标准的 HTTP 方法（如 GET、POST、PUT、DELETE 等）来完成对资源的各种操作，例如创建资源使用 POST，获取资源使用 GET，更新资源使用 PUT，删除资源使用 DELETE。各个方法之间不混用、不交叉。
- (4) **合适的状态码：**我们建立了正确响应 HTTP 状态码，比如 200 OK，404

Not Found, 500 Internal Server Error 等。状态码设立符合计算机网络规范与相关标准。

- (5) **明确的错误处理：**当出现错误时，返回具有明确提示的错误信息和状态码，便于客户端进行处理。

以下是其中一个接口设计的示例——

接口 1：获取完整贷款产品的接口设计范例

获取完整贷款产品

- **接口状态：** ☒ 已完成
- **接口 URL：** localhost:8080/loanProducts/loan_products
- **云端 MockUrl：** https://console-mock.apipost.cn/mock/11bcefcc-0ac0-40a4-b91c-a0121627508c/loanProducts/loan_products
- **Content-Type：** application/json
- **请求方式：** GET

接口备注：

用于获取完整贷款产品信息，该接口同时运用在 app 端与管理端

请求体参数说明：

参数名	示例值	参数类型	是否必填	参数描述
无请求参数				

返回参数说明：

(200) 成功

参数名	示例值	参数类型	参数描述
code	0	int	结果标识(0:成功；其他:服务器响应错误)
msg	-	object	返回的结果响应体
products	-	object	贷款产品对象
loanProductId	1	int	贷款产品 ID
loanProductName	普通贷款	string	贷款产品名称
isRelease	true	boolean	是否发布
loanProductMaxMoney	1000	int	最大借款限额
AcceptScore	30	int	审批阈值
tag	12	string	产品标签编号

第二章 针对复杂工程问题的方案设计与实现

interestways	-	object	产品利率与对应分期对象
time	6	int	分期
rate	1.2	float	月利率

(404) 失败

参数名	示例值	参数类型	参数描述
code	1	int	结果标识(0:成功; 其他:服务器响应错误)
msg	-	object	返回的结果响应体
info	-	object	失败消息

返回示例:

(200) 成功

```
{
  "code": 0,
  "msg": {
    "products": [
      {
        "loanProductId": 1,
        "loanProductName": "贷款",
        "isRelease": true,
        "loanProductMaxMoney": 1000,
        "AcceptScore": 30,
        "tag": "12"
      }
    ],
    "interestways": [
      {
        "loanProductId": 1,
        "time": 2,
        "rate": 1.1
      }
    ]
  }
}
```

(404) 失败

```
{
  "code": 1,
  "msg": {
```

```

    "info": "serve error"
  }
}

```

通过接口设计，确立了各个接口的调用规范，为前后端的并行设计开发打下基础。

2.1.7 信息采集维度设计

在风控领域中，建立一套科学、合理、详实的信息采集方案是非常重要的。这套方案不仅可以为风控模型提供有效的输入，而且还可以为模型训练、调整以及模型效果的评估等过程提供必要的技术支持。因此，科学地设计信息采集方案是决定风控模型优劣的关键所在。

不同类型的信息有着不同的重要性和价值，因此，在确定要收集哪些信息时需要权衡好各方面的影响因素，以确保所收集的信息既有代表性又具备足够的细节，这样才能更好地反映用户的实际情况。小组基于对微众银行的调研，总结了微众银行的信用评分体系。

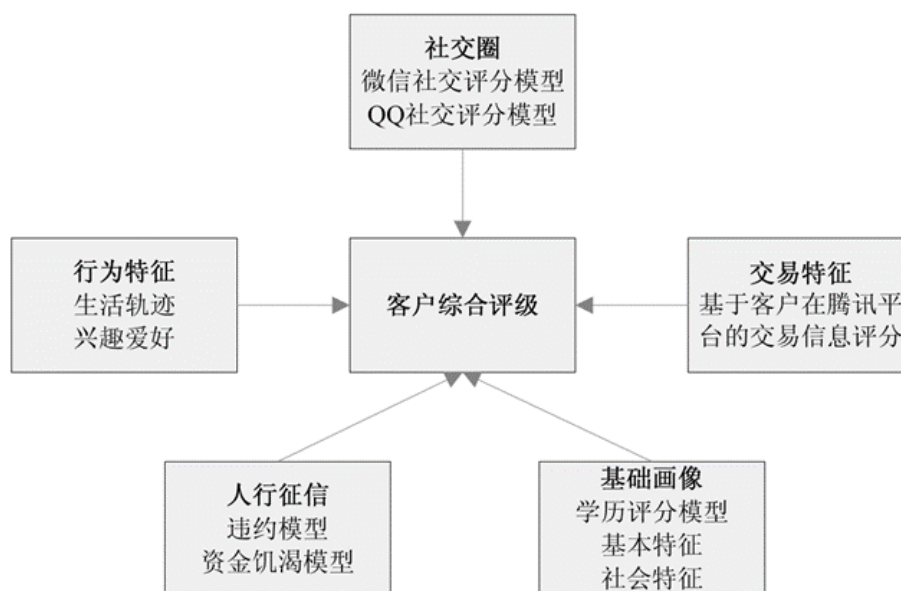


图 25：微众银行客户信用评分体系

小组广泛收集并组织了大量的参考文献资料，以帮助我们对贷款风险管理和还款管理领域的专业知识有一个全面的理解。在《信用评分卡在个人信贷中的应

用探究》的文章中，作者强调"5C 原则"，其中包括 Character（品行）、Capacity（偿付能力）、Capital（资产）、Collateral（担保物）和 Condition（环境）。基于这项原则，我们的小组成员查阅了更多相关文献和参考资料，并进一步完善了我们的数据收集框架。设计了三维信息采集维度。

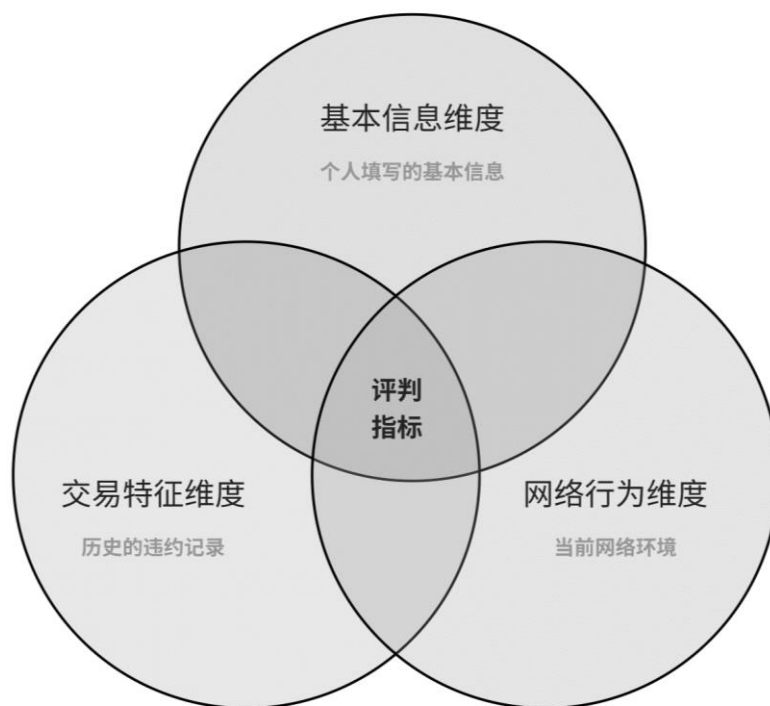


图 26：三维信息维度

三维信息维度分别从三个方面描述了用户的基本特征信息。

基本信息、交易特征、网络行为的三维信息维度分别从三个方面描述了用户的基本特征信息。

基本信息维度主要包括用户填写的基本信息，如姓名、性别、年龄、职业等。这些信息能够反映用户的个人特征和社会背景，为后续的信用评估提供基础数据。

交易特征维度主要涵盖了用户的交易历史和行为特征。通过分析用户的交易记录，可以了解用户的支付能力以及信用状况。这些信息对于判断用户的信用风险和还款能力具有重要意义。

网络行为维度则包括了用户的网络环境和行为轨迹。通过监测用户的网络活动，可以获取用户的网络行为特征和社交网络关系。这些信息能够反映用户的兴趣爱好、社交圈子以及网络信用状况，为全面评估用户的信用等级提供有力支持。

综合这三个维度的信息，可以对用户的基本特征进行全面描述，从而为后续的信用评估和风险控制提供有力支持。我们将每个维度详细采集指标列举如表 13 所示。

表 13：用户数据维度详细信息表

数据维度	小类	举例
基本信息维度	基础信息	学历、年龄、城市评级等
	详细信息	工资、职位、教育程度、婚姻状况、车辆状况等
	网络行为维度	IP 地址、地理位置特征、注册时间等
	交易特征维度	违约次数、逾期金额、itv 等

信息采集的具体方案将在后续章节详细论述，此处不再论述。

2.1.8 安全设计



图 27：安全架构设计

数据安全是贷款系统设计的重点之一，图 27 展示了这个系统的安全架构设计思路。

在展示层，主要涉及用户界面的安全性和脱敏性。这一层的主要目标是确保只有经过授权的用户才能访问系统，因此前端层面会对页面进行路由守卫（即防止未登入用户直接进入系统）；其次，对于一些核心的隐私信息，如身份证号、手机号等，我们采取数据脱敏，以特殊字符进行信息传递。

在行为层，主要关注的是用户行为的安全性。在接口传递至后端过程中，会对行为进行相应审计、审查、监控，以日志形式计入数据库中。例如，用户申请贷款时，会同时上传用户的 IP 和地理位置信息用于信用评估与审核。敏感操作也会进行二次身份验证（包括指纹验证、验证码验证）。

在数据层，安全架构主要关注的是数据的机密性、完整性和防篡改。这一层的目标是确保数据不会被未经授权的人员访问，通过数据库的角色分类和权限控制，保证数据的正确访问。

同时，我们还引入了哈希指纹，哈希指纹是一种数据完整性保护机制，它通过将数据哈希编码为一个标识符（即“指纹”）来工作。这个哈希值是对数据内容的摘要，任何对数据的更改都会导致哈希值的变化，系统可以验证这个指纹是否与之前存储的正确指纹匹配。如果指纹不匹配，那么系统就知道数据已经被篡改或伪造。因此，哈希指纹可以用来验证数据的完整性和真实性。在我们的贷款系统中，哈希指纹可以用于验证贷款申请信息的完整性和真实性，防止数据被篡改或伪造。详细的方案推演过程将会在后续讲述。

通过这样的安全架构设计，可以尽可能保证系统数据的安全性。

2.2 针对完整系统工程的方案实现

基于综合项目课题设计方案，根据软件系统的应用场景，我们选择合适的开发环境、工具与技术标准进行软件系统的开发。本节主要讲述的是前后端系统的基础方案实现。评分卡模型、信息采集等重难点工程将在下一节讲述。

2.2.1 开发工具、环境与规范概述

“工欲善其事，必先利其器”。良好的开发工具是项目开发的基石。以下是该项目使用的开发工具、环境、框架、核心库、技术标准：

1 前端开发工具：

- VScode：开源的代码编辑器，支持多种编程语言和插件。
- HbuilderX：集成开发环境（IDE），支持 Vue 等前端开发语言。
- Gitee：代码托管平台，支持 Git 版本控制和协作开发。
- ApiPost：API 接口测试工具，提供 API 测试、文档管理、Mock 服务等功能。
- Vue：流行的 JavaScript 框架，用于开发用户界面。
- ElementUI：基于 Vue.js 的 UI 组件库，提供丰富的组件和样式。
- UView：高效的小程序 UI 框架，提供丰富的组件和样式。
- Uniapp：基于 Vue.js 的跨平台框架，可同时开发小程序、H5、APP 等多端应用。
- Axios：基于 Promise 的 HTTP 客户端，用于在前后端进行数据交互。
- Moment.js：处理日期和时间的 JavaScript 库。
- emailjs：通过 SMTP 协议发送电子邮件的 JavaScript 库。
- 5+runtime 标准：HTML5 等技术的应用规范，实现了跨平台互通。

2 后端开发工具：

- IntelliJ IDEA：Java 的集成开发环境（IDE），提供代码编辑、调试、部署等功能。
- Ktor：Kotlin 语言的 Web 框架，提供高性能和扩展性。
- Ktorm：Kotlin 语言的 ORM 框架，用于操作数据库。
- PostgreSQL：开源的数据库管理系统，支持关系型数据库。
- Gradle：基于 Groovy 语言的自动化构建工具，用于编译、测试、打包等操作。
- Gson：Google 提供的 Java 对象与 JSON 数据之间的转换库。
- Druid：阿里巴巴开源的数据库连接池，具有监控、统计等功能，性能相比传统方案更高。

该项目使用一系列前端和后端工具和框架进行开发，VScode、HbuilderX、Vue、ElementUI（前端）和 IntelliJ IDEA、Ktor、Ktorm、PostgreSQL、Gradle、Gson、Druid（后端），助力开发人员提高开发效率、代码质量和稳定性的目标。

在前端方面，工具和框架提供了诸如页面设计等功能，帮助开发人员更高效地开发和维护用户界面。同时，像 UView 和 Uniapp 这样的跨平台框架还可以实

第二章 针对复杂工程问题的方案设计与实现

现小程序、H5 和 APP 的统一开发标准，实现代码复用并加快迭代速度。

在后端，借助这些工具和框架，开发人员能够完成 Web 服务的开发和管理任务，包括路由配置、数据库连接等。特别是在 Kotlin 语言的应用中，不仅提升了开发效率，还确保了代码的安全性和可读性。

项目开发环境如表 14 所示。

表 14：软件开发环境

端类型	工具	版本	说明
前端	Windows	Windows11 22000.2057	操作系统
	VSCode	2023	开发工具
	node	16.x 及以上	JS 运行环境
	npm	9.x 及以上	包管理工具
	Chrome	100.x 及以上	浏览器
后端	SDK	Amazon Corretto version 11.0.19	工具包
	IntelliJ IDEA	2022.3.3	开发工具
	PostgreSQL	15	数据库
	pgAdmin 4	6.19	数据库管理工具

同时，项目采用了各类技术和规范，如 5+runtime 标准、HTTP 协议、RESTful API 等，以确保产品的可扩展性、安全性以及与其他系统的兼容性。其中，采用 RESTful API 有助于简化不同系统间的数据交互，增加互操作性和代码重用率，也有利于开发人员管理和维护系统的接口。

表 15：项目涉及的规范标准

规范标准	描述	优势
HTML5+标准	针对 APP 开发的一套通用技术标准，通过 HTML5+可以方便调用 APP 原生 API	可复用、可迁移。支持跨平台
Restful API	针对接口设计和开发的一套标准，利用 Restful API 封装接口可以使得接口数据结构清晰明了	可伸缩、可移植
5+RunTime 标准	5+RunTime 标准是针对 APP 原生权限调用的一套方案，通过标准化	性能好、跨平台

ArcoDesign 设计原则	接口可以进行手机信息获取
	字节跳动推行的前端设计开发原则，符合这个原则的组件应用能遵从美观度、用户体验感。从用户习惯，信息反馈直观。
	针对数据库设计开发的一套标准，
四大范式	符合四大范式的数据库模型可以保证规范性、安全性、数据库的一致性和完整性。

通过这些开发环境、工具、规范，可以提高开发效率，提升用户体验。

2.2.2 数据结构构建概述

项目的中心数据是用户还款信息，在还款成功申请之后，这个关键数据会被用于执行许多功能，包括每月还款金额的计算、用户还款可能性的预测以及还款提醒的通知等等。为了满足这些需求，我们需要一个易于存储且具有高性能计算能力的数据结构来处理此核心数据。我们对还款信息的数据结构设计采用如下实现方案——

代码 1：还款信息的数据结构

```
"payback_money": ["535.58", "535.58", "535.58"]
"actual_payback_time": ["1671504796", "1674745996", "-1"]
"scheduled_payback_time": ["1671534796", "1674213196", "1676891596"]
```

如代码 1 所示，我们定义了三个数组来表征用户的三项核心还款信息。注意，这些数组中的元素都为字符串类型，以便于它们被存入后台数据库。其中一个数组“payback_money”代表用户每一期需要偿还的款项，每个数据保留两位小数，其数量与用户申请贷款的数量相等，这样就可以确保每一笔还款都能对应每一期。在前端申请阶段，可以通过下面的代码来实现这一目标：

代码 2：计算每期还款金额代码

```
function calcEqualPay(amount, rate, periods) {
    var monthlyPayment = amount * rate * Math.pow(1 + rate,
periods) / (Math.pow(1 + rate, periods) - 1);    // 每月还款金额
    var payments = [];                            // 用于存储每期还款的金额数组
    for (var i = 0; i < periods; i++) {
        var interest = amount * rate;            // 本期利息
```

第二章 针对复杂工程问题的方案设计与实现

```
var principal = monthlyPayment - interest;    // 本期本金
amount -= principal;                          // 剩余本金
payments.push(monthlyPayment.toFixed(2));
// 将本期还款金额加入数组
}
return payments;
}
```

“实际还款时间”是由数组 "actual_payback_time" 来标记的。用 -1 表示尚未偿还本期欠款，其他非 -1 数字则表明已偿还过。这里我们将“时间戳”作为表示还款日期的方法引入进来，时间戳是一种表达距离固定起点（比如 1970 年 1 月 1 日凌晨 0 点 UTC）有多少秒钟或毫秒流逝的方法。

同理，数组“scheduling_payback_time”标识用户应该还款的计划时间，其在申请时通过 momentjs 库即可实现计算。通过比较“scheduling_payback_time”和“actual_payback_time”即可实现判断用户本期是否逾期。判断的方法流程如图 28 所示。

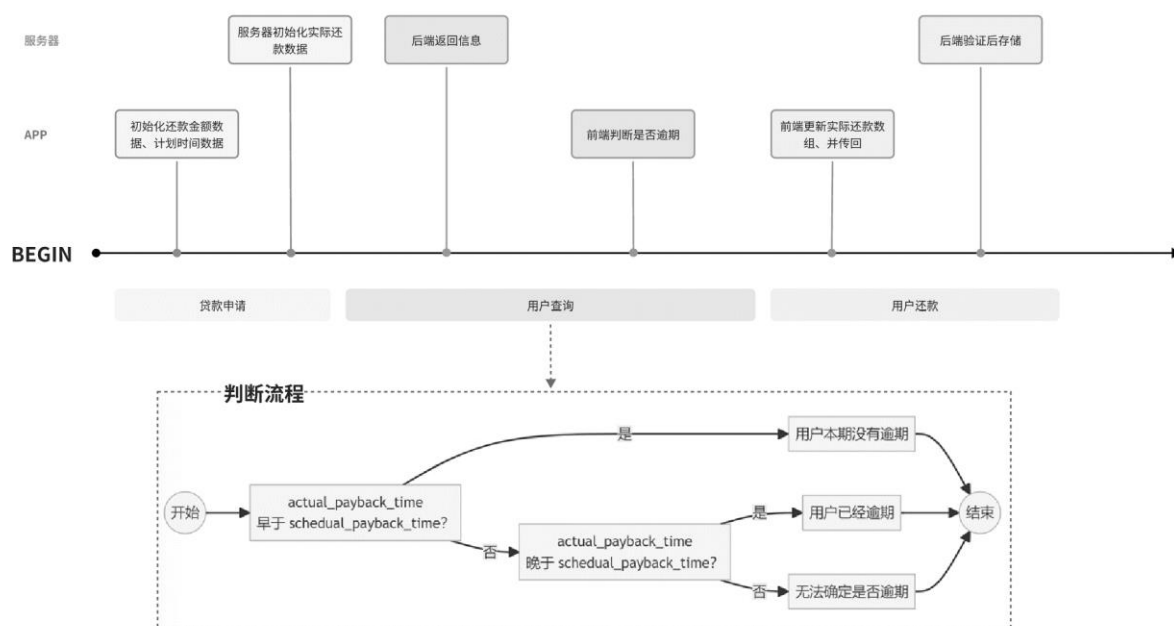


图 28：代码中时间戳的流程控制

通过对时间戳进行合理的设计和应用，我们能够有效地优化数据存储并提高系统的整体性能。具体而言，在数据结构方面，时间戳可以帮助我们极大地减小存储的需求，因为每个数据项只需要一个唯一的标识符即可，而无需像传统的方

法那样为每条记录都分配额外的空间。此外，时间戳在前端应用程序中也非常容易处理，因为它可以用各种常见的编程语言来表示，比如 JavaScript、Kotlin 等。这使得开发者可以在客户端进行大量的计算和比较，从而减轻了服务器的压力，也提高了系统的响应速度。

此外，对于过期的判定，由于时间戳很容易被转换成用户友好的格式，所以我们可以将其作为一项任务交给前端来完成。这意味着当用户需要查看或修改某个数据项时，系统只需向服务器请求当前的时间戳并与数据库中保存的时间戳进行比较即可确定该数据是否已经过期，这样就可以节省宝贵的算力资源，提高服务器性能。

2.2.3 前端框架及关键代码分析

前端主要采用 Vue 作为主要框架，它为我们提供了五级基本的模块层次，分别是静态资源存储、组件、视图展示、前端路由跳转以及数据管理和工具函数。每一个核心表或数据类型都会对应一套完整的 assets、componets、views、router、store、utils 进行控制。各个模块的依赖过程如图 29 所示。

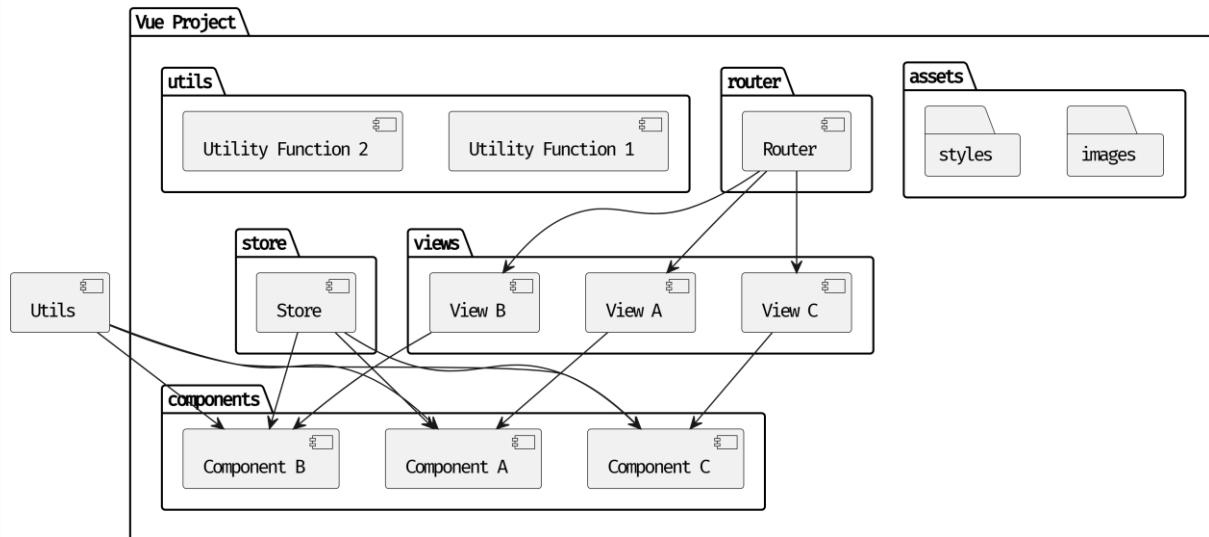


图 29：Vue 框架文件结构（核心模块依赖图）

下面对一些核心的代码作具体的分析

（1）还款清单页面布局

还款清单页面展示是用户浏览还款详情的核心界面，页面采用了标准流的上下布局搭建整体架构，符合产品 UI 设计规范。

在实现上，页面通过三个 `view` 标签进行分割，总体由三部分组成：顶部导航、信贷产品展示和非固定组件。

我们通过使用 Vue 的响应式语法进行模板的渲染，其中主要用到了如下几个编码特性——

- **v-for** : Vue.js 提供的一种模板语法，用于渲染数据和循环遍历数据数组或对象。它使得前端开发者可以通过简单的语法结构，在 HTML 中直接渲染数据，并且从根本上解决了前端代码编写时的繁琐、冗杂和重复问题。
- **v-if** : Vue.js 中常用的逻辑判断指令，通过该指令实现了空页面的判断。
- **flex 布局**: 一种流布局，可以通过将容器的子元素按比例分配空间来实现灵活而自适应的页面布局。简化了页面的布局过程，并且方便响应式设计。本段代码的导航栏即进行了 flex 布局设计。

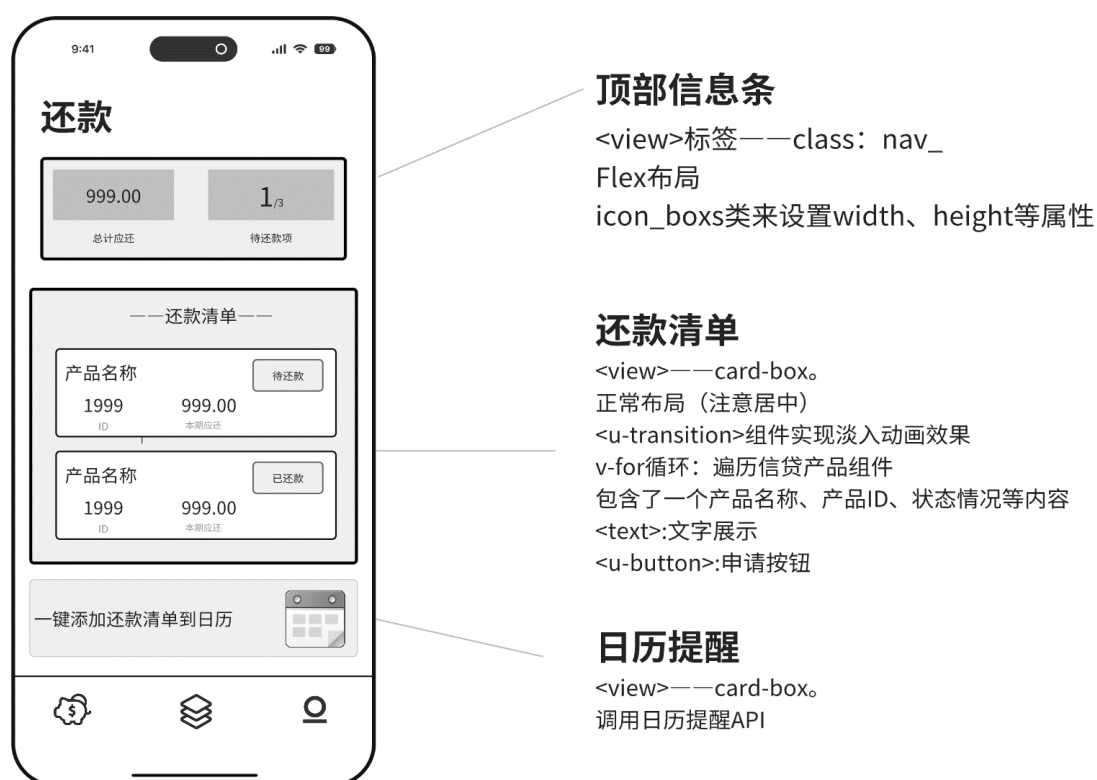


图 30：还款页页面布局

页面结构代码如下所示——

```
<template>
<view>
  <view class="back_ground"></view>
  <view class="big-title-box">
    <view class="title_" style="margin-left: 30rpx;">还款</view>
    <!-- 本月应该还款 -->
    <view class="sum">
      <!-- 创建一个视图元素，包含两个子视图元素，分别表示待还款项目的数量和总金额 -->
      <view class="sum_card">
        <view class="sum_block">
          <view class="sum_block_inner">
            <view class="sum_block_inner_title">待还款项目</view>
            <view class="sum_block_inner_num">
              {{ all_application.length - hasbenn_payback }} /
            {{ all_application.length }}
            </view>
          </view>
          <view class="sum_block_inner">
            <view class="sum_block_inner_title">本期总计待还款</view>
            <view class="sum_block_inner_num">
              ¥{{ total_payback }}
            </view>
          </view>
        </view>
        <u-alert title="点击查看全部贷款申请 >" type="warning"
@click="nav_T0('/pages/tabbar/lend/my_application/my_application')"></u-
alert>
      </view>
    </view>
  </view>
  <!--还款清单视图列表 -->
  <view class="pay-box">
    <view class="pay-box-tit">本期还款清单</view>
    <!--使用 V-for 读取 -->
    <view v-for="(item) in all_application" :key="item.application_id"
class="pay-card" @click="nav_application_info(item.application_id)">
      <view>
        <view class="pay-card-tit-box">
          <view class="pay-card-tit">{{ item.product_name }}</view>

```

第二章 针对复杂工程问题的方案设计与实现

```
<u-tag :text="item.now_status" :type="item.tagtype"
@click="nav_application_info(item.application_id)"></u-tag>
</view>
<view class="pay-card-info">
  <view class="pay-card-info-box">
    <view class="pay-card-info-box-title">申请号</view>
    <view class="pay-card-info-box-
num">{{ item.application_id }}</view>
  </view>
  <hr />
  <view class="pay-card-info-box">
    <view v-if="item.now_status === '已逾期'" class="pay-card-info-box-
title">逾期首期应还: </view>
    <view v-else-if="item.now_status === '本期已还'" class="pay-card-
info-box-title">本期已经还款: </view>
    <view v-else class="pay-card-info-box-title">本期应该还款: </view>
    <view class="pay-card-info-box-num">{{ item.shouldpay }}</view>
  </view>
  <hr />
</view>
</template>
```

同时,通过一些封装的非固定组件(如返回顶部、无贷款申请)来实现对不同用户不同情形的操作行为处理。提升了用户的信息交互性,用户得到的反馈性较强。符合特定反馈的特性。

(2) 还款日历提醒

还款日历提醒是用户手机端的一大特色功能。通过日历提醒的方式,不仅能帮助用户合理安排资金,确保按期还款,避免逾期带来的经济损失,还能提升用户的金融素养,树立正确的消费观念。

为了兼容跨平台的日历需求,我们采用 Uniapp 统一封装的日历接口进行开发,利用 plus 库调用 Android 方法,实现日历和提醒的添加。

代码 4: 日历提醒调用

```
// 导入所需类
import { plus } from '.....';

// 定义添加事件到日历的方法
function addEvent_calendar(title, desc, time, index) {
  // 定义日历、事件、提醒的 URL
```

```
const calanderURL = 'content://com.android.calendar/calendars';
const calanderEventURL = 'content://com.android.calendar/events';
const calanderRemiderURL = 'content://com.android.calendar/reminders';

// 获取日历 ID
let calId;
let userCursor = plus.android.invoke(plus.android.runtimeMainActivity(),
'query', Uri.parse(calanderURL), null, null, null, null);

// 查询日历数量
if (userCursor.getCount() <= 0) {
    // 如果没有日历, 初始化日历
    this.initCalendars();
} else {
    // 获取最后一个日历 ID
    plus.android.invoke(userCursor, 'moveToLast');
    calId = userCursor.getCount();
}

// 构建事件内容
const events = new ContentValues();
events.put('title', title);
events.put('description', desc);
events.put('calendar_id', calId);
events.put('dtstart', time * 1000);
events.put('dtend', time * 1000);
events.put('hasAlarm', 1);
events.put('eventTimezone', 'Asia/Shanghai');

// 插入事件
const newEvent =
plus.android.invoke(plus.android.runtimeMainActivity().getContentResolver(),
'insert', Uri.parse(calanderEventURL), events);

// 获取事件 ID
const id = plus.android.invoke(newEvent, 'getLastPathSegment');

// 构建提醒内容
const values = new ContentValues();
values.put('event_id', id);
```

第二章 针对复杂工程问题的方案设计与实现

```
values.put('minutes', '1440');
values.put('method', '1');

// 插入提醒

plus.android.invoke(plus.android.runtimeMainActivity().getContentResolver(),
'insert', Uri.parse(calanderRemiderURL), values);

// 提示插入事件成功
uni.toast('插入事件成功, 可打开本地日历查看');
}
```

在上述代码中, 我们使用了 `plus` 类作为调用基础。`plus` 类是一个功能强大、易用、可扩展的库, 用于实现与 `Android` 系统的交互, 如获取联系人、日历、文件管理等。通过这些方法, 开发者可以在前端实现对 `Android` 设备的操作, 提高开发效率。

`plus` 类具有跨平台和跨浏览器的特点, 可以在多种设备和环境下运行, 提高了代码的复用性。同时, `plus` 类提供了丰富的 API, 使得开发者能够轻松调用 `Android` 系统的功能。

代码流程图如下所示。

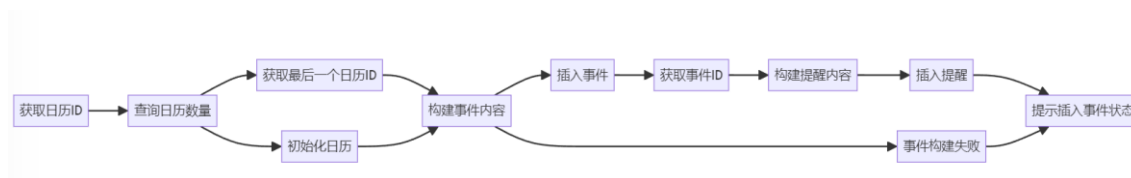


图 31: 日历提醒代码流程图

(3) 贷款申请提交

贷款申请提交过程复杂, 包括前端表单预判断、网络行为特征获取、网络请求发送和异步处理等多个环节。用户填写完信息并提交后, 触发运行提交函数 `confirmApplication()`, 以下是其执行流程。:

- 1 验证码验证或指纹认证: 对用户进行身份核验
- 2 获取位置信息: 若进行了定位, 则会调用 `uni.getLocation()` 方法获取用户当前位置的经纬度, 并将结果记录到 `position` 对象中;
- 3 获取 IP 信息: 调用 `getApp().glb_getIP()` 方法获取用户的 IP 信息
- 4 发送 POST 请求: 使用 `uni.request()` 方法, 向服务器发送了一个 POST 请求, 传递了包括用户个人信息、定位信息和 IP 信息等数据。请求成功后, 我

收到了包含申请编号的响应结果。

5 处理响应结果：根据响应结果来进行相应操作，如果请求成功，则跳转到申请成功页面，完成申请。

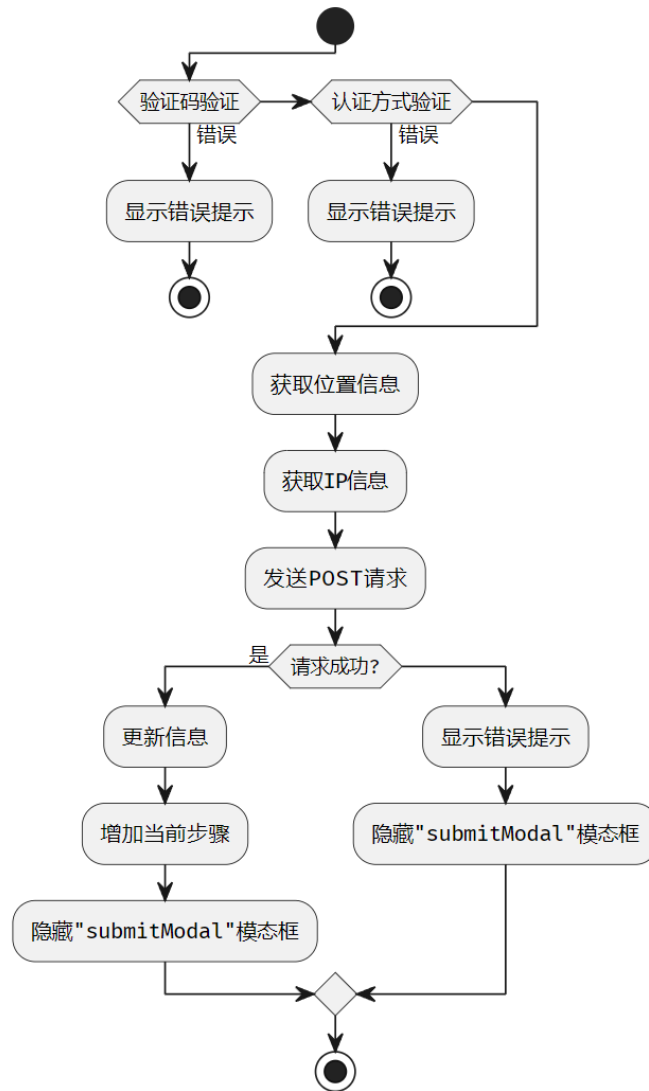


图 32：贷款申请提交逻辑流程图

以下是源代码：

代码 5：申请提交代码段

```

async function comfirmApplication() {
  try {
    // 验证码验证
    if (code.codeSend && code.codeValue !== code.trueCode
    && !code.fingerRes) {
      showToast('error', '验证码错误');
    }
  }
}
  
```

```

    return;
  }
  // 认证方式验证
  if (!code.codeSend && !code.fingerRes) {
    showToast('error', '请选择任意一种方式进行认证');
    return;
  }
  // 获取位置信息
  let position = {};
  #ifdef APP-PLUS
  position = await getApp().glb_getLocation();
  #endif
  // 获取 IP 信息
  const myipinfo = await getApp().glb_getIP();
  // 发送 POST 请求
  const res = await uni.request({
    url: `${getApp().globalData.ip}/loanProducts/submit_loan`,
    method: 'POST',
    data: {
      phone: parseInt(uni.getStorageSync('syd_usr_phone')),
      loanProductId: loadProduct.loadProductId,
      applyTime: moment().unix(),
      calcuInterestWay: infoStep1.usrChooseInterestCalculateWay,
      rate: infoStep1.usrRate,
      time: infoStep1.usrFenqi,
      loanAmount: parseInt(infoStep1.usrLoadMoney),
      payMoney: infoStep2.usrPayBackEachMonth.map(item =>
item.toString()),
      payTime: infoStep2.usrPayBackTime.map(item => item.toString()),
      usage: infoStep1.usrUsage,
      ip: myipinfo,
      position: position
    },
    timeout: 3000 // 超时
  });
  // 处理响应结果
  if (res.data.code === 0) {
    infoStep4.applicationId = res.data.msg.application_id;
    currentStep += 2;
    hideModal('submitModal');
  } else {

```

```

    showToast('error', '服务器无法提交, 请稍后重试或联系管理员! 或检查是否达到申请额度上限', 5000);
    hideModal('submitModal');
  }
} catch (err) {
  showToast('error', '出错了, 请稍后重试!', 5000);
  hideModal('submitModal');
}
}
}

```

这段代码以模块化方式组织, 将各个步骤封装成独立函数, 使代码结构清晰、易读、易于维护和扩展。通过 `catch` 捕获错误, 并利用封装好的 `showToast()` 函数处理错误提示, 有效避免程序崩溃或无法正常工作。此外, 代码采用 `async` 和 `await` 等 JS 异步编程手段控制执行顺序, 协调异步操作如上传 IP 信息和贷款信息。

在编码过程中, 运用 ES2017 提供的异步编程方法, 使代码更易理解和维护。通过将函数定义为 `async` 函数, 将异步操作变为同步模式。在需要获取异步操作返回结果时, 使用 `await` 关键字阻塞线程, 等待异步操作完成, 并将其结果赋值给变量。这种模式使异步编程变得简单易懂, 提高代码可读性, 降低维护难度。

同时, 代码还使用模板字符串和 ES6 解构赋值等语法糖, 使代码更加简洁, 提升可读性, 降低出错概率。综上所述, 这段代码注重代码结构、可读性、错误处理和异步操作协调, 具有较高的维护性和稳定性。

(4) 智能客服（大模型）

在 AI 时代, 生成式人工智能模型已经逐渐进入各行各业, 为各领域带来前所未有的变革。为了提升用户体验, 我们引入了智能客服模块, 依托科大讯飞大模型的强大技术支持, 实现了高效、智能、人性化的客户服务。

大模型交互代码如下。

代码 6: 大模型交互代码

```

async sendToSpark() {
  let myUrl = await this.getWebSocketUrl();
  this.tempRes = "";
  let realThis = this;

  const connectOptions = {
    url: myUrl,
    method: 'GET',
    success: res => {

```

```

        realThis.wsLiveFlag = true;
    }
};

realThis.socketTask = uni.connectSocket(connectOptions);

realThis.socketTask.onError((res) => {
    // 错误处理, 在此从略
});
// websokcet 的打开阶段, 进行消息的处理
realThis.socketTask.onOpen((res) => {
    realThis.TextList=[];
    realThis.TextList.push({
        "role": "user",
        "content": realThis.template+realThis.TEXT
    });
    // 大模型返回消息, 加入消息列表
    let params = {
        "header": {
            "app_id": realThis.APPID,
            "uid": "uid 在此保密"
        },
        "parameter": {
            // 参数从略
        },
        "payload": {
            "message": {
                "text":realThis.TextList
                // 传递用户提问
            }
        }
    };
    realThis.socketTask.send(JSON.stringify(params));
});

// 接收消息
realThis.socketTask.onMessage((res) => {
    let obj = JSON.parse(res.data);
    if (obj.header.code !== 0) {
        //错误处理, 在此从略
        realThis.socketTask.close();
    }
});

```

```

    } else if (obj.header.status === 2) {
      setTimeout(() => {
        realThis.socketTask.close();
      }, 1000);
    }
  });
}

```

这段代码是使用 uni-app 进行 WebSocket 与科大讯飞星火大模型平台连接和通信的实现。它首先获取 WebSocket 的 URL，然后创建一个 WebSocket 连接。在连接成功后，发送第一帧数据。当接收到服务器返回的消息时，逐帧解析并处理消息内容。如果接收到错误状态码，则显示错误提示并关闭连接；否则，在满足特定条件时，延迟一段时间后关闭连接。

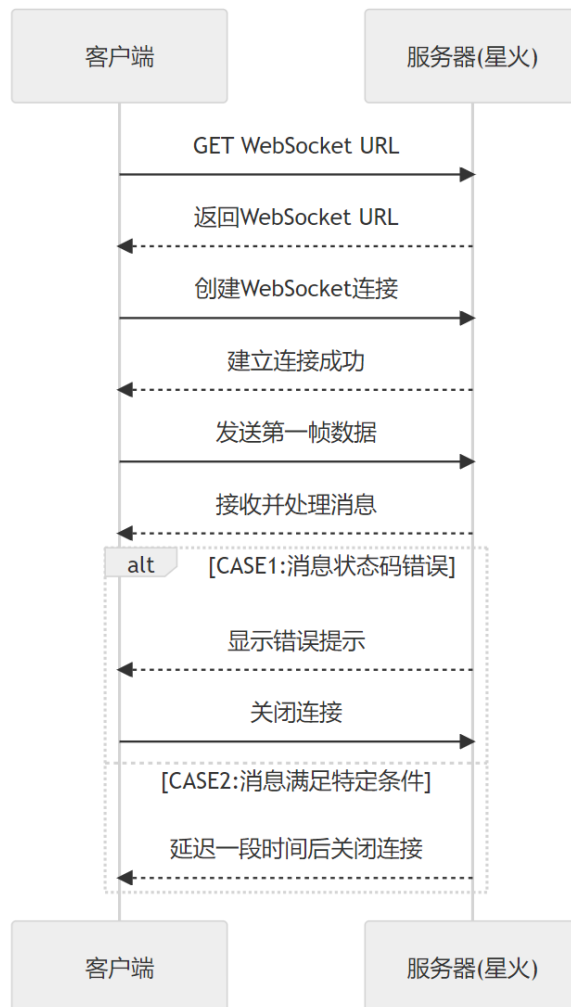


图 33：交互时序图

在这个智能客服代码中,WebSocket用于实现客户端与服务器之间的实时通信。流式输出是指数据以连续、实时的方式传输和处理,而不是一次性全部发送或接收。在该代码中,当用户输入文本后,通过 WebSocket 连接将消息(包括用户输入的文本)发送给服务器。服务器接收到消息后,根据输入内容生成回复,并通过相同的 WebSocket 连接将回复实时传回给客户端。客户端接收到回复后,在界面上显示出来。

这种流式输出的作用是提高用户体验,使得用户能够更快地得到机器人的回复,避免了传统 HTTP 请求-响应模式下的延迟问题。同时,由于 WebSocket 支持双向通信,也使得服务端可以主动向客户端推送更新或者提示信息,增强了系统的交互性。

最终得到的与大模型交互的效果如下——

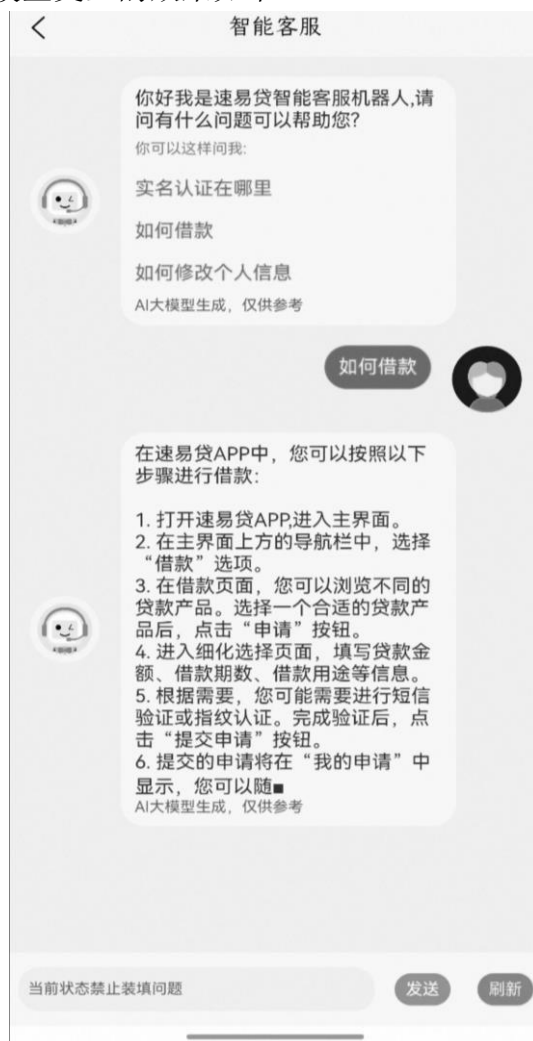


图 34: 智能客服效果展示

如何让大模型能够正确回答用户问题呢？介于 API 调用上下文消耗 token 量的限制，我们采用的方案是每次 API 请求携带一段关于 APP 使用描述的 prompt（指令语言）。我们所描述的 prompt 如下：

你现在是一名软件说明书，需要帮助用户解决软件问题。如果用户提问超过使用说明，请回答对不起我还在学习中。软件的使用说明为：速易贷是一款智能贷款管理 APP，用户端分为借款、还款、我的。如需借款，……（功能性描述从略）。现在，请根据以上内容回答用户问题：

这段模型语言采用了角色扮演的手法描述，有效的限制了大模型基于我们所给出的语言内容进行回答。实测显示，大模型可以依照我们的 prompt 进行自动分点分段概述流程，并作适当补充拓展，这种设计方式，既保证了回答的准确性，又提高了回答的效率。Prompt 设计有效。

2.2.4 后端框架及关键代码分析

后端的 Ktor、druid 为我们提供了 5 级基本的模块层，功能涵盖控制、异常出错、数据库模型连接、插件和视图获取。对于每个核心的表或数据类型，框架中包含有相应的 controllers、exceptions、models、plugins、views 进行控制。

（1）session 会话保持

在 Web 开发中，我们常常会引入 session 来在多个请求之间保持用户的状态信息，使得网络连接的无状态变为“有状态”。每当用户请求一个网页时，这个请求会被服务器处理并保存在服务器的内存中，session 就是用来存储这些信息的。具体来讲，session 的功能有：

- **保持用户状态：**在用户浏览不同的网页时，session 可以存储用户的登录信息，如用户名、权限等，这样用户在网站上的所有行为都能够得到正确的处理，不需要在每个页面都重新验证用户的身份。
- **简化代码：**通过 session，开发者可以在一个地方管理用户的状态，避免了在每个页面中重复编写管理用户状态的代码，使得代码更加简洁、易于维护。
- **提高性能：**将用户的状态信息存储在服务器的内存中，比每次都访问数据库或文件来获取信息要快得多，这可以显著提高用户请求的处理速度。

Session 进行会话保持的流程如下图所示。

Session的交互流程



图 35: session 交互流程示意图

基于此，后端设计了一个管理 session 的类。

代码 7: session 管理

```
// 导入相关的包和类
import io.ktor.server.application.*
import io.ktor.server.sessions.*
import online.rainy.models.Session
import online.rainy.models.sessions
import org.ktorm.dsl.eq
import org.ktorm.entity.add
import org.ktorm.entity.find
import org.ktorm.entity.removeIf

// 安装session管理器，用于管理用户会话
fun Application.installSessions() = install(Sessions) {
    // 设置session的存储方式为SessionStorageDatabase
    header<UserSession>("cart_session", SessionStorageDatabase())
}

// UserSession 数据类，用于表示用户会话
data class UserSession(val user: Long)

// SessionStorageDatabase 类，实现了SessionStorage 接口，用于存储和管理用户会话
class SessionStorageDatabase : SessionStorage {
    // invalidate 方法用于删除指定的session
```

```

override suspend fun invalidate(id: String) {
    // 删除数据库中 id 为该字符串的 session
    database.sessions.removeIf { it.id eq id }
}

// read 方法用于读取指定的 session
override suspend fun read(id: String): String =
    // 在数据库中查找 id 为该字符串的 session，并返回其 value 属性
    database.sessions.find { it.id eq id }?.value ?: throw
NoSuchElementException("Session $id not found")

// write 方法用于写入新的 session
override suspend fun write(id: String, value: String) {
    // 在数据库中添加一个新的 session，其 id 和 value 属性分别为传入的字符串
    database.sessions.add(Session{
        this.id = id
        this.value = value
    })
}
}

```

这段代码通过 Ktor 框架定义了一个 `UserSession` 数据类，表示用户会话，以及一个 `SessionStorageDatabase` 类，用作会话存储，并且实现了 `SessionStorage` 接口的相关操作。

`Session` 技术在服务器端存储的信息量远大于 `Cookie`，可以存储用户的状态信息、认证信息等；同时，`Session` 技术可以为用户提供更安全的认证方式，如通过加密传输会话 ID，防止会话 ID 被篡改。

(2) 贷款申请处理

贷款申请的提交时后端处理贷款的核心一环，提交后的贷款申请需要经过一系列严谨的数据流程，包括对利息的计算、空间的开辟等。有关贷款判定是否审批通过的内容将在后续章节专门讲述，此处仅罗列后端数据处理的基本工作流程。

代码 8：贷款申请后端接口处理

```

// 定义一个名为 Route.routeSubmitLoan 的函数，该函数为 POST 请求提交贷款的路由
fun Route.routeSubmitLoan() {
    // 使用 post 方法创建一个处理 POST 请求的路由，请求的 URL 为 "submit_loan"
    post("submit_loan") {
        // 从请求中获取参数，参数类型为 submitLoanParams
        val params = call.receive<submitLoanParams>()
    }
}

```

第二章 针对复杂工程问题的方案设计与实现

```
// 在数据库的 interestWays 中查找与参数中的时间和利率相等的记录
val interestWay = database.interestWays.find { (it.time eq
params.time) and (it.rate eq params.rate) }
if (interestWay == null)
call.respond(SubmitLoanView.responseNoInterestWay())

else {
    // 定义一个变量 status 用于记录应用状态, 类型为整数
    val status : Int
    val user = database.users.find { it.phone eq params.phone }
    // 在数据库的 products 中查找 LoanProductId 等于参数中的
    // LoanProductId 的记录
    val product = database.products.find { it.loanProductId eq
params.loanProductId }
    val plus =
        附加评分从略...

    // 创建一个 LoanApplication 对象, 填充相关属性值
    val application = LoanApplication {
        loanProductId = params.loanProductId
        phone = params.phone
        interestWayID = interestWay.interestWayID
        loanAmount = params.loanAmount
        applicationStatus = status
        calcuInterestWay = params.calcuInterestWay
        payMoney = params.payMoney
        payTime = params.payTime
        actualPayBackTime = Array(params.payTime.size){"-1"}
        usage = params.usage
        applyTime = params.applyTime
        riskIndex = user.creditScore.toFloat()
    }

    // 将 LoanApplication 对象添加到数据库的 applications 中
    database.applications.add(application)
    // 如果状态为 1, 则调用 risikoRechnen 方法计算风险值, 传入参数为找
    // 到的 LoanApplicationId
    if(status == 1)
        risikoRechnen(database.applications.find { it.applyTime eq
params.applyTime }!!.loanApplicationId)
    // 返回提交成功的响应
    call.respond(SubmitLoanView.responseSuccess())
}
}
}
```

这段代码定义了一个名为 `Route.routeSubmitLoan()` 的方法，这个方法处理了一个提交贷款的流程。其核心功能大致可以分为以下几个步骤：

首先，通过调用 `call.receive<submitLoanParams>()` 来接收一些参数。这些参数可能包括贷款产品 ID、电话号码、贷款金额等等。

接着，这段代码会检查数据库中的 `interestWays` 表，看是否有匹配传入参数的时间和利率的记录。如果没有找到匹配的记录，就会返回一个提示响应，表示没有找到对应的贷款方式。

如果找到了匹配的记录，代码会继续检查一些条件来计算贷款申请的状态。这些条件包括用户的信用评分、产品接受的最低评分等等。

根据计算出的状态，代码会创建一个新的贷款申请记录，并将其添加到数据库的 `applications` 表中。如果申请状态为 1，代码会调用 `risikoRechnen()` 方法来计算风险。最后，代码会返回一个成功的响应。

代码整体流程如下所示。

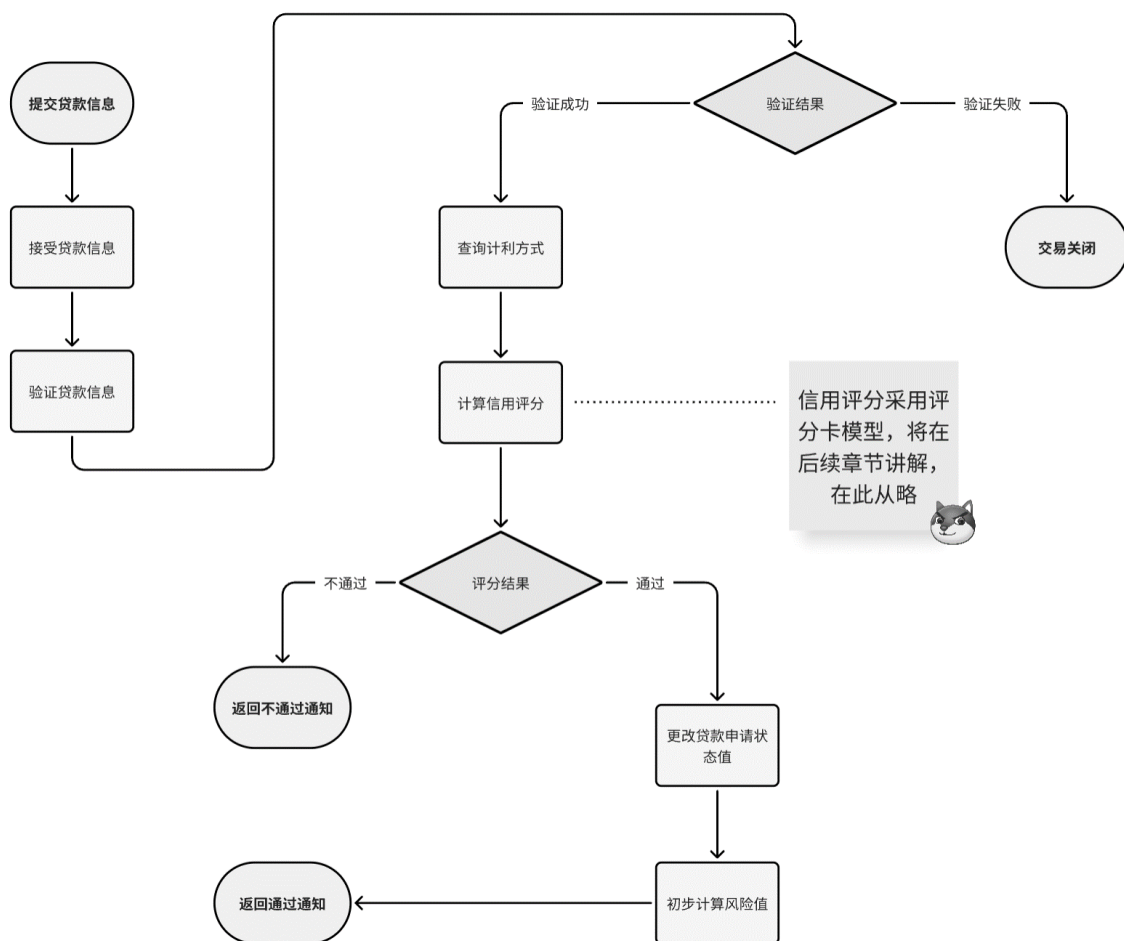


图 36：贷款申请接口的处理流程

在代码中很好的体现了面向对象的特性。面向对象的设计是一种编程范式，它通过将数据和操作封装在对象中来模拟现实世界中的实体。在这段代码中，我们可以看到许多类（如 `submitLoanParams`、`interestWay`、`user`、`product`、`plus`、`application`）的实例被创建，这些实例拥有属性（即数据）和方法（即操作）。

- `submitLoanParams` 代表一个贷款申请的参数，有属性如 `loanProductId`、`phone`、`loanAmount` 等，这些属性存储了关于贷款申请的信息。
- `interestWay` 代表一种贷款方式，它有属性如 `time` 和 `rate`，这些属性存储了关于贷款方式的信息。
- `user` 和 `product` 则分别代表一个用户和一个贷款产品，他们的属性包括 `phone`、`creditScore`、`loanProductId` 等。
- `application` 则代表一个贷款申请，它的属性如 `loanProductId`、`phone`、`interestWayID` 等代表了申请的相关信息。

通过面向对象的设计，这些对象提供了封装和抽象的机制，使得代码更加模块化和易于理解。修改这些类的实现不会影响到其他部分的代码，因为它们的行为是通过定义好的接口来定义的。此外，通过面向对象的设计，代码的可重用性和可维护性也大大提高了。

这段代码也体现了模块化设计思想。我们通过将不同的功能封装在不同的方法中，实现了良好的模块化设计。

`post("submit_loan")`方法是一个很好的例子，它封装了处理贷款申请的整个流程。在这个方法内部，首先通过调用 `call.receive<submitLoanParams>()`接收参数，然后根据这些参数进行一系列的操作，如查找数据库、计算状态、创建新的贷款申请等。这个方法的设计使得代码更加清晰和易于理解，因为所有的贷款申请处理逻辑都被封装在一个地方。

通过模块化设计，代码的复杂度被降低，因为每个模块都有明确的功能和输入输出。这使得代码更容易测试和维护，因为每个模块都可以独立地运行和调试。此外，模块化设计也提高了代码的可重用性，因为不同的模块可以组合成新的功能。

（3）贷款产品的编辑与新增

贷款产品是贯穿于整个贷款过程的一个重要参数，在管理员端，我们可以对其进行增删查改操作。为了接口的复用性，根据代码规范，我们将新增和编辑接口进行复用处理

相关代码如下所示。

代码 9：贷款产品的编辑和新增

```
// 定义一个路由方法，用于编辑贷款产品信息
fun Route.routeLoanProductEdit(){
    // 使用HTTP PUT 方法，并设置路由名为"loan_products_edit"
    put("loan_products_edit") {
        // 从请求中接收 LoanProductEditParams 类型的参数
        val param = call.receive<LoanProductEditParams>()

        // 根据接收到的参数创建一个新的Product 对象
        val product = Product{
            loanProductId = param.loanProductId
            loanProductName = param.loanProductName
            isRelease = param.isRelease
            loanProductMaxMoney = param.loanProductMaxMoney
            acceptScore = param.acceptScore
            tag = param.tag
        }

        // 在数据库中查找是否存在与传入参数相同的贷款产品
        var result = database.products.find { it.loanProductId eq
param.loanProductId }

        // 如果在数据库中没有找到相同的产品，则将新创建的产品添加到数据库中
        if(result == null){
            if(database.products.add(product) != 1)
                call.respond(LoanProductEditView.responseFailed())
        }
        else{
            // 否则更新已存在的产品信息
            result = product
            if(database.products.update(result) != 1)
                call.respond(LoanProductEditView.responseFailed())
        }

        // 遍历接收到的参数中的所有利息方式，并将其添加或更新到数据库中
        for(interestWay in param.interestWays)
        {
            val temp = InterestWay{
                loanProductId = param.loanProductId
                time = interestWay.time
                rate = interestWay.rate
            }
            var resultInterestWay = database.interestWays.find
{ (it.loanProductId eq param.loanProductId) and (it.time eq
```

第二章 针对复杂工程问题的方案设计与实现

```
interestWay.time) }  
  
    // 如果在数据库中没有找到相同的利息方式，则将新的利息方式添加到数据  
    库中  
  
    if(resultInterestWay == null){  
        if(database.interestWays.add(temp) != 1)  
            call.respond(LoanProductEditView.responseFailed())  
    }  
    else{  
        // 否则更新已存在的利息方式信息  
        resultInterestWay.rate = temp.rate  
        if(database.interestWays.update(resultInterestWay) != 1)  
            call.respond(LoanProductEditView.responseFailed())  
    }  
  
    // 如果一切操作都成功，返回响应成功的视图  
    call.respond(LoanProductEditView.responseSuccess())  
}  
}
```

一个贷款产品会涉及到较多的修改参数，且介于数据范式要求，计利方式被分布在单独一张表中。我们可以对整个贷款产品的可修改参数与前端表单效果图展示如下。

编辑贷款产品

产品ID: 1

数据库

产品名称: 贷款

产品发布状态: ☒

最大借款值: 10000

审批阈值: 30

产品利率与分期: 期数: 2 - 利率: 1.1 %

期数: 5 - 利率: 2.1 %

+添加

标签: ☒ 超低利率 ☒ 限时上架 ☐ 热销中

取消 确定

触发接口

产品表

计利方式表

ProductName ←

isRealse ←

MaxLoan ←

LowestScore ←

Time ←

Rate ←

图 37: 贷款产品的可修改参数与效果图

这段代码充分利用了 Ktor 框架的特性来实现贷款产品信息编辑的路由处理逻辑。Ktor 框架因其简洁、轻量级的设计特点而广受欢迎，使得编写和维护这样的路由变得简单易行，并提供了出色的性能和扩展性。

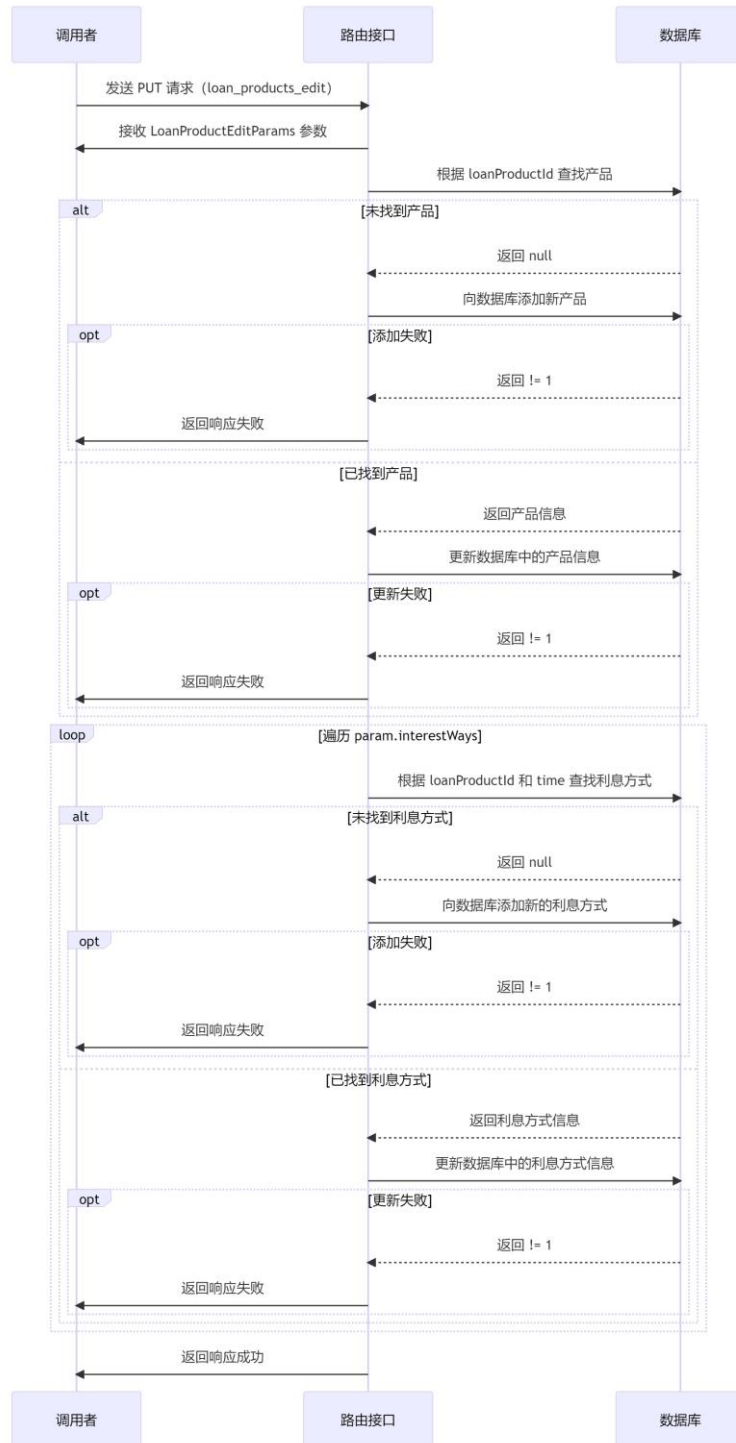


图 38：贷款产品修改和新增的时序交互图

首先，通过使用 Ktor 框架，这段代码采用了基于协程的异步非阻塞处理模型。从图 38 可以看到，中间的路由接口作为中介，可以起到分发请求、处理请求和返回响应的作用。在这个流程图中，路由接口接收来自调用者的请求，然后根据请求的内容和参数，决定将请求发送到哪个数据库进行处理。这种设计模式的优势在于能够有效地处理并发请求，从而提高系统的吞吐量和响应速度。这意味着在高负载情况下，服务器仍然能够保持高效运行，并提供快速的服务响应时间。

其次，Ktor 框架提供了一个简单易用的 API，使开发者可以轻松地编写和维护路由处理逻辑。例如，在代码中可以看到 `put("loan_products_edit") { ... }` 这样的语法，它简洁地定义了一个 PUT 方法的路由处理逻辑。这种直观的 API 设计降低了代码阅读难度曲线，使得开发人员能够更快地上手。

此外，为了确保类型安全并减少因类型转换错误导致的问题，我们在代码中使用了 Kotlin 的数据类（如 `LoanProductEditParams`）来接收和发送数据。这种方式提高了代码的健壮性和可维护性，同时也简化了数据传输过程。

最后，该段代码遵循了 RESTful API 设计原则，使用 HTTP PUT 方法来更新资源（即贷款产品信息）。这种方式符合现代 Web 应用程序的标准做法，易于理解和使用。这有助于构建出更加规范且易于与第三方系统集成的接口。

2.3 针对核心重难点工程的方案实施

除基础的前后端逻辑外，本项目所涉及的一些功能点具有一定的技术核心重难点，下面进行逐一的拆解分析。

2.3.1 信息采集

在第一章中，我们详细分析了需要采集哪些信息维度。但是对于信息的捕获采集仍然是一个难点问题。在信息采集过程中，如何优化用户体验，减少用户操作繁琐程度，提高用户满意度是一个关键问题。针对这三个维度不同的特性，我们选择不同的方式进行采集。

1. 用户基本信息维度：我们通过用户手动填写的方式采集，用户在首次使用 APP 时需要填写这些信息，以便 APP 能够建立完整的用户画像。这些信息将存储在 APP 的数据库中，并用于后续的贷款申请和风险评估。一

定时间后，我们会提醒用户更新信息内容。手动填写可以让用户更准确地提供个人信息，避免信息不准确或缺失。此外，通过用户手动填写的方式采集信息也有助于提高用户对 APP 的信任度和使用体验，因为这种方式给用户提供了更多的控制权和透明度。

2. 网络行为维度：当用户在 APP 中进行登录等操作时，APP 会收集相关的日志数据，此外，在用户进行申请提交时，会在告知用户的前提下进行 IP 信息、设备标识码、地理位置等信息的采集，用以辅助风险评估和坏账预测。
3. 交易特征维度：数据库会收集并管理用户的交易历史记录（特别是历史违约情况），以便评估用户的信用状况和风险水平。这些历史记录将用于为后续的贷款申请提供参考。

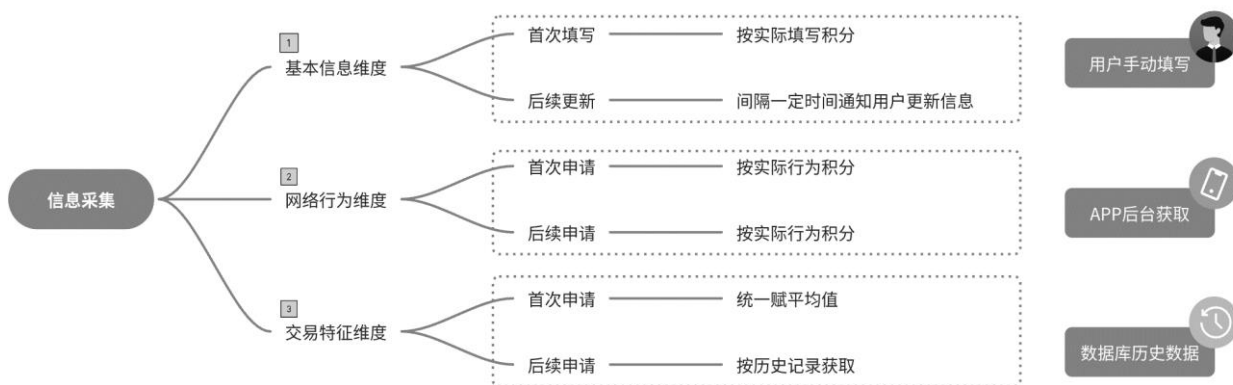


图 39：三类信息采集方案

网络行为特征的获取是贷款 APP 中比较复杂的一部分，因为需要依赖 APP 与手机原生信息源进行数据交互。这涉及到权限请求、数据解析等多个环节，且存在跨平台难题。为了解决传统方案在系统原生权限调用方面存在的问题，小组调研了 5+RunTime 技术。该技术通过调用增强浏览器引擎进行系统原生权限调用，相比传统的方案，具有更优的性能和更强的可扩展性。

5+RunTime 技术的主要优势在于其调用增强浏览器引擎进行系统原生权限调用的方式。这种方式可以实现更快的执行速度和更低的资源消耗，同时还可以支持多端适配，不受终端限制影响。相比传统的方案，5+RunTime 技术可以更好地适应不同的操作系统和终端设备，提高系统的可扩展性和灵活性。

此外，5+RunTime 技术还具有更好的安全性和可靠性。由于其调用增强浏览器引擎的方式，可以更好地保护系统的安全性和稳定性。同时，该技术还可以提

供更好的错误处理和异常处理机制，提高系统的可靠性和稳定性。

下面（图 40）给出了使用 5+Runtime 技术标准常见的一些 API 调用请求，通过 plus 类用以获取系统相关信息。

 5+ RunTime	功能调用	使用目的
	plus.device.uuid;	获取设别标识
	plus.fingerprint.isEnrolledFingerprints();	调用设备指纹认证
	plus.geolocation.getCurrentPosition()	获取设备定位信息

图 40：5+RunTime 技术

通过 5+Runtime 技术，我们可以实现对信息采集模块的快速功能封装，代码 10 展示了指纹认证的原生调用封装交互过程。

代码 10：指纹认证交互接口调用

```
// 检测是否支持指纹认证
uni.checkIsSoterEnrolledInDevice({
  checkAuthMode: 'fingerPrint',
  success(res) {
    this.canFingerPrint = true
  },
  fail(err) {
    this.canFingerPrint = false
  },
})
// 调用指纹认证
uni.startSoterAuthentication({
  requestAuthModes: ['fingerPrint'],
  challenge: '123456',
  authContent: '请用指纹验证身份',
  success(res) {
    if (res.errCode == 0) {
      uni.$u.toast('指纹认证成功');
    } else if (res.errCode == 90010) {
      uni.$u.toast('失败次数过多，已经冻结！请改用方式一');
    } else if (res.errCode == 90008) {
      uni.$u.toast('用户取消授权');
    } else if (res.errCode == 90003 || res.errCode == 90002 ||
res.errCode == 90001) {
```

信软学院进阶式挑战性综合项目 III 报告

```
uni.$u.toast('指纹认证不被设备支持，请改用方式一');
    } else {
        uni.$u.toast('认证失败！');
    }
},
fail(err) {
    uni.$u.toast('指纹验证失败，建议使用方式一');
},
})
```

基于 5+Runtime 技术，前端部分实现了对以下信息的初步捕捉

表 16：用户信息收集

序号	信息	描述	获取方式
1	用户 IP	通过用户的 IP 信息，初步锁定其所在大致位置	进行 API 网络请求
2	用户位置定位信息	通过手机自带卫星定位，进行位置信息捕获，锁定用户实际位置	通过手机卫星定位模块获取
3	用户本机指纹验证	在支持指纹识别的手机终端，可以获得唯一的指纹匹配码，由手机终端验证指纹是否匹配以确认身份（注：仅作参考验证，不作信息上传，仅在前端实现）	通过手机内置身份验证模块进行获取
4	手机终端标识	获取手机终端标识来唯一标注手机。	

以上信息均应征得用户同意，除定位和 IP 外，其余两类敏感信息不会上传至后端服务器，仅仅在前端完成身份验证和信息排查功能，保证用户隐私安全。

2.3.2 评分卡模型

（一）数据集预处理

数据集的质量决定了模型训练的优良程度，我们以 Kaggle 以及天池金融风控

第二章 针对复杂工程问题的方案设计与实现

的数据集为基础。这两个数据集的内容丰富，充盈。数据集包含了大量借款人的信用记录，包括借款人的基本信息、财务状况、资产情况等多个方面的数据。

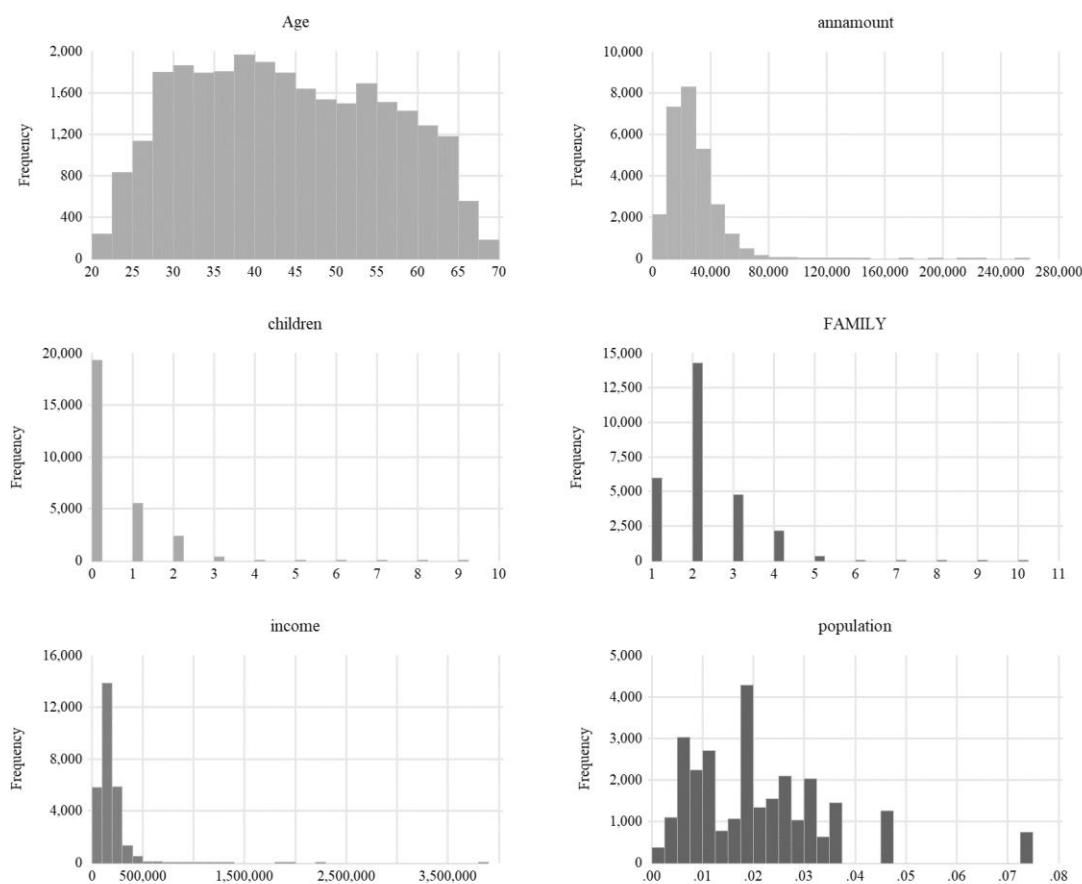


图 41：整数字段数据分布情况

数据集共含有 27569 个有效数据。可以看到，年龄字段分布最为均衡，年度还款金额和收入水平较为集中。借款人申请的贷款年金和收入水平差距不大。

在预处理时，首先，我们需要剔除数据集中的异常值。以第二个数据集为例，其中的性别信息被分为三个变量，分别为“MALE”表示男性、“FEMALE”表示女性，以及“OTHER”表示其他。第三个变量“OTHER”的出现，是由于数据集在性别这一项的录入存在不足，部分用户并未填写性别信息。因此，我们需要将这些异常值排除在外。

其次，我们需要对数据集中的某些变量设置虚拟变量。以第一个数据集中的堆对城市评级为例，其评级由字符串“GOOD”，“NORMAL”和“BAD”表示。为了简化后续操作。为此，我们需要事先将这些变量转换为 1 和 0 的数值形式，设定两个虚拟变量“POOR”和“RICH”，分别标识该地区评级是否为“GOOD”、该地区评级是否为“BAD”。以便在后续的分析过程中能顺利进行。

对于异常数据清理部分，小组综合了以下的方式进行整合，对于数据源的异常值空缺值进行合理的预估或省略——

- ① **缺失值处理**：缺失值通常会导致模型的不准确和无法使用。因此，需要检测出缺失数据并进行填充或删除。对于连续变量，小组使用均值、中位数或插补法等方法进行填充；对于分类变量，使用众数进行填充；如果可接受，也可以直接从数据集中删除缺失数据。
- ② **异常值处理**：异常值通常指超出正常范围的极端观测值，可能会导致模型表现降低或偏差增加。因此，需要使用异常值检测技术（如箱形图检测）来识别和处理异常值。常用的区间范围有 1.5 倍四分位范围或 3 倍四分位范围。
- ③ **重复数据处理**：重复观测可能会产生模型偏差或误导，因此可以通过删除或合并重复数据（如根据客户 ID 或申请时间）来避免这些问题。
- ④ **数据集成与标准化**：在多个数据集里可能含有多种来源的信息，这意味着数据结构、数据类型、日期格式等可能不一致，这时，需要对数据进行集成和标准化操作。

以基于箱型图法的异常值处理为例子，可使用如下 Python 代码进行异常值的初步筛查——

代码 11：使用箱型图法剔除异常值

```
import csv
import numpy as np

def remove_outliers_box_csv(file_path, k=1.5):
    """
    使用箱型图法剔除 CSV 文件中每一列的异常值
    :param file_path: CSV 文件路径
    :param k: 倍数，默认为1.5 倍
    :return: 处理后的数据字典，键为列名，值为处理后的结果列表
    """

    # 打开CSV文件并读取其中的数据，将不同列的数据保存到一个字典中
    with open(file_path) as f:
        reader = csv.DictReader(f)
        columns = reader.fieldnames
        data_dict = {c: [] for c in columns}    # 初始化字典
        for row in reader:
```

```

        for c in columns:
            data_dict[c].append(float(row[c]))

# 对每一列的数据基于箱型图法去除异常值，并将处理后的结果保存到一个新的字典中
result_dict = {}
for c in columns:
    data = data_dict[c]
    q1, q3 = np.percentile(data, [25, 75]) # 计算上下四分位数
    iqr = q3 - q1 # 计算四分位差
    low = q1 - k * iqr # 计算下限
    high = q3 + k * iqr # 计算上限
    result = []
    with open(file_path) as f:
        reader = csv.DictReader(f)
        for row in reader:
            val = float(row[c])
            if low <= val <= high: # 判断是否为异常值
                result.append(val)
    result_dict[c] = result

return result_dict

```

通过以上方案，完成数据预处理工作。

（二）特征工程分箱与 WOE 编码

A. 变量分箱介绍

变量分箱是一种常用的特征工程方法，意在增强变量的可解释性与预测能力。变量分箱方法主要用于连续变量，对于变量取值较稀疏的离散变量也应该进行分箱处理

变量分箱在数据分析和机器学习中具有多重优势。首先，它能有效降低异常值对模型的影响，增强模型的稳定性。其次，分箱允许缺失值作为特殊类别参与，减少了处理缺失值时的不确定性。此外，分箱提升了变量的可解释性，尤其是与变量编码结合时。同时，分箱作为一种非线性变换，能够增强模型捕捉非线性关系的能力。最后，分箱有助于确保训练集和测试集的同分布性，这对于模型的有效性和泛化能力至关重要。

B. 卡方分箱

分箱方法分为无监督分箱和有监督分箱。常用的无监督分箱方法有等频分箱，等距分箱和聚类分箱。有监督分箱主要有 best-ks 分箱和卡方分箱。

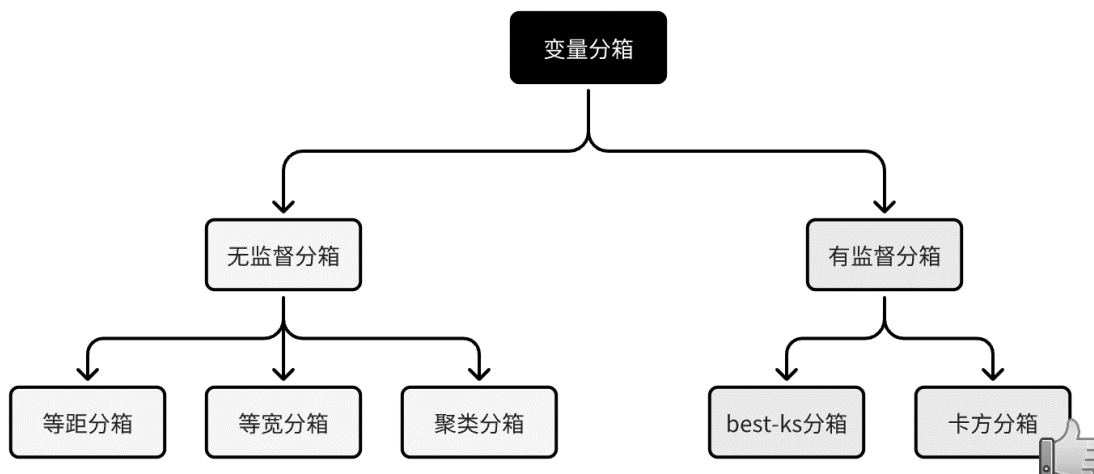


图 42：常见分箱方法分类

在我们的项目中，采用卡方分箱的方法。卡方分箱是典型的基于合并机制的自底向上离散化方法。其基于如下假设：如果两个相邻的区间具有非常类似的类分布，则这两个区间可以合并；否则，它们应当保持分开。此处衡量分布相似性的指标就是卡方值。卡方值越低，类分布的相似度越高。

其算法为——

算法：卡方分箱算法

- 1: 预先设定一个卡方的阈值 α
 - 2: 初始化：根据要离散的属性对实例进行排序,在排序的基础上进行合并，每一个实例属于一个区间
 - 3: **while** $\exists \chi_i^2 < \alpha$
 - 4: 计算卡方：计算每对相邻区间的卡方值

$$\chi^2 = \sum_{i=1}^2 \sum_{j=1}^2 \frac{(A_{ij} - E_{ij})^2}{E_{ij}}$$

其中 A_{ij} 是第 i 区间第 j 类的实例的数量， E_{ij} 是期望频率 $E_{ij} = \frac{N_i \times c_j}{N}$
 - 4: 合并区间：将卡方值最小的一堆区间合并
 - 5: **End while**
-

C. WOE 和 IV 值

WOE (Weight of Evidence, 即证据权重), 是一种统计量, 用于揭示一个可预测变量与二分类结果之间的关联强度。它衡量了某个特定值或值区间对分类结果的预测能力, 从而帮助我们理解变量对分类结果的贡献程度。在信用评分和风险评估中, WOE 是一种常用的工具, 它通过比较不同值区间内正负样本的比例差异, 来评估该变量对分类结果的影响力。在本项目中, 其值描述了某个分组 (箱) 与整个样本中好坏比的关系, 是对“好”客户和“坏”客户的比较。

其计算方式为——

$$WOE_i = \ln\left(\frac{Bad_i}{Bad_T} / \frac{Good_i}{Good_T}\right) = \ln\left(\frac{Bad_i}{Bad_T}\right) - \ln\left(\frac{Good_i}{Good_T}\right)$$

WOE 编码可以有效地增加特征的互信息, 并把数据从原来的非线性可分空间映射到新的近似线性可分空间。这种线性化特征可以将分类变量转换为数值型特征, 使得评分卡模型能够更好地处理分类变量, 并且避免了不同类别之间的物理距离影响模型结果。

IV 值 (Information Value, 即信息价值) 是对 WOE 值进行加权求和的结果, 它提供了一个量化的指标来衡量 WOE 所表示的变量在区分两个类别 (如好客户与坏客户) 方面的能力。IV 值通过考虑 WOE 值的分布和每个值区间的频率, 综合评估了变量对分类结果的预测贡献。一个较高的 IV 值意味着该变量在区分两个类别时具有更强的预测能力, 而一个较低的 IV 值则表示该变量的区分能力较弱。在信用评分模型中, IV 值常用于特征选择, 帮助识别对风险预测最有影响力的变量, 从而优化模型的性能。其计算公式为——

$$IV_i = \left(\frac{Bad_i}{Bad_T} - \frac{Good_i}{Good_T}\right) \times WOE_i$$

$$IV = \sum_{i=1}^n IV_i$$

一般认为, $IV < 0.02$ 为无用特征, $0.3 < IV < 0.5$ 具有一定特征, 高于 0.5 则认为是一个不正常的值。不真实。

D. 核心代码

对数据进行分箱和特征化处理、分箱的代码如下。

代码 12: 特征化工程 (含分箱) 代码

```
# 导入必要的库和模块
from toad.metrics import KS, F1, AUC
import toad
```

```
# 部分导入库省略
# 读取训练数据集
filename = './train.csv'
data = pd.read_csv(filename)
print("读取完成")

# 指定需要删除的列
to_drop = ['SK_ID_CURR']

# 使用 toad 库计算 IV 值
print(toad.quality(data.drop(to_drop, axis=1), 'TARGET',
iv_only=True))

# 划分训练集和测试集
train = data[:200000]
test = data[200000:]
print('train size:', train.shape, '\nOOT size:', test.shape)

# 使用 toad 库进行异常值检测
print(toad.detector.detect(data))

# 特征选择, 使用 IV 值和相关性进行筛选
train_selected, dropped = toad.selection.select(data, target='TARGET',
empty=0.5, iv=0.02,
corr=0.5,
return_drop=True,
exclude=['SK_ID_CURR'])

# 创建 Combiner 对象用于特征工程
c = toad.transform.Combiner()

# 使用 Combiner 对数据进行分箱
c.fit(data.drop(to_drop, axis=1), y='TARGET', method='chi',
min_samples=0.05) # empty_separate = False
print(datetime.datetime.now(), "分箱完成")

# 使用 WOETransformer 对数据进行 WOE 变换
transer = toad.transform.WOETransformer()

# 训练数据的 WOE 变换
```

```

train_woe = transer.fit_transform(c.transform(train_selected),
train_selected['TARGET'], exclude=to_drop + ['TARGET'])
test_woe = transer.transform(c.transform(test))

# 查看变换后的数据
print(train_woe.head())
print(datetime.datetime.now(), "转化 woe 完成")

# 使用逐步回归选择特征
final_data = toad.selection.stepwise(train_woe.drop(to_drop, axis=1),
target='TARGET', direction='both', criterion='aic')
print(datetime.datetime.now(), "回归完成")
print(final_data, type(final_data))

# 最终数据集，去除目标列
final_data = train_woe.drop(to_drop, axis=1)
final_test = test_woe

# 打印最终数据集的形状和特征列
print(final_data.shape)
print(final_data.columns)
print(final_data)

# 列出最终数据集的特征列
col = list(final_data.drop(['TARGET'], axis=1).columns)
print(col)

```

（三）逻辑回归推演

A. 逻辑回归模型

Logistic Regression 虽然被称为回归，但其实际上是分类模型，并常用于二分类。Logistic Regression 因其简单、可并行化、可解释强深受工业界喜爱。Logistic 回归的本质是：假设数据服从这个分布，然后使用极大似然估计做参数的估计。

对于一些二元数据

$$D = (x_1, y_1), (x_2, y_2), \dots, (x_N, y_N), x_i \subseteq R^n, y_i \in \{0, 1\}, i = 1, 2, \dots, N$$

对于其中一个数据 x_i 我们预期模型去学习 $P(Y = 1|x_i)$ ，往往逻辑回归模型

会与 Sigmoid 函数相联系由于 Sigmoid 的值域和非线性的性质，并且 Sigmoid 是一个任意阶可导的凸函数，因此具有许多良好的性质，其中对于一元自变量的 Sigmoid 函数如下

$$y = \frac{1}{1 + e^{-(w^T x + b)}}, \quad y' = y(1 - y)$$

两边进行恒等变换

$$ye^{-(w^T x + b)} + y = 1$$

两边取对数后得到

$$\ln \frac{y}{1 - y} = w^T x + b$$

下面定义 y 为 x 下为 1 的概率，那么 $1-y$ 即是 x 下为 0 的概率，定义

$$odds = \frac{y}{1 - y}, \quad \ln(odds) = \ln \frac{y}{1 - y}$$

得到

$$P(Y = 1 | x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

逻辑回归先拟合决策边界(不局限于线性，还可以是多项式)，再建立这个边界与分类的概率联系，从而得到了二分类情况下的概率。直接对分类的概率建模，无需实现假设数据分布，从而避免了假设分布不准确带来的问题（区别于生成式模型）并且不仅可预测出类别，还能得到该预测的概率，这对一些利用概率辅助决策的任务很有用。

B. 建立损失函数

损失函数的构建是为了衡量模型预测值与实际值之间的差异，它的核心在于最大化正确分类的概率，同时最小化错误分类的代价。通过优化损失函数，我们可以调整模型参数，使得模型的预测结果更加接近真实情况。下面定义损失函数，本质是似然函数，使得下面独立事件相乘的概率最大

$$L(w) = \prod [p(x_i)]^{y_i} [1 - p(x_i)]^{1 - y_i}$$

为了方便求解，一般会把似然函数两边取对数

$$L(w) = \sum [y_i \ln p(x_i) + (1 - y_i) \ln (1 - p(x_i))]$$

代价函数即把损失函数在数据集上做平均，即：

$$J(w) = -\frac{1}{N} \ln L(w)$$

C. 建立优化器

优化器在逻辑回归模型中扮演着至关重要的角色。它的主要作用是通过迭代

优化算法来调整模型的参数（权重和偏置），以最小化损失函数（如对数似然损失），从而提高模型的预测性能。优化器的选择直接影响到模型训练的效率 and 最终的模型质量。

逻辑回归主要用的优化方法是随机梯度下降 SGD 和牛顿法。

首先，针对随机梯度下降，其计算为：

$$g_i = \frac{\partial J(w_i)}{\partial w_i}, w_i^{k+1} = w_i^k - \alpha g_i$$

其中 k 为迭代次数，每次更新参数可以比较 $|J(w^{k+1}) - J(w^k)|$ 来设置迭代退出的条件。或者设置一定的 k 来退出迭代。

牛顿法的基本思路是，在现有极小点估计值的附近对 $f(x)$ 做二阶泰勒展开，进而找到极小点的下一个估计值，其泰勒展开为

$$\varphi(w) = J(w^k) + J'(w^k)(w - w^k) + \frac{1}{2} J''(w^k)(w - w^k)^2$$

下面令 $\varphi(w)' = 0$ 得到迭代表达式 $w^{k+1} = w^k - \frac{J'(w^k)}{J''(w^k)} = w^k - H_k^{-1} \cdot g_k$ 其中 H_k 是海森矩阵。

$$H(f) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

而 Sigmoid 函数的二阶可导的，即满足牛顿法的使用。

D. 正则化

L1 正则化和 L2 正则化都是在机器学习中用来防止模型过拟合的技术。过拟合指的是模型在训练数据上表现得非常好，但是在新的、未见过的数据上表现不佳。为了避免这种情况，我们在模型的损失函数中添加一个正则化项，以限制模型复杂度，提高其泛化能力。

L1 正则化又称 Lasso 回归，其核心函数为

$$L(\omega, \lambda) = J(\omega) + \lambda \sum_{i=1}^n |w_i|$$

L1 正则化通过在损失函数中添加模型参数绝对值的总和来实现。这相当于在模型中引入了一个假设，即模型参数可能非常接近于零。这种正则化方法的结果是，一些参数可能会被压缩到完全为零，从而实现了特征选择。这意味着模型会忽略那些对预测结果影响不大的特征，只保留最重要的特征。这在特征数量很多时尤其有用，因为它可以帮助我们识别出最有影响力的特征。

L2 正则化又称 Ridge 回归，其是通过在损失函数中添加模型参数平方和的一半来实现。这相当于假设模型参数可能服从零均值的正态分布。

与 L1 正则化不同，L2 正则化不会将参数压缩到零，而是会使得参数值变小。这有助于防止模型参数过大，从而减少过拟合的风险。L2 正则化通常用于特征之间存在多重共线性的情况，因为它可以平滑参数估计，使得模型更加稳健。

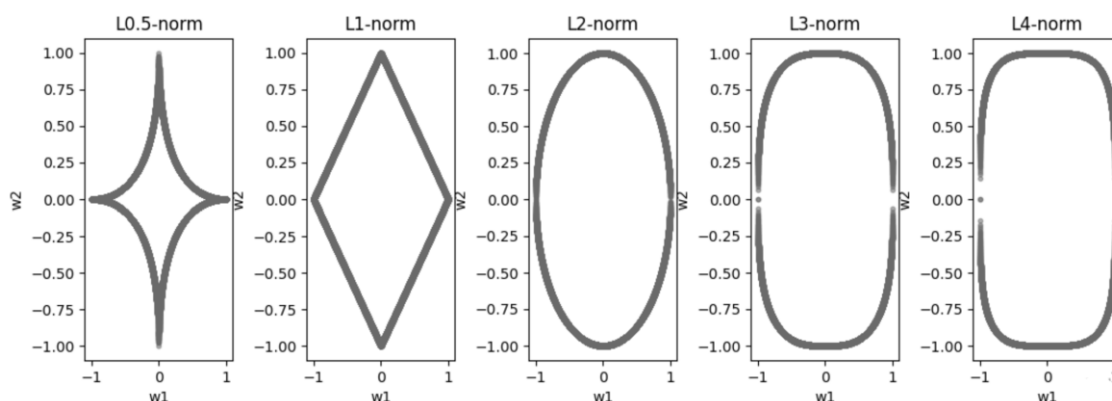


图 43：不同的正则化方法的对比

E. 核心代码

具体代码实现中，我们定义了一个简单的逻辑回归模型，使用了 PyTorch 框架。模型包含一个线性层和一个 Sigmoid 激活函数，适合处理二分类问题。优化器使用随机梯度下降（SGD）算法，并且设置了动量来加速训练过程。损失函数选择了二分类交叉熵（BCELoss），这是一种常用的损失函数，用于衡量模型输出与真实标签之间的差异。

代码 13：定义一个初始逻辑回归

```
import torch
import torch.nn as nn

# 初始化优化器，使用随机梯度下降（SGD）算法，设置学习率为0.01，动量为0.9
optimizer = torch.optim.SGD(lr_net.parameters(), lr=0.01, momentum=0.9)

# 定义损失函数，使用二分类交叉熵（BCELoss），适用于二分类问题
loss_fn = nn.BCELoss()
```

```

# 定义逻辑回归模型
class LR(nn.Module):
    def __init__(self): # 构造函数
        super(LR, self).__init__() # 调用父类构造函数
        # 定义一个线性层, 输入特征数量为n, 输出1个特征
        self.features = nn.Linear(n, 1)
        # 使用Sigmoid激活函数, 将输出限制在0到1之间, 适合二分类问题
        self.sigmoid = nn.Sigmoid()

    def forward(self, x): # 前向传播函数
        # 将输入x通过线性层, 得到中间结果
        x = self.features(x)
        # 应用Sigmoid激活函数, 得到最终输出
        x = self.sigmoid(x)
        # 返回激活后的输出
        return x

```

(四) 评分卡转化

A. 概率转化为分数

逻辑回归模型得到的客户逾期的概率，而评分卡最终输出的是客户评分。模型转评分就是指将逻辑回归模型转为评分输出。转换方法是把逻辑回归模型的线性部分抽出，并作一定的线性缩放，即

$$\text{Score} = \text{offset} + \text{factor} \times (WX + b)$$

其中，factor 和 offset 不直接设定，一般通过设定 init_socre, init_odds, delta_score, delta_rate 四个参数间接计算得到，这四个间接参数的意义如下：

- init_socre、init_odds: odds 为 init_odds 时，评分为 init_socre
- delta_score、delta_rate: odds 每降低 delta_rate 倍时，分数提升 delta_score 分
- odds: 它是坏客户的概率与是好客户的概率的比值。

factor 和 offset 的计算方式为

$$\text{factor} = -\frac{\text{delta_score}}{\ln(\text{delta_rate})}$$

$$\text{offset} = \text{init_socre} - \text{factor} \times \ln(\text{init_odds})$$

offset + factor × b 作为基准分，offset + factor × b 作为特征分。那么总分就是每

个特征的得分，再加上基础分。

计算用户得分时，只需算得用户各个特征所属分组，然后在表中查得特征得分，再加上基础分，就是用户的总评分。

表 17：评分卡

时期	一级指标	二级指标	评分
信用分	实名认证	实名认证	实名认证：300 非实名：0
	基础信息	年龄	54 岁以上：37 45 岁（不含）-54 岁（含）：28 36 岁（不含）-45 岁（含）：17 24 岁（不含）-36 岁（含）：9 18 岁（含）-24 岁（含）：0
		性别	男：10 女：33
		居住地	A 级城市：63 B 级城市：27 C 级城市：6
	详细信息	婚姻情况	已婚：23 未婚：20 离异：14
		学历	硕士及以上：55 本科：42 高中/高职：39 初中及以下：12
		工作	学生等未工作、自由职业：21 退休人员：26 公务员、军人：44 商人、企业创始人：35 办事人员和有关人员：32 农、林、牧、渔、水利生产人员：37 专业技术人员：41 其他在职：30

第二章 针对复杂工程问题的方案设计与实现

		年收入	0-7 万： 20 7-9 万： 22 9 万-15 万： 24 15-21 万： 27 21-30： 23 30 以上： 20
		在职时长	17 年及以上： 56 10-17： 44 6-10： 31 4-6： 20 2-4： 9 0-2： 1
		是否有车	是: 39 否: 12
		贷前	
定位和 IP 是否符合	是： 17 否： 0		
定位和城市是否符合	是： 23 否： 12		
		贷款总金额	0-2700: 37 2700 ~ 5450： 28 5450 ~ 7800: 20 7800 ~ 12000: 11 12000 以上: 18
		贷中	
		收入债务比	0.5%及以下： -150 0.5%-1%： -120

		1%-1.5%: -60 1.5%-2%: -30 2%以上: 0
--	--	---

（五）模型评估

A. 混淆矩阵

在二分类中，可以将样本根据其真实结果和模型的预测结果的组合划分为真阳性（true positive, TP）、真阴性（true negative, TN）、假阳性（false positive, FP）、假阴性（false negative, FN）。根据 TP、TN、FP、FN 即可得到二分类的混淆矩阵。

		PREDICTED CLASS	
ACTUAL CLASS		Class=Yes	Class=No
	Class=Yes	a (TP)	b (FN)
	Class=No	c (FP)	d (TN)

图 44：混淆矩阵

基于这个混淆矩阵，我们可以得到如下的 3 个评估指标。

a) 准确率

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = \frac{\text{预测正确的}}{\text{所有样本}}$$

b) 精准率

$$Precision = \frac{TP}{TP + FP} = \frac{\text{预测对的正样本}}{\text{判断为正样本}}$$

c) 召回率

$$Recall = \frac{TP}{TP + FN} = \frac{\text{预测对的正样本}}{\text{所有正样本}}$$

B. ROC 矩阵和 AUC

在分类任务中设定一个阈值，大于该阈值的为正例，小于该阈值的为负例。如果我们减小这个阈值，那么会有更多的样本被识别为正类，这会提高正类的识别率，但同时会降低负类的识别率。

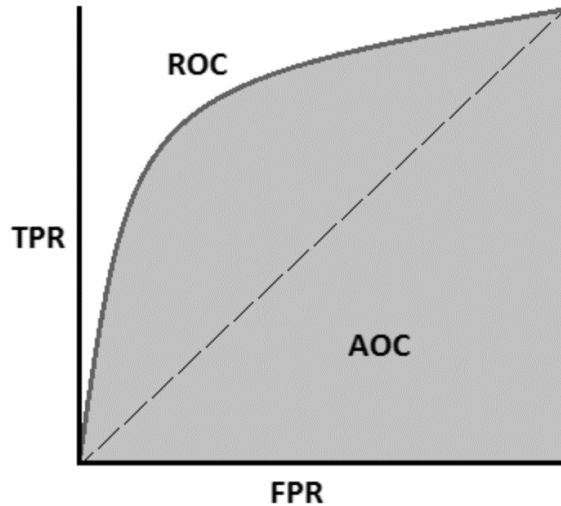


图 45: ROC 曲线

ROC 曲线的横坐标为假阳性率 (False Positive Rate, FPR)，即将负例错分为正例的概率

$$FPR = \frac{FP}{TN + FP}$$

纵坐标为真阳性率 (True Positive Rate, TPR) 即召回率

$$TPR = \frac{TP}{TP + FN}$$

设定不同的阈值从而得到 ROC 曲线，越靠近左上角越好，即召回率越高，误诊率越低。而当 $FPR = 0, TPR = 1$ 时为最优分类点，分类器对应的 ROC 曲线应该尽可能靠近坐标轴的左上角，而对角线的位置意味着分类器的效果和随机猜测一样的差。

C. 评估代码与结果

我们使用下面的代码对模型进行评估

代码 14: 评估结果代码

```
model = LogisticRegression()
model.fit(final_data[col], final_data['TARGET'])
EYtr_proba = model.predict_proba(Xtr)[: ,1]
EYtr = model.predict(Xtr)
```

```

print('Training error')
print('AUC:', AUC(EYtr_proba,Ytr))
# 其他输出指标略

EYts_proba = model.predict_proba(Xts)[:,-1]
EYts = (model.predict(Xts))

print('\nTest error')
print('AUC:', AUC(EYts_proba,Yts))
# 其他输出指标略

```

根据代码输出，我们的评估结果较为良好。模型具有一定的拟合效果，AUC 值约为 0.678，较为接近 1；KS 值为 0.38288。综上，表明模型在区分正负样本方面具有一定的区分的能力。这表明我们的模型能够有效地区分高风险和低风险的贷款申请。

2.3.3 坏账预估模型

坏账金额预测模型是针对可能发生违约的贷款，通过分析违约贷款的债务资产比、信用评级、年龄等因素，拟合出了一个预测模型。该模型可以预测出贷款发生违约时的坏账金额，为贷款机构在发生坏账时提供了一定的防范和应对措施。

该模型的预测主要依靠计量经济学方法，该建模方法使用统计学的方法对经济、金融问题进行计量分析。基于此，小组成员基于本学期《计量经济学》课程所学的最小二乘法模型，拟合回归一个简易的坏账预估模型，可以对风险贷款进行坏账预估，协助银行等提前做好坏账准备，备抵因贷款发放不当而造成的信用减值损失。最小二乘法模型（OLS）是一种常用的线性回归分析方法。它通过最小化残差平方和来估计回归方程中的参数，是计量经济学中最经典、应用最广泛的估计方法。相比机器学习，其有如下地优势——

- 📖 **经济理论基础：**计量经济学模型基于经济理论，能够更好地解释客户信用风险的驱动因素。与机器学习模型相比，计量经济学模型能够提供有关变量影响的直观解释，有助于金融机构理解客户信用风险的成因。
- 📖 **可解释性：**计量经济学模型具有较强的可解释性，能够清晰地展示各个变量对违约风险的影响程度。与黑盒式的机器学习模型相比，计量经济学模型能够提供更有意义的结果，有助于金融机构制定更有针对性的风

险管理策略。

📖 **变量选择：**计量经济学模型可以通过经济理论对变量进行筛选，避免引入无关变量或冗余变量。与机器学习模型相比，计量经济学模型能够更好地处理变量选择问题，提高模型的预测性能。

📖 **模型优化：**计量经济学模型可以通过参数估计和模型选择等方法进行优化，以提高模型的预测性能。与机器学习模型相比，计量经济学模型能够更好地适应市场变化，提高风险管理的有效性。

建立坏账预估模型的基本步骤如下所示。

计量经济学模型建立Step

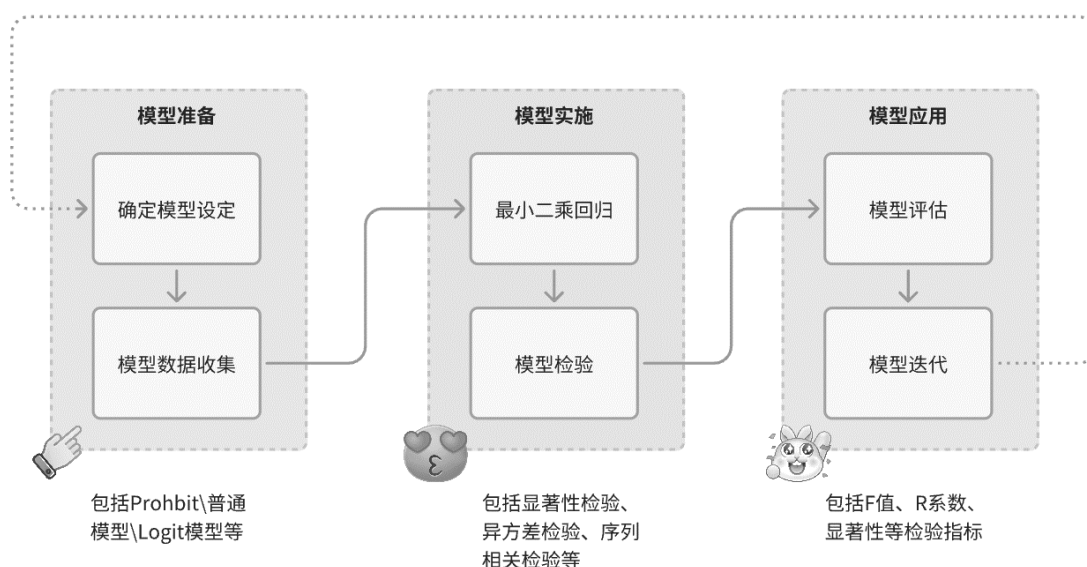


图 46：计量经济学模型建立步骤

该模型的建立主要依靠经济学和统计学知识，因此，其原理部分在此从略。小组同学使用 Eviews 软件（计量经济学分析软件）进行模型的回归拟合。并对模型进行不相干变量、多重共线性、异方差等多个问题检验和修正，在经过理论、拟合优度、显著性、偏误四因素的验证，最终得到了最终如图 47 的坏账模型。

Equation: UNTITLED Workfile: 坏账数据集EViews版::Untitled\				
View Proc Object Print Name Freeze Estimate Forecast Stats Resids				
Dependent Variable: BADDEBT				
Method: Least Squares				
Date: 11/11/23 Time: 21:59				
Sample: 1 26686				
Included observations: 26686				
Huber-White-Hinkley (HC1) heteroskedasticity consistent standard errors and covariance				
Variable	Coefficient	Std. Error	t-Statistic	Prob.
C	53644.12	906.3131	59.18939	0.0000
AGE	27.86515	8.749739	3.184684	0.0015
LTV	42667.94	663.9466	64.26411	0.0000
SCORE	-110.2494	1.393154	-79.13658	0.0000
OLDCUSTOMER	9268.519	366.0270	25.32195	0.0000
T	-8.156161	1.108366	-7.358726	0.0000
INCOME	0.076735	0.003033	25.29832	0.0000
JOB	-8981.047	356.5598	-25.18806	0.0000
R-squared	0.555734	Mean dependent var	21808.91	
Adjusted R-squared	0.555617	S.D. dependent var	25188.62	
S.E. of regression	16791.24	Akaike info crite...	22.29540	
Sum squared resid	7.52E+12	Schwarz criterion	22.29786	
Log likelihood	-297479.6	Hannan-Quinn criter.	22.29619	
F-statistic	4767.374	Durbin-Watson stat	1.982890	
Prob(F-statistic)	0.000000	Wald F-statistic	3684.156	
Prob(Wald F-statistic)	0.000000			

图 47：通过 Eviews 对坏账模型进行预测

模拟拟合的方程结果为：

$$\begin{aligned}
 \text{BADDEBT} = & \text{LeftLoanAmount} \times 0.5 + 27.87 \times \text{AGE} + 42667.94 \times \text{LTV} \\
 & - 110.25 \times \text{SCORE} + 9268.52 \times \text{OLDCUSTOMER} - 8.16 \times \text{T} \\
 & + 0.0767 \times \text{INCOME} - 8981.05 \times \text{JOB}
 \end{aligned}$$

其中，*LTV* 反映了借款人借款金额和拥有资产的比率。在文献《个人住房抵押贷款的市场风险研究》[3]显示 *LTV* 值越大，最终违约概率越大，坏账金额越多。在这种情况下，若借款人违约，银行收回的资产可能无法弥补贷款损失，从而导致银行的风险暴露越大。此外，较高的 *LTV* 值也可能意味着借款人负担过重，还款压力较大，进一步增加了违约的可能性。根据拟合结果显示，在其他条件不变的情况下，债务资产比每增加一个单位，坏账金额就有可能增加 42667.94 元。

计量经济学模型虽然看似简单，但其实蕴含了丰富的经济学原理，它是一种非常有效的预估坏账的方法。我们将这个模型嵌入至管理员页面，前端展示效果如图 48 所示。

贷款申请详情

申请ID: 2222

状态: 审核通过, 请注意风险状态

借款人信息

查看人员详情

姓名: 朱紫郑

电话: 15677791425

信用分: 49

申请信息

查看产品详情

产品: 小额快贷 (ID:3)

借款金额: ¥ 3000

分期方式: 6期

月利率: 2%

计利方式: 等额本金

申请时间: 2023-02-20 11:48:09

贷款用途: 生活

坏账金额预估

¥ 1,874.53

基于贷款信息和计量经济学最小二乘回归模型拟合得出, 仅供参考预测

图 48: 坏账预测模型前端展示效果

2.3.4 多渠道提醒

在我们的金融贷款管理系统中, 我们实施了一套全面的提醒机制, 旨在通过多种渠道确保用户能够及时收到还款通知。这一机制的核心是“全方位提醒服务”, 它结合了电子邮件通知和日历事件提醒两种方式, 为用户提供了一种无缝的还款提醒体验。

首先, 我们的系统还支持将还款日期自动添加到用户的日历应用中。这一功能允许用户在日历中直观地看到接下来的还款事项, 从而实现更好的时间管理。日历提醒通常会在还款日期的前几天(默认设置为前一天)自动弹出, 提醒用户准备还款, 这样用户就无需频繁检查邮箱或贷款管理系统, 也能确保不会错过任何还款时限。



图 49：日历提醒设置的效果

完成上述功能的核心代码如下——

代码 15：添加日历提醒的代码函数

```
addEvent_calendar(title,desc,time,index) {  
    var calanderURL = "content://com.android.calendar/calendars";  
    var calanderEventURL = "content://com.android.calendar/events";  
    var calanderRemiderURL = "content://com.android.calendar/reminders";  
    var calId;  
    var Cursor = plus.android.importClass("android.database.Cursor");  
    var Uri = plus.android.importClass("android.net.Uri");  
    var Calendar = plus.android.importClass("java.util.Calendar");  
    var main = plus.android.runtimeMainActivity();  
    var userCursor = plus.android.invoke(main.getContentResolver(),  
    "query", Uri.parse(calanderURL), null, null, null, null);
```

```

plus.android.invoke(userCursor, "moveToLast");
calId = index;
var ContentValues =
plus.android.importClass("android.content.ContentValues");
var events = new ContentValues();
events.put("title", title);
events.put("description", desc);
// 插入账户
events.put("calendar_id", calId);
var mCalendar = Calendar.getInstance();
plus.android.invoke(mCalendar, "set", Calendar.HOUR_OF_DAY, 11);
plus.android.invoke(mCalendar, "set", Calendar.MINUTE, 0);

events.put("dtstart", time*1000); // 设置开始时间
events.put("dtend", time*1000); // 设置结束时间
events.put("hasAlarm", 1); // 设置提醒
events.put("eventTimezone", "Asia/Shanghai"); // 设置时区, 默认中国上海
// 添加事件
var newEvent =
plus.android.invoke(plus.android.runtimeMainActivity().getContentResolver
(), "insert", Uri.parse(calanderEventURL), events);
// 事件提醒的设定
var id = plus.android.invoke(newEvent, "getLastPathSegment");
var values = new ContentValues();
values.put("event_id", id);

values.put("minutes", "1440");
values.put("method", "1");

plus.android.invoke(main.getContentResolver(), "insert",
Uri.parse(calanderRemiderURL), values); // 完成插入
}

```

这段代码即是一个用于在 Android 设备上创建日历事件的 JavaScript 函数。我们使用了 HTML5+ 的 API，这是 uni-app 框架中用于开发跨平台应用的一种技术。支持跨平台配置。这个函数的目的是在用户的日历应用中添加一个带有提醒的事件。下面是对这段代码的核心步骤简化分析：

1. **导入必要的类和接口：**Cursor, Uri, Calendar, ContentValues 等类用于操作数据库和日历事件。
2. **获取日历 ID：**使用 getContentResolver() 方法查询用户已存在的日历账户，

如果不存在则创建一个。

3. **创建日历事件：**使用 `ContentValues` 对象来设置事件的属性，如标题、描述、开始时间、结束时间等。
4. **设置日历 ID：**这通常与用户选择的日历账户相关联。
5. **设置事件的时区：**“Asia/Shanghai”。
6. **插入事件：**使用 `insert()` 方法将事件插入到用户的日历中。
7. **设置事件提醒：**创建一个新的 `ContentValues` 对象来设置提醒的属性。

其次，为了进一步增强提醒的即时性，我们的系统的管理端内置邮件提醒功能，当管理员（催收员）触发相关按钮后，自动向用户的注册邮箱发送提醒邮件。

我们通过 EmailJS 的中转服务器进行邮件发送，通过 markdown 语法的预设匹配功能设置统一的发信模板，仅需传递相关发信内容即可通过邮件中转服务器快速发送邮件，代码量少且配置简单。

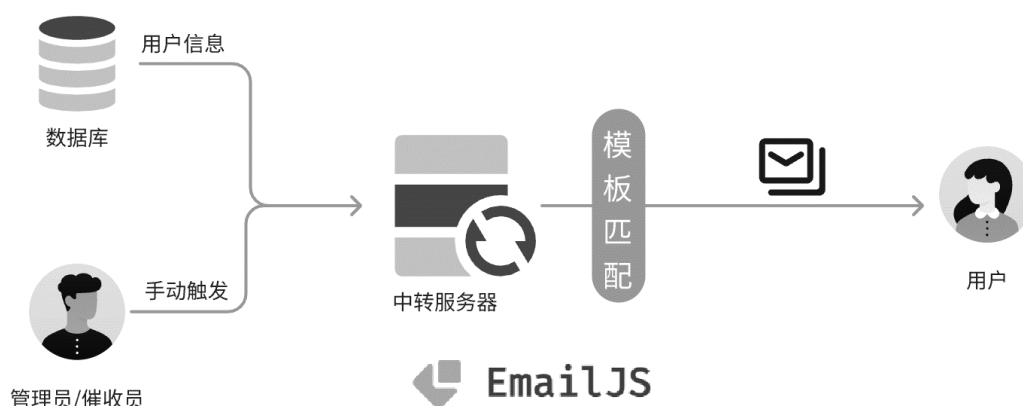


图 50：邮件提醒的流程

其核心代码如下所示——

代码 16：邮件提醒前端发送代码示例

```

window.onload = function() {
    document.getElementById('contact-
form').addEventListener('submit', function(event) {
        event.preventDefault();
        // 创建一个邮箱示例
        this.contact_number.value = Math.random() * 100000 | 0;
        // 邮件 ID (通过 EmailJS 获取)
        // 模板匹配的过程从略
        emailjs.sendForm('contact_service', 'contact_form', this)
            .then(function() {

```

第二章 针对复杂工程问题的方案设计与实现

```
        console.log('SUCCESS!');
    }, function(error) {
        console.log('FAILED...', error);
    });
});
}
```

邮件发送效果如图所示——



图 51：邮件提醒效果展示

通过这种电子邮件与日历提醒相结合的方式，我们不仅提升了用户体验，减少了用户忘记还款的可能性，而且也提高了我们服务的便捷性和可靠性。这种全方位提醒服务是我们信贷管理体系的基石，它为用户提供了一种既高效又友好的还款体验。

2.3.5 智能客服大模型

在 AI 技术的推动下，智能客服正逐渐成为提升用户体验的关键工具。为了实现这一目标，我们集成了科大讯飞的大模型技术，打造了一个高效、智能且人性化的客服系统。通过 uni-app 框架，我们实现了与星火大模型平台的 WebSocket 实时通信，以提供流畅的交互体验。

在实现过程中，我们首先通过 `getWebSocketUrl` 方法获取 `WebSocket` 的连接地址。接着，创建了一个 `WebSocket` 连接，并在连接成功后发送初始数据帧。这些数据包括用户输入的文本，以及一段预定义的 `prompt`，用于指导大模型如何回应用户的问题。`prompt` 的设计采用了角色扮演的手法，明确指出大模型应扮演软件说明书的角色，帮助用户解决软件问题。

在 `WebSocket` 通信的实现中，我们利用了 `uni-app` 提供的 `connectSocket` 方法来建立连接，并设置了相应的错误处理和消息接收回调。当服务器响应时，我们解析服务器返回的 `JSON` 数据，并根据数据内容执行相应的操作。如果接收到错误状态码，我们会显示错误提示并关闭连接；如果服务器状态正常，我们会继续监听消息，直到满足特定条件后关闭连接。

代码 17：智能大模型前后端交互代码

```
async function sendToSpark() {
  let myUrl = await this.getWebSocketUrl();
  // 设置WebSocket连接选项（从略）
  const connectOptions = {
    url: myUrl,
    method: 'GET',
    success: res => {
      realThis.wsLiveFlag = true; // 设置WebSocket连接状态标志
    }
  };

  // 创建WebSocket连接任务
  realThis.socketTask = uni.connectSocket(connectOptions);
  // 错误处理回调函数（从略）

  realThis.socketTask.onOpen((res) => {
    realThis.TextList = [];
    // 添加用户消息到列表
    realThis.TextList.push({
      "role": "user",
      "content": realThis.template + realThis.TEXT
    });
    // 构建发送给大模型的参数（从略）

    // 发送构建好的参数给大模型
    realThis.socketTask.send(JSON.stringify(params));
```

```
});  
  
// 接收大模型返回的消息  
realThis.socketTask.onMessage((res) => {  
    let obj = JSON.parse(res.data);  
    // 如果返回的状态码非零, 处理错误  
    if (obj.header.code !== 0) {  
        // 在这里处理错误  
        realThis.socketTask.close();  
    } else if (obj.header.status === 2) {  
        // 如果状态为2, 等待1秒后关闭连接  
        setTimeout(() => {  
            realThis.socketTask.close();  
        }, 1000);  
    }  
});  
}
```

这种实时通信的实现,使得用户能够快速获得智能客服的回复,显著提高了交互的响应速度。WebSocket 的双向通信特性也使得服务器能够主动推送信息,增强了系统的交互性。

为了确保大模型能够准确回答用户的问题,我们精心设计了 **prompt**, 限制了大模型的回答范围,使其仅基于我们提供的软件使用说明进行回答。通过这种方式,我们既保证了回答的准确性,又提高了回答的效率。实测结果表明,大模型能够根据 **prompt** 进行自动分点分段概述流程,并作适当补充拓展,证明了 **prompt** 设计的有效性。

智能客服系统不仅提升了用户体验,通过与科大讯飞大模型的深度融合,展现了强大的问题解决能力。

2.3.6 数据指纹技术

随着互联网的快速发展,网络贷款行业也面临着越来越多的安全风险。为了解决这个问题,我们的贷款系统引入了数字指纹技术,即通过 SHA256 加密,用于验证贷款申请数据的未被篡改。

数字指纹技术是一种基于哈希函数的加密技术,是区块链技术的底层方案。其中 SHA256 是一种常见的哈希函数,具有很高的安全性和不可逆性。它将任何文件或数据转换为唯一的标识符,类似于人类的指纹。这种标识符具有不可篡改

性和不可抵赖性，可以用来验证数据的完整性和真实性。

当用户提交贷款申请时，系统将自动使用 SHA256 加密对申请数据进行唯一标识，并将标识后的数据存储在数据库中。如果后续需要对申请数据进行验证或核查，可以通过比对数据库中存储的数字指纹来确认数据的真实性和完整性。数字指纹技术的唯一性和不可逆性可以防止欺诈行为的发生，保护用户的个人信息安全。

代码 18、代码 19 分别展示了后端数据指纹生成和前端数据指纹验证的过程。

代码 18: Kotlin 对贷款信息进行指纹生成

```
// SHA256 生成函数
fun hashSHA256(input: String): String {
    val SHA256_ALGORITHM = "SHA-256"
    try {
        // 通过MessageDigest 的getInstance 方法来获取 SHA-256 的实例，进行哈希计算
        val messageDigest = MessageDigest.getInstance(SHA256_ALGORITHM)
        val hashed =
            messageDigest.digest(input.getBytes(StandardCharsets.UTF_8))
        // 使用bytesToHex 函数将字节数组转换为十六进制的字符串，并返回这个字符串
        return bytesToHex(hashed)
    } catch (e: NoSuchAlgorithmException) {
        e.printStackTrace()
    }
}

private fun bytesToHex(bytes: Array<Byte>): String {
    // 使用map 函数来将字节数组中的每个字节转换为两位的十六进制数
    val hexString = bytes.map { "%02x".format(it) }.joinToString("")
    return hexString
}

// 生成指纹
fun encryptLoanApp(phone: String, usrName: String, loanAmount: Double, T:
Int, date: String, prevHash: String): String {
    val data = "$phone,$usrName,$loanAmount,$T,$date,$prevHash"
    return hashSHA256(data)
}
```

代码 19: Javascript 对贷款信息进行指纹验证

```
function verifySHA256(input, hash) {
    return hash === sha256(input);
}
```

```
}  
    // 生成比对哈希值  
function sha256(input) {  
    var hash = CryptoJS.SHA256(input).toString();  
    return hash;  
}  
    // 验证  
function verifyLoanApp(phone, usrName, loanAmount, T, date, prevHash,  
encryptedData) {  
    var data = [phone, usrName, loanAmount, T, date, prevHash].join(',');  
    return verifySHA256(data, encryptedData);  
}
```

整体流程如图 52 所示。数据指纹由前端生成后端验证，基本原理是在数据生成时，根据数据内容计算出一个唯一的“指纹”，这个“指纹”反映了数据的特征，这个“指纹”反映了数据的特征，并且对数据内容的任何改动都会导致“指纹”的变化。因此，通过对比数据修改前后的“指纹”，就可以判断数据是否被篡改。

在前端生成数据指纹的过程中，生成因子包含了上一申请的哈希值，一同进行新哈希值的生成，这样前后两个申请块就相互关联起来。这种设计可以有效防止数据篡改，因为如果数据被篡改，那么生成的数据与原始数据对比，至少有一位会发生变化，进而导致哈希值的变化。通过对比前后哈希值，就可以发现数据是否被篡改。

在这个过程中，采用的是 SHA256 的指纹生成方法。SHA256 是一种单向散列函数，它可以将任意长度的数据转换为一个固定长度的哈希值。由于 SHA256 加密算法的计算量较大，因此，即使是对同一数据的多次加密，得到的哈希值也会有所不同。这一特性使得 SHA256 算法在数据指纹生成中具有很高的安全性，可以有效防止数据的篡改。

同时，由于 SHA256 算法的加密过程是单向的，即从原始数据可以计算出哈希值，但是从哈希值无法反推出原始数据。因此，基于 SHA256 的数据指纹也能保证贷款信息的第三方不可知性。因此，仅拿到数据指纹的第三方也无法获取到原始数据。

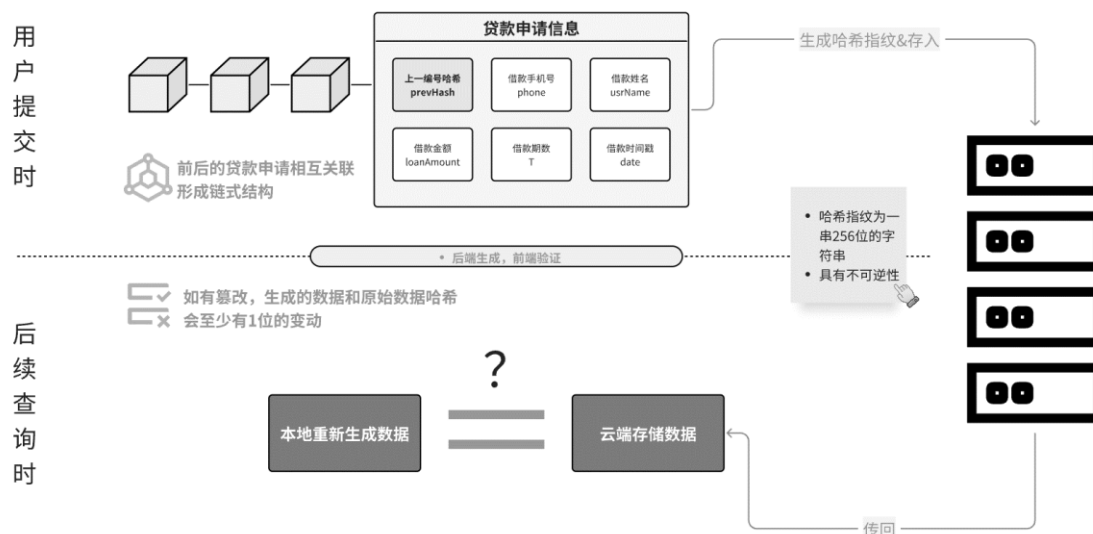


图 52：数据指纹生成和验证过程

数字指纹技术即是当今区块链技术的一个简单的、初步的底层实现，然而，这种技术并非万无一失。尽管数字指纹能够确保数据在未被篡改的情况下保持一致，但是，如果黑客能够攻破数据库的防线，就有可能对数据库中的信息进行修改，从而破坏数据的完整性。为了解决这个问题，最佳的做法是将数据库进行分布式处理。这样，数据就被分散存储在多个节点上，而不是集中存储在一个或几个节点上。这种分布式的存储方式可以有效地提高数据的安全性，因为即便黑客攻破了一部分节点，也无法篡改整个数据库的数据，因为其他节点的数据可以提供校验。受限于开发时间和项目其他核心功能排期安排，在综设 III 中未能实现分布式功能，如果条件允许，可以在后续扩展中实现。

第三章 测试环境构建与测试实施

3.1 测试环境与工具

本次测试主要在以下环境中进行：

表 18：测试环境概述

环境一级指标	环境二级指标	环境参数
安卓端	操作系统	Android 11.0
	设备型号	Huawei Nova 8 Pro
	分辨率	1236 x 2676 pixels
网页端	操作系统	Windows 11
	浏览器	Google Chrome 113.0.5672.127
网络环境		Wi-Fi 和 4G 移动网络
后端	数据库	PostgreSQL
	操作系统	Windows 11 和 Linux（WSL）
其他环境因素	时区和时间	标准北京时间
	多语言环境	仅中文环境

为了有效地完成项目测试，我们的小组选用了多种自动化测试工具。其中包括：

- **HbuilderX-for-uniapp-test**：专为安卓端 APP 设计的自动化测试工具，它可以帮助我们进行高效的 APP 测试。
- **Selenium**：这是一个广泛使用的 Web 应用程序测试包，可以针对网页进行各种复杂的自动化测试。
- **DevTools**：这是一个前端调试工具台，它为我们提供了一个直观、高效的界面来进行前端代码的调试和优化。
- **Apipost**：这款工具集成了 Postman 和 Mock 的功能，使我们能够轻松地进行后端数据库和接口测试。

这些工具的灵活使用和恰当配置，帮助我们以更高的效率和更高的质量完成了项目测试任务。

3.2 测试功能点与样例

为了进行完善的项目测试，我们对项目设计了如下的测试用例

表 19：项目测试功能点

序号	测试端	测试项目	测试点	测试用例	预期结果
1	用户端 APP	注册与登录	输入正确的信息实现注册	账户名： 1333333333 密码 123456	注册成功，跳转登录
2			输入已存在的手机号进行注册		提示“手机号已注册”
3			输入正确用户名和密码登录		登陆成功，跳转首页
4			输入错误的密码登录系统	账户名： 1333333333 密码 123	提示“账号或用户名错误”
5		借款功能	漏填信息进行申请	ID 为 1 的产品下，不选择贷款用途	阻止进入下一步，提示未填写完整
6			正确信息完成借款	ID 为 1 的产品下，填写所有信息	进入下一步
7			查询申请情况	查询上一测试项目的申请贷款	能够正确展示当前贷款状态信息，与数据库一致
8		还款功能	无逾期正常进行还款	为编号 2514 的贷款申请正常还款（该编号下无逾期还款）	可以正常还款，相关数据能够及时更新
9			存在逾期情况下进行还款	为编号 1214 的贷款申请正常还款（该编号下存在 2 项逾期还款）	可以正常还款，相关数据能够及时更新
10		辅助功能	更新个人信息	将居住地更改为四	可以正常更改

第三章 测试环境构建与测试实施

				川省成都市	
11			信用分查询	查询更新个人信息后的信用分	可以正常查询，且查询结果和评分卡一致
12			权限检查	关闭定位权限，再次申请贷款	提示“未开启位置定位权限”
13			智能客服	向客服提问“如何借款”	返回正确的流程步骤
14			日历提醒	点击添加还款计划到日历	系统日历出现相应日程
15		数据大屏	数据大屏查询	点击数据大屏进入	大屏能够正确显示相关数据图标，且与后端一致
16			贷款产品查询	查询 ID 为 1 的贷款产品	能够正常返回相应结果
17		贷款产品	贷款产品新增	新增贷款产品“装修贷”，且发布	列表刷新后能正常显示，且在 APP 端可以看到加入结果
18			贷款产品删除	删除刚刚新增的贷款产品	成功删除
19	管理端 Web	贷款申请	贷款申请详情查询	查询用户 1333333333 的贷款申请	列表成功返回全部数据
20			贷款申请筛选	筛选列表“待审批”的全部申请	列表展示正确结果
21		用户管理	用户列表查询	查询全部用户的信息	列表成功返回全部数据
22			用户详细查询	查询用户 1333333333 的基本信息	列表成功返回全部数据，且与用户填写一致
23		风险查询	申请风险列表查询	查询当前风险的申请的列表	列表展示正确结果

信软学院进阶式挑战性综合项目 III 报告

24			用户邮件提醒	为高危用户发送邮件通知	成功发送邮件，用户端收到邮件
25			坏账预测查询	点击任意高危申请，查询坏账值	根据模型结果能够正确返回坏账的预估值
26			全局设置修改	将坏账预测关闭	不再展示坏账预测
27			全局阈值修改	将风险分下调至400	风险申请400分以上的不再标记为高风险
28	数据库	并发测试	每个接口在1分钟内发送200次查询请求	观察延时情况	延迟低，无阻塞现象发生
29		数据指纹	数据指纹生成正确性	检测各个申请的数据指纹，并对比	各项数据指纹生成正常
30	UI	适配性	检测APP页面是否适配各个型号页面	在Chrome手机模拟器调整页面大小	UI不变形
31		页面性能	检测各个页面的加载性能	利用Chrome开发者工具检测性能	各个页面性能耗用合理

功能性测试样例全面覆盖了贷款系统的管理端和用户端的多个功能模块，包括注册与登录、借款功能、还款功能、辅助功能等。在测试用例设计方面，我们不仅考虑了正常流程的测试，还充分考虑了各种异常情况的处理，例如输入错误的账户名或密码、漏填信息进行申请等。

操作性测试样例涵盖性能、安全、适配，检查了各个接口的并发情况、数据安全情况、适配性。测试用例可以很好的检测出各个系统在不同的运行环境期间的使用情况。

3.3 测试方法与实施

3.3.1 人工测试

人工与工具测试是软件测试最基础、原始的方法，与自动化测试相结合，能够更全面地评估系统的质量。人工测试主要针对以下几个场景的样例进行测试：

第三章 测试环境构建与测试实施

1. 复杂和边缘场景：对于一些复杂或者边缘场景，自动化测试可能无法完全覆盖，此时需要人工测试来进行补充，以确保系统在这些特殊场景下的表现。
2. 用户体验关注点：人工测试能够更贴近真实用户的使用情况，对于系统的用户体验，例如界面的友好性、交互的流畅性等方面，人工测试能够提供更有效的评估。

例如，在进行 UI 适配性测试时，我们可以利用 Chrome 浏览器的手机模拟功能，来模拟各种不同型号、尺寸大小的手机页面适配情况。通过模拟不同型号和尺寸的手机，我们可以更加精确地检查系统在各种设备上的表现，及时发现并解决潜在的 UI 适配问题。这样，我们可以确保贷款系统在各种不同屏幕尺寸和分辨率下的表现都能达到预期，为用户提供良好的视觉效果和操作体验。同时，通过这种方法，我们也能及时发现并解决潜在的 UI 适配问题，提升系统的稳定性和可靠性。

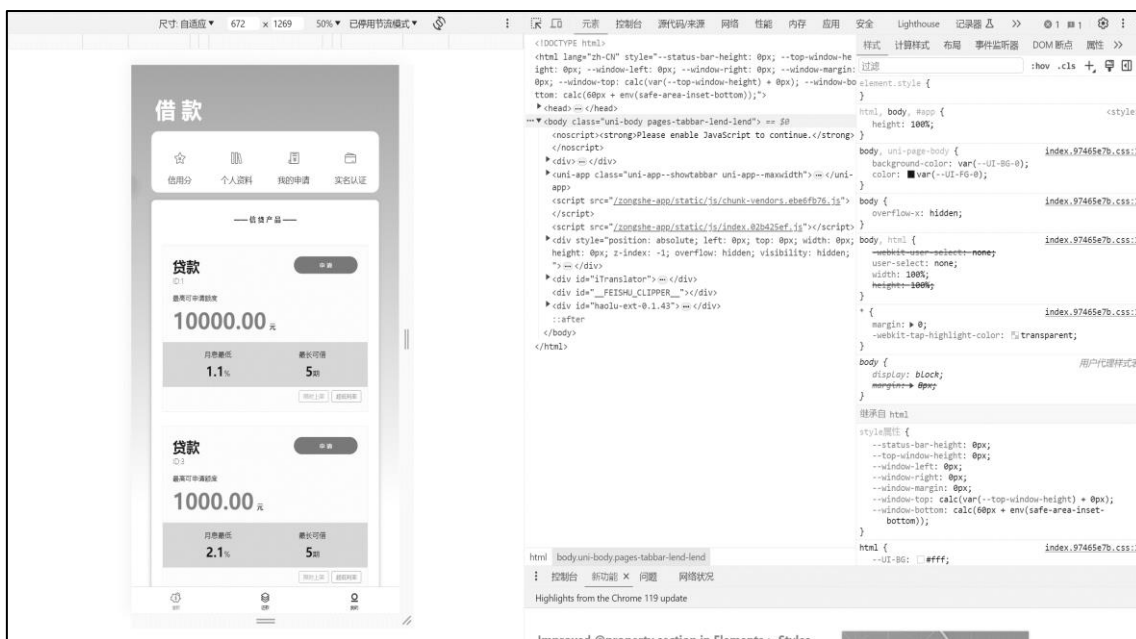


图 53：使用 Chrome 进行 UI 适配性检测

3.3.2 交互性测试

交互测试是针对性能、用户体验等特殊需求的测试，我们使用了 Chrome 内置的交互测试模块——性能分析器和 Lighthouse 来进行测试。

在进行页面性能测试时，我们采用了 Chrome 浏览器的性能测试模块进行人

工测试。通过该模块，我们可以对页面的性能进行深入的分析，包括页面脚本执行效率、内存使用情况、渲染性能等关键指标。这些数据有助于我们更合理地设计脚本逻辑，提升页面的性能。

当我们发现页面性能存在瓶颈时，我们可以采取一系列优化措施来提升页面性能。比如，我们可以优化图片的大小，减少 HTTP 请求的次数，或者设置合适的缓存策略等。这些优化措施都可以有效地提升页面的加载速度和运行效率，从而提升用户体验。测试实施过程产生的日志消息如图 58 所示，从日志消息中，我们可以看到每一段脚本语言的执行时间都被进行了计时测试，其中的“ts”代表的是挂载和执行时间。这些数据可以帮助我们更准确地了解脚本语言的执行效率，从而找到可能存在的性能问题，并进行针对性的优化。

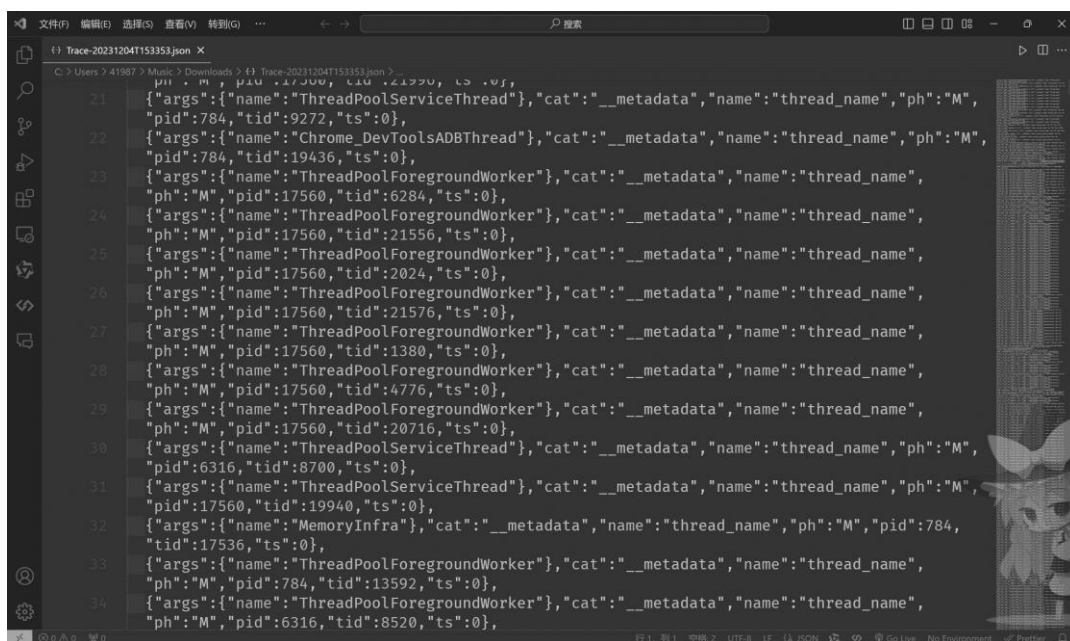


图 54: chrome 页面性能测试日志

同时，我们的小组同学针对 web 端采用了 Lighthouse 工具进行深入的用户体验测试。这款工具能够模拟真实用户的访问行为，包括浏览、点击、滚动等操作，以此来评估网页的性能和用户体验。通过这种模拟真实用户行为的方式，我们可以更准确地识别出网页中存在的问题，并针对性地进行优化，从而提供更好的用户体验。

在测试过程中，测试工具会关注网页的加载速度、响应时间、页面布局等多个方面。这些因素都会影响到用户的体验，因此我们需要对它们进行详细的分析和评估。例如，网页加载速度过慢可能会导致用户失去耐心，从而选择离开页

面；响应时间过长可能会影响用户的操作效率，降低用户的满意度；页面布局不合理可能会使用户找不到需要的信息，影响用户的体验。此外，我们还会对网页的交互设计、导航逻辑等方面进行检查，确保它们能够满足用户的需求，提供直观、易用的操作体验。例如，交互设计是否符合用户的使用习惯，是否易于理解和操作；导航逻辑是否清晰，是否能够帮助用户快速找到需要的信息等。

LightHouse 也会对安全性、渲染性能进行分析，如图为其监测指标。



图 55: LightHouse 的监测指标

3.3.3 自动化测试

自动化测试是一种使用软件工具和脚本来模拟用户行为和操作，对应用程序进行测试的过程。它可以帮助开发团队在软件开发过程中更快速、更准确地发现和解决潜在的问题和漏洞，提高软件质量和稳定性。我们通过 Ktor 的相关测试框架编写了一套针对后端的接口测试脚本。

以查询用户基本信息为例，测试脚本如下。

代码 20: 自动化测试获取用户基本信息接口

```
import io.ktor.application.*
import io.ktor.http.*
import io.ktor.server.testing.*
import io.ktor.server.application.*
import online.rainy.models.User
import online.rainy.views.userInfo.InfoView
import org.amshove.kluent.*
```

信软学院进阶式挑战性综合项目 III 报告

```
import org.junit.jupiter.api.*
import java.io.*

class UserBasicInfoRouteTest {

    @Test
    // 当提供有效的数据时, 应该返回成功
    fun `post basicinfo should return success when valid data`() =
withTestApplication(Application::module) {
    with(handleRequest(HttpMethod.Post, "/basicinfo")) {
        addHeader(HttpHeaders.ContentType,
ContentType.Application.Json.toString())
        setBody("[{\"phone\": 13887878787, \"sex\": \"M\", \"birthday\":
2003-04-15, \"city\": \"四川省-成都市-成华区\", \"email\":
\"user@example.com\"}]")
    } {
        val response = call.respond()
        response.status() assertEquals(HttpStatusCode.OK)
        response.contentLength() > 0
    }
}

    @Test
    // 当电话号码未找到用户时, 应该返回没有用户的信息
    fun `post basicinfo should return no user when phone not found`() =
withTestApplication(Application::module) {
    with(handleRequest(HttpMethod.Post, "/basicinfo")) {
        addHeader(HttpHeaders.ContentType,
ContentType.Application.Json.toString())
        setBody("[{\"phone\": 139999999999, \"sex\": \"M\", \"birthday\":
2003-04-15, \"city\": \"四川省-成都市-成华区\", \"email\":
\"user@example.com\"}]")
    } {
        val response = call.respond()
        response.status() assertEquals(HttpStatusCode.OK)
        response.contentLength() > 0
    }
}
}
```

在这个测试类中，我们使用了 `withTestApplication` 函数来创建一个可以测试的 Ktor 应用程序实例。我们定义了两个测试用例：

- 当提供有效的数据时，`post basicinfo should return success when valid data` 测试验证了正确的响应状态码（200 OK）和响应内容长度。
- 当提供的电话号码在数据库中找不到用户时，`post basicinfo should return no user when phone not found` 测试验证了应用程序是否返回了 `no user` 的响应。

此外，这个测试类还具有很好的可扩展性。你可以根据实际需要添加更多的测试用例，比如验证用户输入的数据格式不正确时，应用程序是否能够正确处理并返回错误信息。只需要添加新的测试方法，并使用相应的 `with(handleRequest)` 来模拟请求和行为即可。

同样，前端的一些模拟操作也可以用自动化测试工具完成。这里我们使用的是 Uniapp 封装的 Jest 测试框架。测试内容为登录接口。

代码 21：自动化测试登录页面

```
// 导入测试框架
const { describe, it, expect } = uniTest;

// 导入需要测试的组件
import login from '@components/login.vue'; // 导入登录组件

describe('登录页面测试', () => {
  it('测试手机号输入框', () => {
    // 创建组件
    const component = login;
    component.phone = '13333333333'; // 设置初始值
    // 查找输入框并模拟输入
    const input = component.$el.find('.phone-input');
    input.val('13333333334');
    // 验证手机号输入框是否可以接收新值
    expect(input.val()).toEqual('13333333334');
  });

  it('测试密码输入框', () => {
    // 创建组件
    const component = login;
    // 查找输入框并模拟输入
```

```
const input = component.$el.find('.password-input');
input.val('新的密码');
// 验证密码输入框是否可以接收新值
expect(input.val()).toEqual('新的密码');
});

it('测试登录按钮', async () => {
  // 创建组件
  const component = login;
  // 模拟输入手机号和密码
  component.phone = '13333333333';
  component.pwd = '123456';
  // 模拟点击登录按钮
  component.login();
  // 验证是否调用了登录方法
  expect(component.login).toHaveBeenCalled();
});

// 更多测试, 在此从略...
});
```

这段代码使用了 Jest 测试框架，对登录页面的功能进行了全面测试。Jest 是由 Facebook 开发的测试框架，具有零配置、并行运行测试和丰富的 API 等优势，使得开发者可以快速开始编写测试。在这段代码中，对登录页面的手机号输入框、密码输入框和登录按钮的功能进行了测试，以确保它们的正常工作。同时，代码使用了模拟和快照测试功能，使得开发者可以轻松地进行模拟测试，验证组件的某些特定行为是否符合预期。整个代码结构非常清晰，每个测试用例都针对一个特定的功能进行测试，易于阅读和维护。通过这段代码，可以确保登录页面的正常工作，同时也有利于后续的维护和优化。

具体来看，这段代码对登录页面的功能进行了测试。首先，它导入了测试框架和需要测试的登录组件。然后，它定义了一个描述登录页面测试的测试套件，其中包含三个测试用例——第一个测试用例是针对手机号输入框的，它创建了登录组件的实例，通过查找输入框并模拟输入，验证手机号输入框是否可以接收新值。第二个测试用例是针对密码输入框的，它同样创建了登录组件的实例，通过查找输入框并模拟输入，验证密码输入框是否可以接收新值。第三个测试用例是针对登录按钮的，它创建了登录组件的实例，模拟输入手机号和密码，然后模拟点击登录按钮，验证是否调用了登录方法。

3.4 测试结果与修复

3.4.1 测试结果

通过人工测试、交互性测试、自动化测试等多项测试。形成的最终测试结果如下所述。

首先，功能性方面。所有测试点均在期末验收时通过全部测试用例。我们重点关注了系统的主要功能模块，包括但不限于用户管理、权限控制、数据处理、业务逻辑等。特别是涉及到数据的增、删、改、查等操作，为了确保数据在系统中的流转和处理符合预期，我们使用 Apipost 对逐个接口进行自动化测试。执行结果 100%通过。

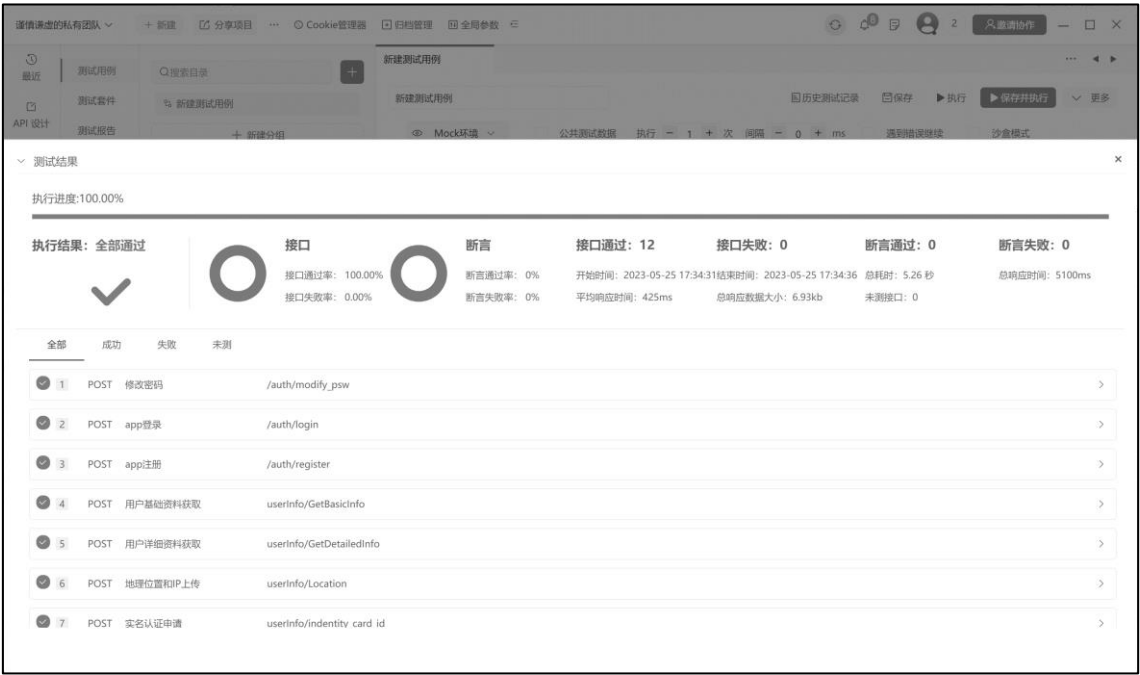


图 56：使用 Apipost 进行接口功能测试的结果

在业务逻辑功能测试中，我们针对系统的核心业务流程进行了深入测试，按照测试用例的设计，覆盖了各种正常和异常场景。通过人工跑通流程，我们验证了业务逻辑的正确性和稳定性。

性能方面，页面渲染性能良好，各项指标处于合理范围以内。我们进行了包括后端压力测试和前端渲染性测试等，系统的响应时间、吞吐量和都基本达到了预期要求，除了个别接口涉及较为复杂的工作流程，查询时间在大数据容量下欠佳等，都表现出良好的指标。这些测试结果证明了系统在高并发和高负载情况下

的优秀性能。图 57 展示的时在贷款申请表单页面进行前端渲染性测试的结果，可以看到性能耗用符合正常范围。

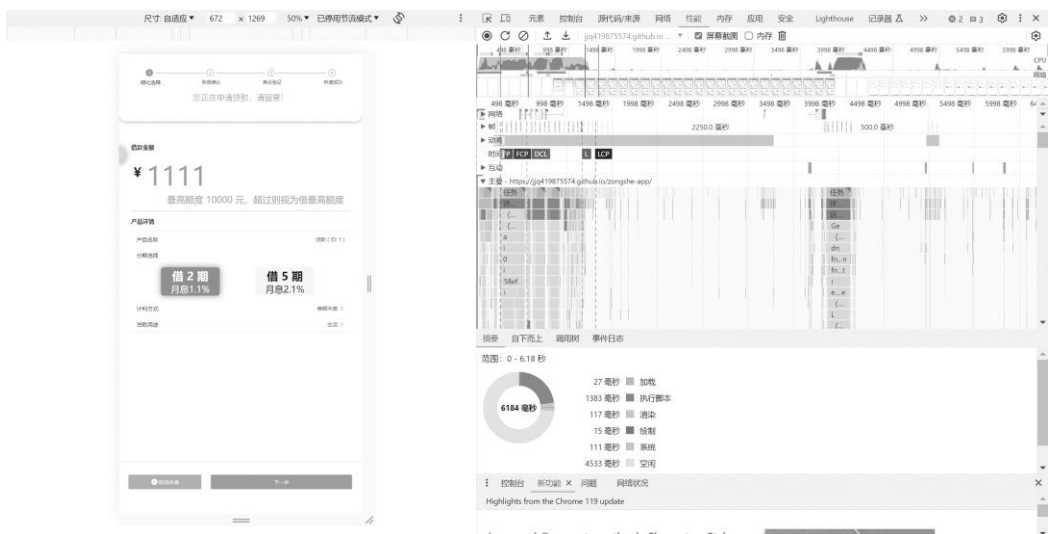


图 57：贷款申请表单页渲染性能测试

用户体验等综合性方面，借助 Lighthouse 自动化测试工具，我们对系统进行了相关测试。以管理端风险告警页页面为例子，整体的页面性能达到了 91 分（满分 100 分）。总阻塞时间¹和累积布局移位²两项指标位于绿色（优秀）水平，三项指标位于黄色（正常）水平，没有指标位于不及格水平。

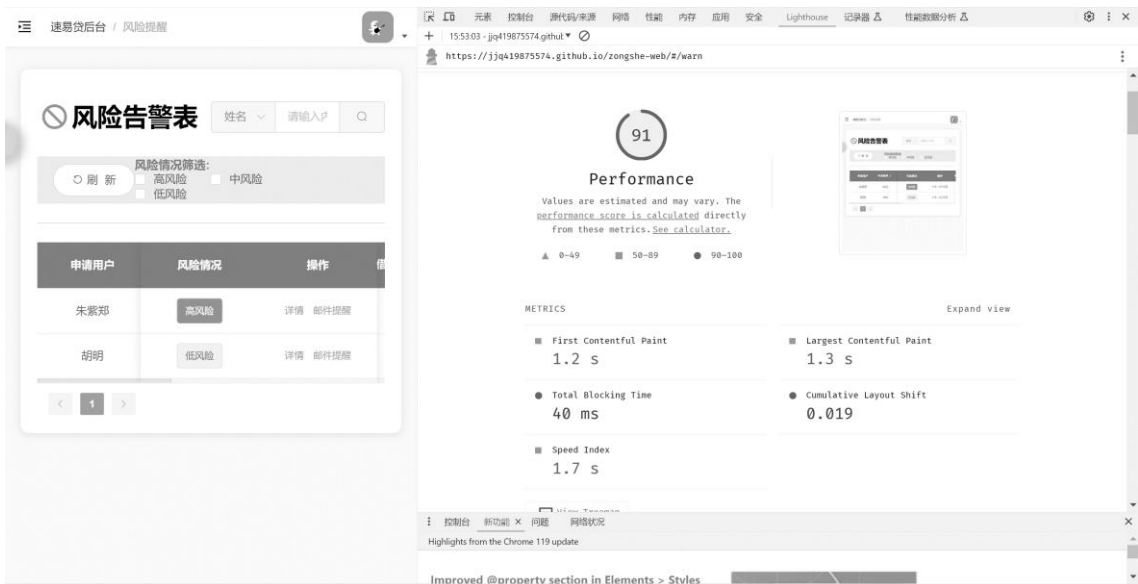


图 58：管理端风险告警页页面综合性能测试

¹ 总阻塞时间：用以评价页面加载性能，是指 JS 代码被阻塞无法继续运行，等待某几行代码执行结束的同时操作的总耗时
² 累计布局移位：评价布局的合理性，是指在不同分辨率、屏幕大小的页面下进行布局测试，观察核心按钮的位置移动。

3.4.2 结果分析

通过期末进行的测试结果，我们可以得出以下结论

- 业务功能：
 - a) 业务逻辑：各业务逻辑流通走向符合预期，数据处理正确恰当，各个功能都能正常运行。
 - b) 易用性：系统提供良好的操作流程、指引提示、帮助文档等，测试结果良好，用户可以轻松上手，方便使用。
 - c) 兼容性测试：产品在不同操作系统、浏览器等环境下的兼容性问题，测试结果良好，产品能在各种环境下正常运行。
- 性能安全：
 - a) 响应速度：测试结果显示，产品的响应时间短，能够迅速响应用户的操作，提高用户的工作效率。
 - b) 稳定性：测试结果显示，产品在长时间运行过程中，系统稳定，无明显卡顿、崩溃等现象。
 - c) 安全性：数据库对核心指标采用 SHA256 进行指纹生成和校验；敏感操作需要得到用户的二次认证，各项基本安全防护使得系统具有一定安全性，
- 用户体验：
 - a) 系统界面：产品界面美观、清晰，符合用户审美和使用习惯。产品操作流程简单明了，易于上手，用户可以快速掌握。
 - b) 帮助与支持：产品提供了详细的帮助文档和在线支持，同时辅助 AI 大模型智能客服，方便用户解决问题。
 - c) 交互设计：产品的交互设计合理，用户可以轻松地完成各种操作。

综上所述，可以认为该产品在质量方面具有正常水平，通过测试。

3.4.3 修复情况

当然，在正常的测试过程中（包括在项目实施过程中的测试，而非最终期末测试），我们的确发现了一些 Bug。

在测试过程中小组通过飞书文档进行整理汇总。交由指定负责人进行跟踪改进。Bug 上报后，我们对 Bug 进行分类、标记、优先级设定和状态更新等操作，

确保每一个 Bug 都能得到有效的跟踪和管理。小组对 BUG 的优先级进行三级分类，P0 为最紧急，关系到系统核心流程进行，P1、P2 级别依次降低。为了保证 Bug 的高效解决，我们制定了一套清晰的清理计划。所有的 Bug 表都会每个月进行一次清理，而 P0 级别的 Bug 则会在每周进行一次清理。这样的计划保证了所有项目的 Bug 都能得到及时和有效的解决。

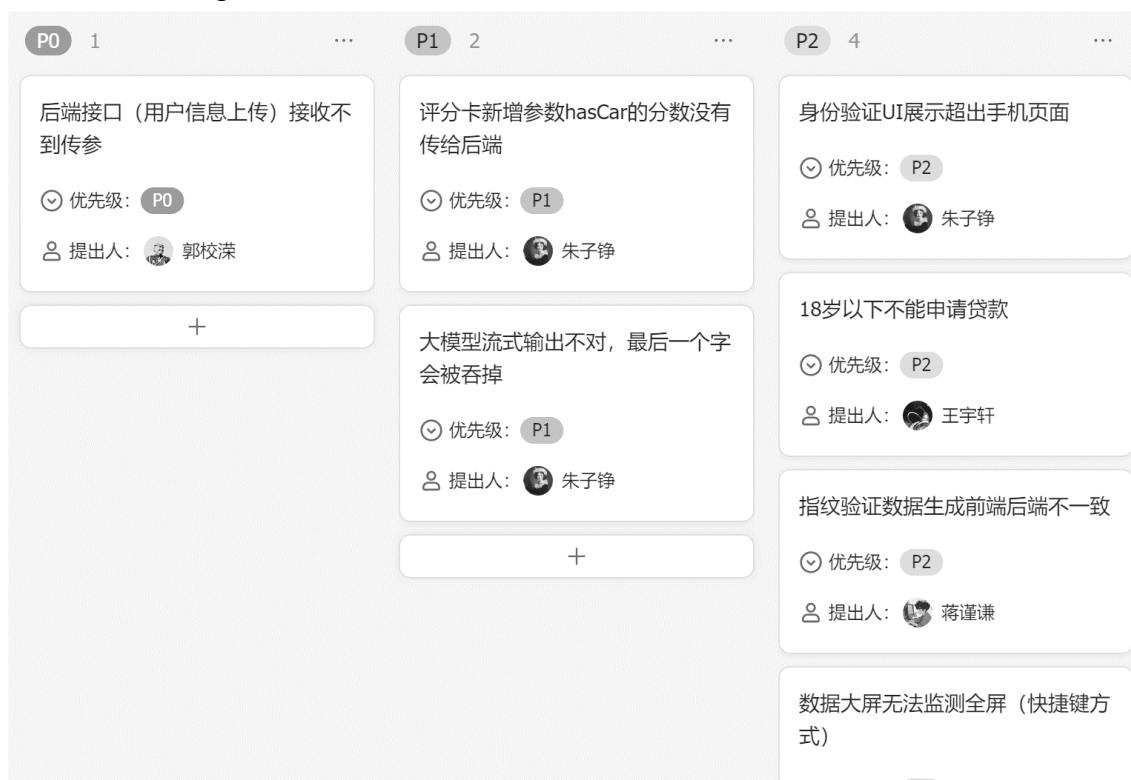


图 59：小组通过飞书文档进行 bug 汇总（部分）

截至目前，所有项目的功能性 Bug 已经全部解决，并验证通过。

第四章 知识技能学习情况

在本次综合设计 III 的实施过程中，小组团队成员们持续吸收新的知识和技能，他们通过扩展个人的知识库和不断的编程实践，实现了自我提高和专业知识的成长，取得了丰富的成果。下面我们将从几个方面具体说明。

4.1 编程语言与技术框架

在我们的金融贷款风控和管理系统项目中，小组成员熟练运用现代编程语言和技术框架，技术先进。

前端方面，我们不仅熟练掌握了 Vue.js 框架和 uniapp 跨平台开发工具的基础语法，还深入理解了组件化、指令系统、生命周期钩子等核心概念，这些知识的应用使得我们的系统功能丰富且易于扩展。

团队成员们不仅停留在基础层面，还精通 Vue 的高级特性，如 Vuex 状态管理模式和 Vue Router 路由系统。这些高级工具的运用，不仅提升了系统的整体架构，还为用户带来了更加流畅和一致的交互体验。通过 Vuex，团队有效地管理了应用的状态，确保了数据的一致性和响应性；而 Vue Router 则为用户导航提供了清晰的路径，简化了页面间的跳转。

在前端开发过程中，团队坚持组件化和模块化的设计思想，这不仅提高了代码的可维护性，也极大地增强了代码的可复用性。此外，通过精心设计的路由系统，用户能够轻松地在贷款应用的不同功能模块间切换，从而提升了整体的使用效率。

跨平台开发的潜力也被充分挖掘，小组成员熟悉并有效利用了 uniapp 的特性，通过其内置的跨平台渲染引擎，快速构建了适应不同操作系统的贷款应用。这种跨平台的能力，为贷款应用提供了更广泛的市场覆盖，同时也减少了开发和维护的成本。

为了提高开发效率，团队成员采取了自动化工具 vue-cli 来搭建前端项目环境。这一举措不仅缩短了开发周期，还确保了项目环境的一致性，为后续开发和测试工作打下了坚实的基础。

在后端部分，我们的团队则展现了对后端开发技术的精湛掌握。他们利用

Kotlin 编程语言和 Ktor 框架构建了贷款应用和管理系统的后端服务。Kotlin 的强类型特性和 Ktor 的高性能特性，为后端服务提供了稳定而强大的支持。

数据库设计方面，我们能够基于《数据库原理及设计》课程的所学，精心设计了数据库结构，确保了数据的完整性和准确性。他们通过 SQL 语言进行高效的数据操作，包括数据的插入、删除、更新和查询，这些操作的精确执行对于金融应用的稳健运行至关重要。

此外，团队还利用 Ktor 框架实现了 RESTful API 接口，为前端提供了高效、可靠的数据交互机制。这些 API 接口的设计遵循了最佳实践，保证了前后端接口的一致性和兼容性，确保了整个系统的协同工作。

在项目的开发过程中，我们的前端和后端团队紧密合作，确保了技术栈的一致性和开发流程的顺畅。这种跨领域的协作不仅提升了系统的质量和性能，也为项目的成功实施奠定了坚实的基础。

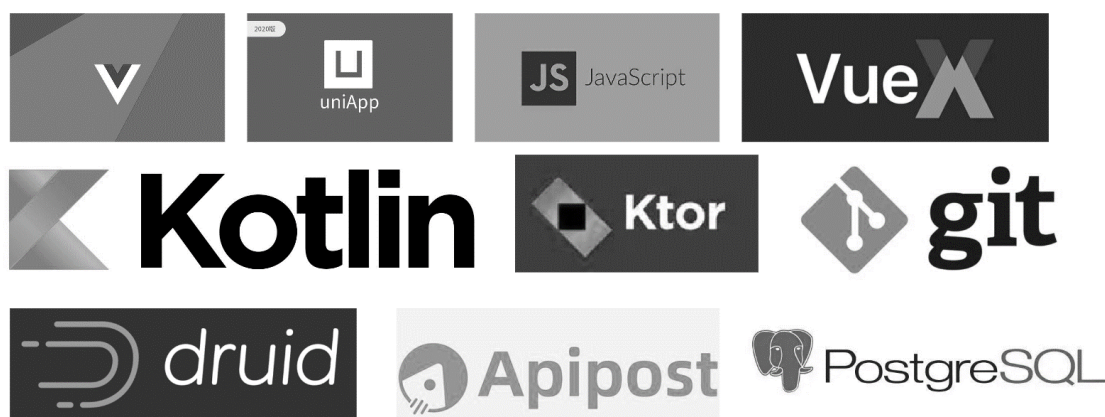


图 60：项目所用主要技术矩阵

4.2 系统设计与模式架构

在贷款项目的开发过程中，我们深入研究了系统设计与模式架构的相关知识。通过对各种设计模式的学习和实践，我们提高了对软件设计的理解和相关应用能力。

首先，我们了解了软件设计的基本原则，如模块化、分层、组件化和面向接口编程等。这些原则帮助我们构建了一个结构清晰、易于维护和扩展的贷款系统。组件化的思想使得我们的开发更并行化，风格更统一化。

其次，我们学习了一些常用的设计模式，如工厂模式、单例模式、观察者模式、策略模式等。这些模式在贷款系统的开发中发挥了重要作用。例如，我们使

用工厂模式创建不同类型的贷款产品，通过一个专门的工厂类来统一管理和控制对象的创建。

此外，我们还学习了一些软件架构模式，如 MVC（模型-视图-控制器）模式等。在贷款系统中，我们采用了 MVC 模式将系统分为三个层次：模型层负责业务逻辑和数据管理，视图层负责用户界面展示，控制器层负责处理用户请求和调度模型层与视图层之间的交互。这种分层架构使得系统各部分的职责明确，降低了模块之间的耦合度，有利于后期的维护和升级。

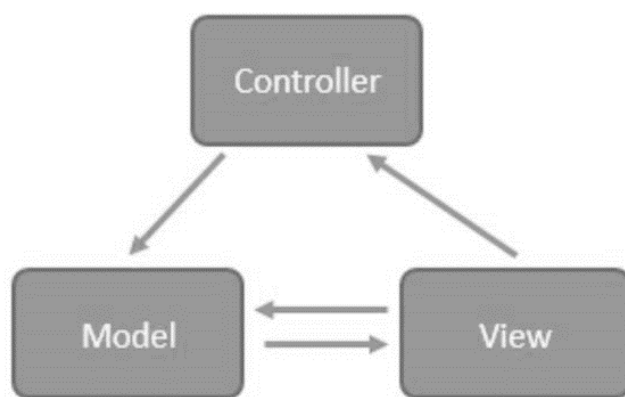


图 61：MVC 模式

4.3 问题解决与资源收集

在贷款项目的开发过程中，我们不仅提高了问题解决能力，还增强了知识学习和资源收集的能力。

首先，我们学会了利用搜索引擎和社区资源进行问题解决，例如 Stack Overflow、GitHub 等。当遇到问题时，我们能够有效地利用网络资源，搜索相关知识和经验，并及时参考社区中其他开发者的解决方案，从而提高了解决问题的效率。在贷款项目中，我们遇到了一个关于数据预处理的问题。起初，我们并不熟悉这方面的知识，但我们知道如何利用搜索引擎和社区资源来寻找解决方案。通过在 Stack Overflow 上搜索相关关键词，我们找到了一个类似问题的解答。在这个解答中，推荐了一个用于数据预处理的 Python 库——pandas。我们立即学习了这个库的使用方法，并成功地应用到了我们的项目中。

其次，我们还深入研究了基于评分卡模型的机器学习原理，并成功地实现了相应的评分卡模型。通过在 GitHub 上找到了一个关于评分卡模型的开源项目，通过阅读代码和文档，我们了解了评分卡模型的实现方式。接着，我们参考了该

项目的设计思路，并使用 Python 实现了自己的评分卡模型，可以对贷款数据进行有效分析和建模。

通过这个项目，我们不仅提升了自己的技术能力，还锻炼了独立学习和资源收集的能力。这些能力的提升使我们更有信心应对未来的挑战，并在实际工作中发挥出更大的潜力。

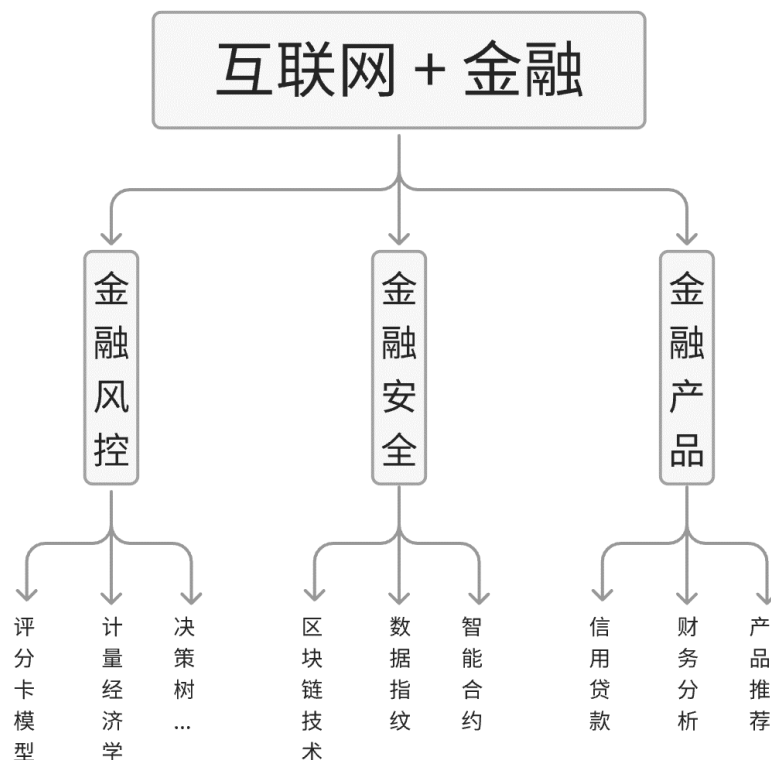


图 62：互联网+金融产品调研与融合创新点

此外，我们通过深入研究网络资料，对互联网+金融的项目特点有了更深入的理解。我们初步地学习了互联网金融的技术创新和风险控制等各方面的知识，并掌握了相应的行业标准和规范。这些知识不仅有助于我们更好地理解互联网金融项目的背景和业务需求，还能够使我们提出更合理的设计方案并完成相应的开发工作。其中，数据指纹的创新点即是对互联网+进入项目中“区块链与智慧金融”的一个底层实现和设计。通过提取数据中的特征，生成数据的“指纹”，从而实现对数据的识别和追踪。我们小组将数据指纹技术与金融数据安全相结合，有效保护了用户的数据安全，防止了数据泄露和滥用。这与“智慧金融”体系的构建（图 62）不谋而合。

4.4 项目管理与产品思维

在贷款系统的开发过程中，项目管理的科学方法和先进工具的运用，为我们提供了强有力的组织和协调机制，确保了项目的按时交付并维持了高标准的质量水平。我们坚持采用敏捷开发的理念，通过迭代开发的模式，逐步丰富系统的功能细节，同时持续进行功能优化和改善。在每个迭代循环开始时，我们都会制定详尽的工作计划，清晰地划分目标和任务，并且在整个开发周期内，持续进行进度的跟踪和效果的评估。这种敏捷的方法论使我们能够迅速适应不断更新的需求的变化，及时识别并解决可能出现的问题和风险，从而最大程度地降低了项目失败的风险。



图 63：敏捷开发板样例

在产品开发的思维模式上，我们始终将用户体验置于核心位置，深入进行需求分析。在开发过程中，我们进行了全面的市场调研，深入挖掘并理解用户在使用现有产品时遇到的主要问题和挑战。我们运用了开发版图、风险管理矩阵、用户行为模型等先进的产品理论知识，精心设计了与用户需求紧密相连的操作流程和用户指导，以确保用户在使用我们的贷款系统时能够享受到流畅和直观的体

验。

通过不断地实践和反思，我们不仅在项目管理和产品思维上积累了宝贵的经验，而且也在不断地学习和优化我们的方法论。我们相信，在未来的软件开发项目中，我们将能够站在更高的视角，全面审视整个项目的进展，达到“横看成岭侧成峰”的境界，即从不同的角度审视问题，发现并把握项目的关键点和潜在的增长点。这种全局性的思考和战略性的规划，将使我们在金融科技的浪潮中，始终保持领先的步伐，为用户创造更大的价值。

4.5 先进技术与融合创新

在智能时代的浪潮中，我们小组在贷款项目的开发中，积极拥抱智能化和复合型技术创新的潮流。我们深知，要在未来的发展中保持竞争力，就必须不断地学习、借鉴和融合前沿技术，以提升我们的复合型知识水平，为未来的融合发展提供坚实的知识支撑。

在这一理念的引领下，我们小组在项目中引入了人工智能大模型技术，将其作为智能客服的核心，为用户提供详尽的软件操作指引。我们采用了科大讯飞提供的开源语言大模型，通过其强大的自然语言处理能力，为用户提供清晰、准确的操作指导，从而显著提升了用户体验和项目的工作效率。在这一过程中，我们不仅初步掌握了大模型技术的基本原理，如语法树分词等关键技术，还深入学习了 `prompt` 的撰写技巧，这使我们能够根据用户的具体需求，设计出更加精准的提示语，引导用户顺利完成操作，从而提高智能客服的准确性和有效性。

同时，我们还积极探索了 `websocket` 的通信技术，这种新兴的网络通信协议允许我们实现客户端和服务器之间更高效的实时协作。通过研究和应用 `websocket` 技术，我们成功解决了智能客服系统在实时互动方面的挑战，为用户提供了更加流畅和即时的服务体验。此外，我们还掌握了流式输出技术，这使得智能客服系统的回答能够以流式的形式展示给用户，极大地增强了信息的可读性和易用性。

在人工智能大模型技术的引入和应用过程中，我们不仅实现了智慧赋能与技术的交叉融合，更深化了对相关技术的理解和掌握，为项目的成功实施提供了强有力的技术支持。

除此之外，我们还巧妙地将计量经济学的知识应用于贷款项目的坏账风险预测。通过这一策略的核心在于运用先进的计量经济学方法和模型，对贷款项目的潜

在坏账风险进行精准预测。通过这种方法，我们不仅能够提前发现潜在的风险点，还能够及时采取有效的风险控制措施，从而确保整个项目的安全性和稳定性。

在这一过程中，我们的团队成员基于《计量经济学》课程所学的相关理论进行深入研究，具体包括但不限于时间序列分析、回归模型、Eviews 软件操作等。这些理论为我们提供了强大的分析工具。我们不仅掌握了这些复杂的建模和统计分析方法，还通过不断的实践和验证，将这些理论知识转化为了实际的风险预测能力。在项目实施过程中，我们深入学习了计量经济学的相关理论，掌握了复杂的建模和统计分析方法，并成功地将这些知识应用于实际的风险预测中，为贷款项目的风险管理提供了有力的支持。通过建立这套补充辅助性的风险监控体系，有利于银行能够及时响应市场变化，计提坏账准备，优化风险控制策略，确保贷款项目的稳健运行。

通过这些跨学科的学习和应用，我们小组不仅提升了项目的技术水平，也为团队成员的个人成长和专业发展提供了宝贵的机会。我们相信，这种复合型的知识结构和创新精神，将使我们在未来的金融科技领域中占据一定优势。提升了我们的融合创新能力。

第五章 分工协作与交流情况

5.1 分工协作与计划完成情况

在完成综合项目 III 的过程中，我们进行了细致的分工和协作，以确保项目能够高效、顺利地进行。以下是对本学期的项目综合分工情况：

表 20：综设 III 项目分工表

任务类型	任务点	负责人	协同人员
评分卡	数据源分析	王宇轩	
	评分回归建立	朱子铮	
	评分回归评估与调参	朱子铮	周雨辰
	坏账模型预测	郭校滢	朱子铮、蒋谨谦
前后端	二期基础前端功能增补	蒋谨谦	
	二期基础后端功能增补	郭校滢、周雨辰	
	数据指纹	蒋谨谦、郭校滢	王宇轩
	数据大屏	蒋谨谦、王宇轩	
	大模型调用	郭校滢、周雨辰	朱子铮
测试	测试样例编写	王宇轩	周雨辰
	人工测试	蒋谨谦、郭校滢、王宇轩	
	自动化测试	周雨辰、朱子铮	
报告	报告文段撰写	郭校滢、蒋谨谦	王宇轩
	报告绘图制表	郭校滢、蒋谨谦	周雨辰
	PPT 制作	朱子铮、郭校滢	蒋谨谦

项目团队在完成项目过程中，均衡分配实现了工作内容的均衡分配和有效协作。通过明确的任务分配和协作管理措施，团队成员能够清楚地了解自己的职

第五章 分工协作与交流情况

责，并在协作平台上实时交流与协调。每个人都全面参与了项目，为项目的成功贡献了自己的力量。在任务分配环节，小组将复杂的工程任务划分为不同的阶段和子任务，并根据每个成员的能力和和经验进行合理分配，通过飞书任务进行下达分发。这一做法不仅提高了工作效率，还确保了工作质量。团队成员之间保持了良好的沟通渠道，随时分享进展情况、解决问题，确保任务按计划完成。

在任务分配的执行过程中，朱子铮同学作为组长，充分发挥了监督和协调作用。他定期与团队成员沟通，了解任务进展和问题，并提供必要的指导和支持，确保任务分配的均衡性和高效性。同时，负责解决团队成员之间的合作问题，保证团队的整体协同运作。小组总体达到了预期的开发目标。



图 64：项目开发总体进程图

通过任务分配、协作平台和领导监督的有机结合，团队成员能够高效协同工作，充分发挥各自的优势，减少重复工作，提高工作效率和成果质量。团队共同努力，按计划完成项目，并实现了预期的目标。

5.2 协同交流情况

项目的协同是综设 III 开发的核心，良好的协同可以帮助我们加快开发进度和修复进展。对于代码的托管部署，我们采用 Gitee 和 Github 结合的方式。Gitee 作为代码跟踪编辑的主要站点，通过灵活运用分支开发和代码合并的功能，可以独

立解决各种常见的版本控制问题，如合并冲突或回退错误提交；Github 作为前端页面部署的平台，记录了各页面的变动情况，可以按需回退。

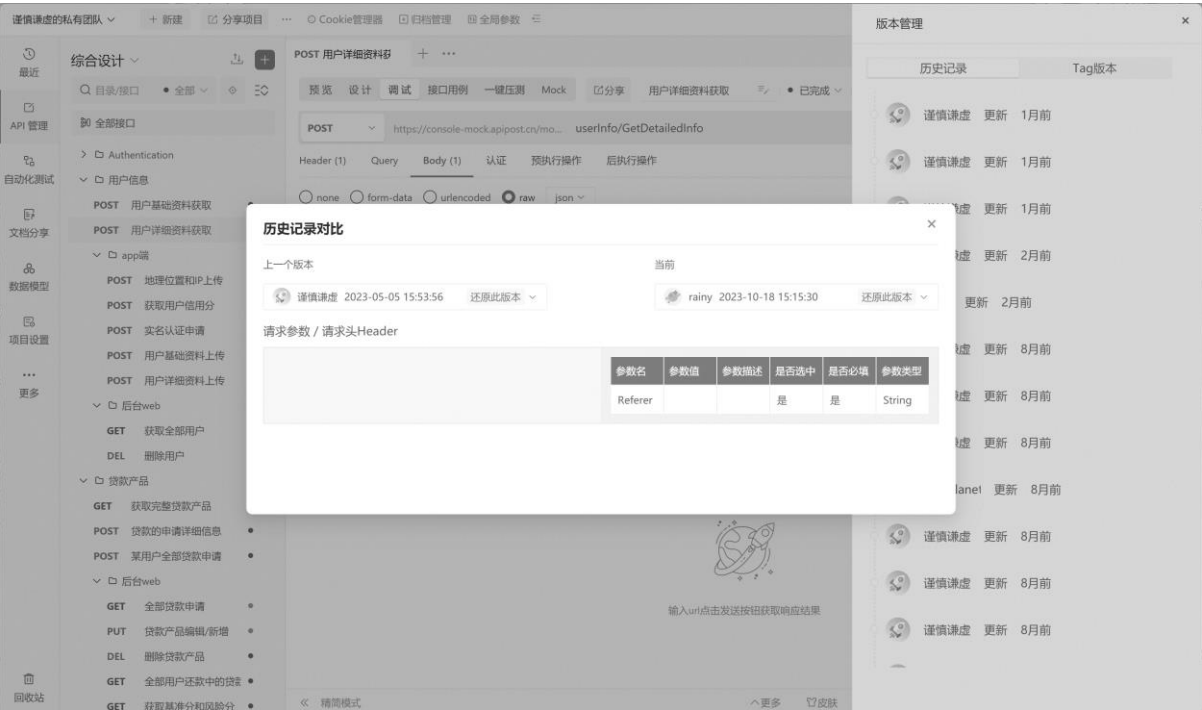


图 65：利用 Apipost 进行接口版本控制

此外，Apipost 和飞书等协作平台也发挥了关键作用，通过 Apipost 实现了接口的线上对接，进一步提升了前后端的协同效率——前端开发者快速理解后端接口，同时也能让后端开发者及时了解前端的需求，从而更好地进行协同开发。Apipost 还提供了实时协作功能，团队成员可以在平台上共同编辑和调试接口，及时解决问题，提高开发效率。同时，Apipost 的文档功能可以帮助团队记录和管理接口文档，确保接口的一致性和可维护性。通过 Apipost 的协同交流，我们的团队在开发过程中能够更好地配合，快速响应需求变更，提高项目的质量和进度。这使得 Apipost 成为我们项目中不可或缺的协同工具，为项目的成功提供了有力支持。

第五章 分工协作与交流情况



图 66：gitee 提出的 issue

在遇到学习的问题时，小组同学选择使用飞书平台进行知识管理，这样能够有效地帮助团队成员更好地进行协同交流。



图 67：小组成员通过飞书进行协作

团队成员可以在平台上共享文档、讨论问题，并及时解决工作中的疑问和困难，促进团队合作和知识共享。同时，在后期，我们基于飞书的多维表格搭建了功能测试反馈表。通过这个表格，团队成员可以实时地记录和反馈测试过程中发现的问题，有助于及时修复 bug 和提高软件质量。飞书的多维表格功能使得我们

信软学院进阶式挑战性综合项目 III 报告

可以根据不同的需求自定义表格内容，方便团队成员进行数据录入和分析。及时跟进 bug 修复情况。

功能测试脚本

3 已经保存到云端

功能测试脚本

功能问题修复状态

+

>> 修复状态

字段配置

视图配置

筛选

1 分组

排序

行高

保存配置

生成表单

评论

☐	问题细节	状态	测试端	问题类型	优先级	测试时间	提出人	跟进人
已验收 7 条记录								
1	身份验证UI展示超出手机页面	已验收	用户端	前端	P2	2023/09/07	朱子铮	蒋谨谦
2	后端接口（用户信息上传）接收不到传参	已验收	管理端 用户端	后端	P0	2023/09/27	郭校深	周雨辰
3	18岁以下不能申请贷款	已验收	用户端	后端 前端	P2	2023/10/04	王宇轩	蒋谨谦 郭校深
4	评分卡新增参数hasCar的分数没有传给后端	已验收	管理端	模型	P1	2023/10/24	朱子铮	朱子铮 周雨辰
5	指纹验证数据生成前端后端不一致	已验收	用户端 管理端	后端 前端	P2	2023/11/10	蒋谨谦	郭校深
6	大模型流式输出不对，最后一个字会被吞掉	已验收	用户端	前端	P1	2023/11/16	朱子铮	蒋谨谦
7	数据大屏无法监测全屏（快捷键方式）	已验收	管理端	前端	P2	2023/11/29	周雨辰	蒋谨谦 王宇轩

图 68：飞书 Bug 修复表

综上，我们小组在分工协作方面做得较为高效。通过多种工具和平台的使用，我们实现了高效的协同交流和合作，加速了项目的开发进度和修复进展。这种良好的协作机制不仅提高了团队的工作效率，也促进了知识的共享和团队合作。

参考文献

- [1] 中国青年网.大学生超前消费调查:近九成用过分期付款,主要用于饮食和购物
http://edu.youth.cn/jyzx/jyxw/201911/t20191108_12114118.htm
- [2] 中国人民银行. 2021 年金融机构贷款投向统计报告[S].人民日报,2022.1.
- [3] 杨青. 个人住房抵押贷款的市场风险研究[D]. 北京:对外经济贸易大学,2011.
DOI:10.7666/d.y1993032.
- [4] 刘帅祺. 商业银行互联网贷款信用风险评估及客户特征发现[D].北京科技大学,2023.DOI:10.26945/d.cnki.gbjku.2023.000344.
- [5] 苏丹. 中国商业银行房地产贷款信用风险管理研究——基于 Logistic 模型的实证分析[D]. 四川:西南财经大学,2015.
- [6] 杨保安,徐晶,查秋野. 基于人工智能贷款风险分类的系统设计[J]. 管理工程学报,2003,17(2):45-49. DOI:10.3969/j.issn.1004-6062.2003.02.012.
- [7] 邢泽东. 基于标准评分卡模型的内蒙古自治区小额贷款公司综合评价研究[D]. 内蒙古财经大学,2021.
- [8] Kumar V, Saheb S S, Ghayas A, et al. AI-Based Hybrid Models for Predicting Loan Risk in the Banking Sector[J]. Big Data Mining and Analytics, 2023, 6(4): 478-490.
- [9] 林治乾,严志辉. 信用评分卡在中小企业贷款业务中的应用[J]. 银行家,2013(2):77-79.
DOI:10.3969/j.issn.1671-1238.2013.02.018.
- [10] 吴敏. 申请评分卡在小额贷款公司运营管理中的应用——以 ABC 公司对授薪客户分析为例[D]. 上海:上海交通大学,2014.
- [11] Dietrich J R, Kaplan R S. Empirical analysis of the commercial loan classification decision[J]. Accounting Review, 1982: 18-38.
- [12] 吴欣雅,洪子绚,曾莹,等. 区块链在银行贷款业务的应用与探索[J]. 全国流通经济,2022(11):139-141. DOI:10.3969/j.issn.1009-5292.2022.11.041.
- [13] 何源. Z 银行支付结算业务数字化转型问题研究[D]. 河南:郑州大学,2020.
- [14] 沈建国,沈佳坤,杨赐. 互联网金融对商业银行贷款影响的实证研究--以蚂蚁搬家为例[J]. 中国市场,2016(37):88-89. DOI:10.13939/j.cnki.zgsc.2016.37.088.

致谢

本报告的工作是在我们的指导教师傅翀老师的悉心指导下完成的，在整个项目的研究与实施过程中，傅老师始终给予我们悉心的辅导和宝贵的意见，对此我们深表感激。傅老师的专业建议和耐心指导是我们能够顺利推进项目、克服种种挑战的关键所在。在项目实施期间，傅老师不仅提供了许多建设性的意见，还定期组织会议，跟进小组成果，并在我们遇到难题时伸出援手。他的细心关照、热情支持与精神鼓舞，为我们的研究工作注入了强大的动力。

此外，我们还要感谢每一位在项目实施过程中给予帮助的老师 and 同学们。大家齐心协力、攻克难关，为项目的圆满完成贡献了自己的智慧和汗水。在讨论、建言献策以及承担责任等方面，每个人都展现出了极大的热情 and 责任感，为我们的项目增添了无限的活力与创意。

最后，我们对小组内每一位成员的辛勤付出表示衷心的感谢。面对项目中的种种挑战，每一位成员都展现出了非凡的毅力与决心，充分发挥了自己的潜能，为项目的成功做出了不可或缺贡献。大家之间的理解与支持，让我们的团队更加团结协作，共同向着目标迈进。

随着本报告的完成，我们的综合设计项目也画上了圆满的句号。这一路走来，我们收获了知识和技能，积累了宝贵的经验。虽然综合设计的学习旅程已经结束，但我们的探索精神和创新意识将继续前行，未来我们将以更加成熟和自信的姿态，迎接新的挑战。